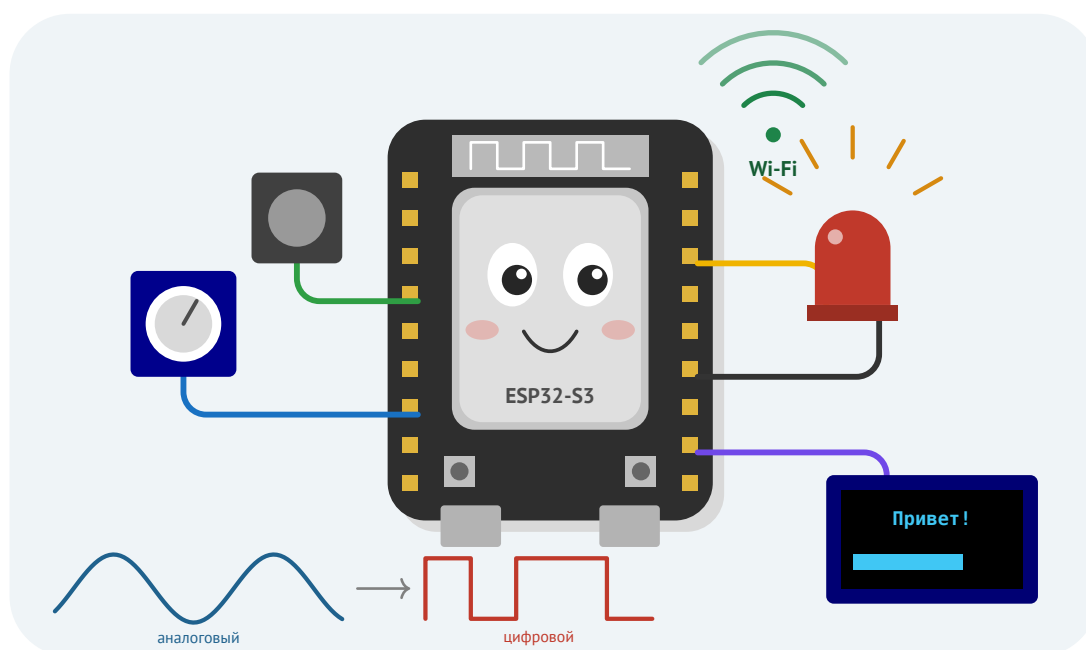


ESP32-S3

Первые шаги в электронике и программировании микроконтроллеров



1. Знакомство с платой
2. Сигналы: аналоговые и цифровые
3. Выводы платы: что есть что
4. Arduino IDE и первая программа
5. Светодиод: закон Ома и мигание
6. ШИМ: яркость и частота
7. Радио: Wi-Fi и Bluetooth
8. Веб-сервер: HTTP, HTML и CSS
9. Последовательный порт
10. Кнопки и прерывания
11. АЦП: измеряем напряжение
12. Шина I²C: экран
13. FreeRTOS: несколько дел сразу
14. Сон и энергосбережение

Корягин Сергей Андреевич
Полякова Полина Алексеевна

педагоги дополнительного образования
МОБУ «СОШ «ЦО «Кудрово»

Все примеры — для Arduino-ESP32 версии 3.x и отладочной платы ESP32-S3-DevKitC-1 (или совместимой).

Содержание

1 Знакомство с платой	4
1.1 Смотрим на плату	4
1.2 Что такое микроконтроллер	5
1.3 Что умеет ESP32-S3	5
1.4 Выводы: первое знакомство	6
2 Сигналы: аналоговые и цифровые	7
2.1 Что такое сигнал	7
2.2 Аналоговый сигнал	7
2.3 Цифровой сигнал	8
2.4 Логические уровни: где кончается ноль и начинается единица	9
2.5 Почему цифровой сигнал надёжнее	9
2.6 Двоичные числа	10
2.7 Из аналогового в цифровой: АЦП	10
2.7.1 Как часто нужно измерять?	11
2.8 Из цифрового в аналоговый: ЦАП и ШИМ	12
2.9 Путь сигнала внутри устройства	12
3 Выводы платы: что есть что	14
3.1 Питание и земля	14
3.2 Цифровые выводы GPIO	15
3.3 Аналоговые входы (АЦП)	16
3.4 Сенсорные входы	16
3.5 Выводы для связи и особые выводы	17
3.6 Выводы, которые мы будем использовать	17
4 Arduino IDE и первая программа	18
4.1 Как программа попадает в плату	18
4.2 Установка Arduino IDE	18
4.3 Из чего состоит скетч	19
4.4 Первая программа	19
4.4.1 Если прошивка не начинается	20
4.5 Учимся программировать: как записывается программа	20
4.6 Учимся программировать: переменные	21
5 Светодиод: закон Ома и мигание	24
5.1 Светодиод	24
5.2 Зачем нужен резистор: закон Ома	24
5.2.1 Почему светодиод сгорает без резистора	25
5.2.2 Рассчитываем резистор	25
5.2.3 Как узнать сопротивление резистора	26
5.3 Макетная плата и два вида схем	27
5.4 Подключаем светодиод	28
5.5 Программа Blink: мигаем светодиодом	29
5.5.1 Учимся программировать: константы и вызов функций	30
5.5.2 Что видит глаз	30
5.6 Мигание без delay()	31
5.7 Учимся программировать: условие if	31
5.8 Практика: бегущий огонь	33

5.9	Учимся программировать: массивы и цикл for	34
5.10	Встроенный RGB-светодиод	36
6	ШИМ: яркость светодиода и частота	38
6.1	Можно ли светить вполсилы?	38
6.2	Опыт: мигаем всё быстрее	38
6.3	Инерция зрения	39
6.4	ШИМ — широтно-импульсная модуляция	40
6.5	ШИМ в ESP32-S3: модуль LEDC	40
6.6	Светодиод с линейно меняющейся яркостью	41
6.7	Почему «линейно» выглядит нелинейно	41
6.8	Учимся программировать: свои функции	42
6.9	Какую частоту ШИМ выбрать	43
6.10	ШИМ и звук	44
6.11	Практика: плавная волна	45
7	Радио: Wi-Fi и Bluetooth	47
7.1	Что такое радиоволны	47
7.2	Wi-Fi и его каналы	48
7.3	Bluetooth	49
7.4	Два режима Wi-Fi: клиент и точка доступа	50
7.4.1	Пример 1: плата — клиент	50
7.4.2	Учимся программировать: цикл while и логическое «и»	50
7.4.3	Пример 2: плата — точка доступа	51
7.5	MAC-адрес и IP-адрес	52
7.5.1	Кто раздаёт IP-адреса	53
7.5.2	Как задать IP, MAC и имя в программе	53
7.6	Имя вместо адреса: mDNS	54
8	Веб-сервер: HTTP, HTML и CSS	56
8.1	Клиент и сервер	56
8.2	Из чего состоит адрес	57
8.3	Методы GET и POST	57
8.3.1	Пример: управление через GET	58
8.3.2	Пример: отправка данных через POST	60
8.4	HTML: из чего сделана страница	61
8.5	CSS: делаем красиво	62
8.6	Практика: панель управления светодиодом	62
9	Последовательный порт: разговор с компьютером	67
9.1	Как передать букву по одному проводу	67
9.2	Serial и Serial Monitor	67
9.3	Практика: светодиод слушается команд	68
9.4	Учимся программировать: строки и цепочка условий	69
9.5	Практика: игра «Угадай число»	70
9.6	Второй UART: связь с другими устройствами	72
10	Цифровой вход: кнопки и прерывания	74
10.1	Как устроена кнопка	74
10.2	Проблема «висящего» входа	74
10.3	Схема подключения	75
10.4	Читаем кнопку	76
10.5	Дребезг контактов	77

10.6	Учимся программировать: логические операции и static	78
10.7	Практика: кнопка-переключатель яркости	78
10.8	Прерывания	79
11	АЦП: измеряем аналоговый сигнал	81
11.1	Как работает АЦП ESP32-S3	81
11.2	Потенциометр — «крутилка»	82
11.3	Шум и усреднение	84
11.4	Сервопривод	84
11.5	Практика: ручка поворачивает сервопривод	85
11.6	Измеряем напряжение больше 3,3 В	86
12	Шина I²C: экран	88
12.1	Два провода на всех	88
12.2	Как выглядит разговор по I ² C	89
12.3	Подключаем экран	89
12.4	Сканер шины	90
12.5	Первая надпись на экране	91
12.6	Практика: вольтметр с экраном	92
13	FreeRTOS: несколько дел одновременно	95
13.1	Зачем нужна операционная система	95
13.2	Жизнь задачи	96
13.3	Первые задачи	96
13.4	Очередь: передаём данные между задачами	97
13.5	Практика: мьютекс — «ключ» от общего ресурса	97
14	Сон и энергосбережение	100
14.1	Зачем устройству спать	100
14.2	Сколько проработает батарейка	100
14.3	Сон по будильнику-таймеру	101
14.4	Пробуждение кнопкой	102
14.5	Практика: датчик на батарейке	102
A	Справочник	104

Урок 1. Знакомство с платой

Цель	рассмотреть плату ESP32-S3-DevKitC-1, узнать, из каких частей она состоит и что такое микроконтроллер
Опирается на	ничего — это первый урок
Практика	находим на своей плате все детали, о которых говорится в уроке
Понадобится	плата ESP32-S3-DevKitC-1 (или похожая) и любопытство

1.1 Смотрим на плату

Возьми плату в руки и рассмотри её. Сначала кажется, что на ней очень много всего, но всё устроено вполне понятно: в центре — «мозг», по краям — «ножки» для подключения, снизу — разъёмы для связи с компьютером.

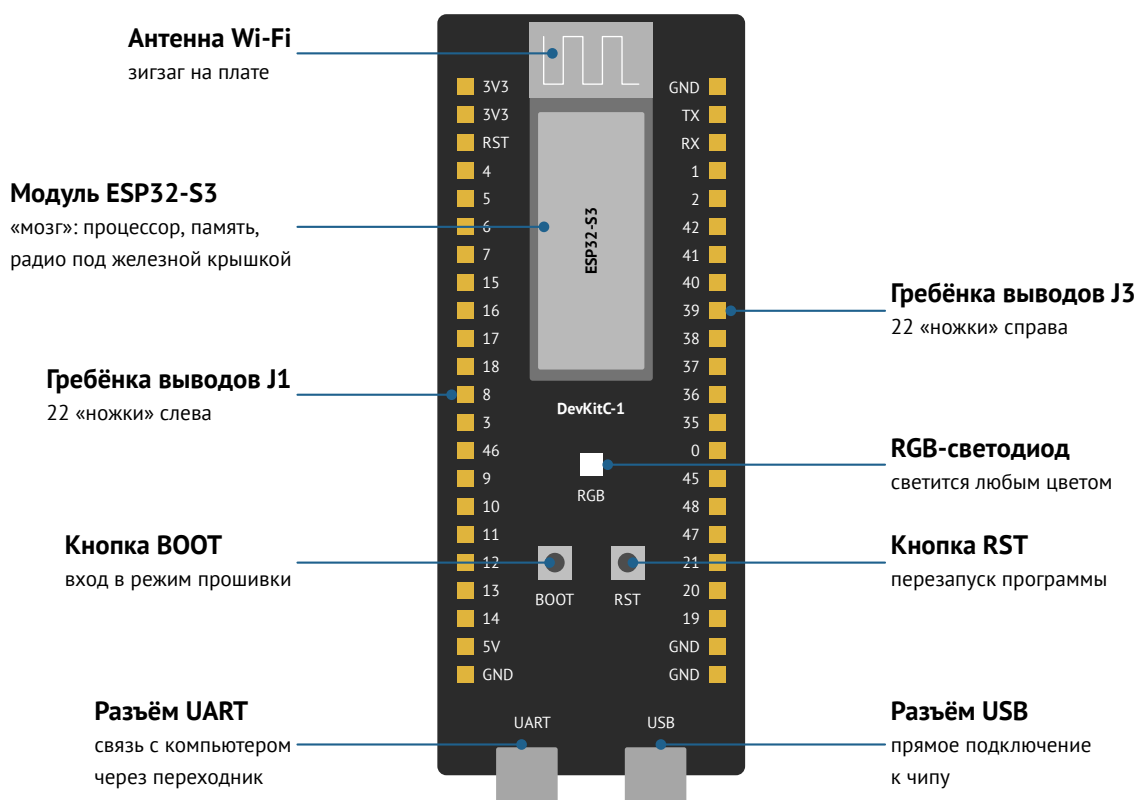


Рис. 1: Отладочная плата ESP32-S3-DevKitC-1, вид сверху. Цифры у выводов — их номера

- **Модуль** с надписью ESP32-S3 — главная часть. Под металлической крышкой спрятаны сам чип, память и детали для радиосвязи. Зигзаг на краю — антенна Wi-Fi, её нельзя закрывать металлом и руками.
- **Две гребёнки выводов** по бокам — через них к плате подключают светодиоды, кнопки, датчики и экраны. Выводы подписаны: цифрами (номера) или буквами (питание, земля).
- **Два разъёма USB** снизу — для питания и связи с компьютером. Подойдёт любой из них.
- **Две кнопки** — BOOT и RST (от англ. *reset* — сброс).

- **RGB-светодиод** — маленький квадратик, который умеет светиться любым цветом.

Совет

Отладочная плата создана для экспериментов: всё, что нужно для работы чипа (питание, связь, кнопки), уже распаяно. В готовом устройстве, например в умной розетке, стоит только сам модуль, без лишнего.

1.2 Что такое микроконтроллер

ESP32-S3 — это **микроконтроллер**: маленький компьютер, целиком помещённый в одну микросхему. В обычном компьютере процессор, память и видеокарта — отдельные детали, а здесь всё на одном кусочке кремния размером с ноготь.

Микроконтроллер работает как человек, который выполняет инструкции: «органы чувств» получают сигналы из окружающего мира, «мозг» принимает решение, а «руки» что-то делают.



Рис. 2: Любое устройство с микроконтроллером: входы → обработка → выходы

Микроконтроллеры окружают нас повсюду: они есть в стиральной машине, микроволновке, пульте от телевизора, игрушках, автомобиле (там их десятки!). Каждый выполняет одну программу, которую в него однажды записали. В этом курсе программу будем писать мы сами.

1.3 Что умеет ESP32-S3

ESP32-S3 выпускает китайская компания Espressif. Это мощный микроконтроллер, который особенно любят за встроенный Wi-Fi:

- **два ядра** с частотой до 240 миллионов операций в секунду — два «мозга», которые могут работать одновременно;
- **512 КБ оперативной памяти** (для переменных) и 8–16 МБ памяти для программы;
- **Wi-Fi и Bluetooth** — можно подключиться к интернету или к телефону;
- **45 выводов общего назначения** (GPIO), АЦП для измерения напряжения, сенсорные входы, USB.

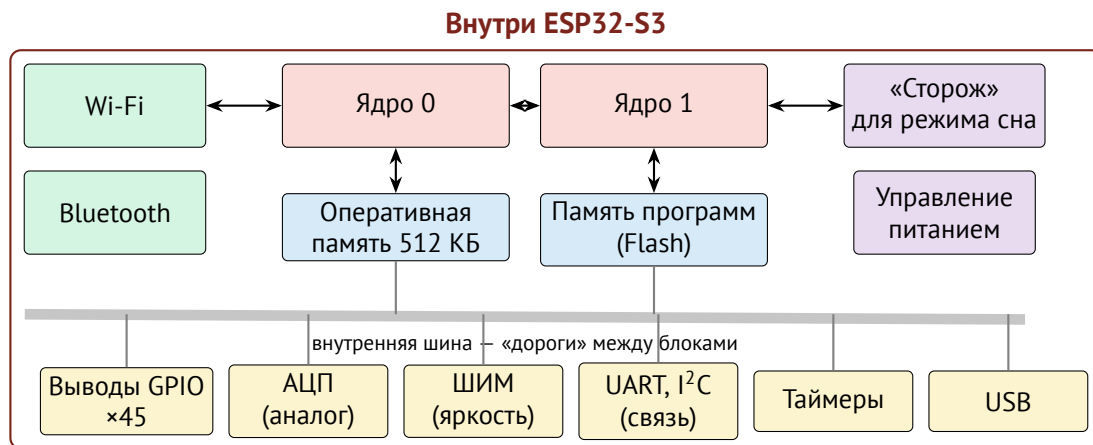


Рис. 3: Упрощённая схема того, что находится внутри чипа. Каждый блок нижнего ряда мы задействуем в одном из уроков

Это интересно

На металлической крышке модуля написано его полное название, например ESP32-S3-WROOM-1-N16R8. Буква N и число после неё — объём памяти для программ (16 МБ), буква R — объём дополнительной оперативной памяти PSRAM (8 МБ). Посмотри, какой модуль стоит на твоей плате.

1.4 Выводы: первое знакомство

Самое интересное для нас — выводы по краям платы. Их 44, и почти у каждого своя подпись. Присмотрись к ним и к рисунку выше:

- у части выводов написаны **буквы**: 3V3, 5V, GND, RST — это питание, «земля» и сброс;
- у остальных — **номера**: 1, 2, 4, 5... Это выводы общего назначения (GPIO, от англ. *General Purpose Input/Output* — вход-выход общего назначения).

Если открыть документацию на плату, окажется, что возле номеров есть дополнительные подписи. Одни выводы названы просто GPIO, у других приписано ADC (аналого-цифровой преобразователь), у третьих — TOUCH (сенсорный вход). Значит, выводы умеют работать с *разными* сигналами: одни понимают только «есть напряжение / нет напряжения», а другие ещё и умеют измерять, *сколько* его.

Чтобы разобраться, чем эти сигналы отличаются, нужно познакомиться с двумя важнейшими понятиями электроники — **аналоговым** и **цифровым** сигналом. Этим и займёмся в следующем уроке, а потом вернёмся к выводам и узнаем, что умеет каждый из них.

Итоги урока

- Отладочная плата — это модуль ESP32-S3 и всё, что нужно для экспериментов: USB, кнопки, светодиод, выводы.
- Микроконтроллер — маленький компьютер в одной микросхеме: получает сигналы, обрабатывает, управляет.
- Выводы бывают с буквенными подписями (питание, земля) и с номерами (GPIO). Разные GPIO умеют разное.

Проверь себя

1. Найди на своей плате антенну, обе кнопки и RGB-светодиод.
2. Чем микроконтроллер отличается от обычного компьютера?
3. Назови три прибора у тебя дома, в которых наверняка есть микроконтроллер.

Урок 2. Сигналы: аналоговые и цифровые

Цель	понять, что такое сигнал, чем аналоговый сигнал отличается от цифрового и как один превращается в другой
Опирается на	урок 1: мы видели на плате выводы с подписями ADC и GPIO
Практика	учимся читать графики сигналов и переводить числа в двоичную систему
Понадобится	карандаш, бумага и калькулятор — паяльник сегодня не нужен

В прошлом уроке мы заметили, что у выводов платы разные подписи: одни умеют работать только с «включено/выключено», другие — ещё и измерять напряжение (ADC). Чтобы понять, в чём разница, разберёмся, с чем вообще работает микроконтроллер. Он не видит, не слышит и не чувствует — он умеет только измерять **напряжение** на своих ножках и выдавать напряжение сам. Всё, что происходит вокруг — свет, звук, нажатие кнопки, — должно превратиться в напряжение, то есть в **электрический сигнал**.

2.1 Что такое сигнал

Сигнал — это величина, которая меняется и этим что-то сообщает. Светофор сигнализирует цветом, учитель — голосом, градусник — высотой столбика. В электронике сигнал — это почти всегда напряжение, которое меняется со временем.

Сигналы удобно рисовать графиками. По горизонтали откладываем **время** t , по вертикали — **напряжение** U . Каждая точка линии отвечает на вопрос: «какое напряжение было в этот момент?»

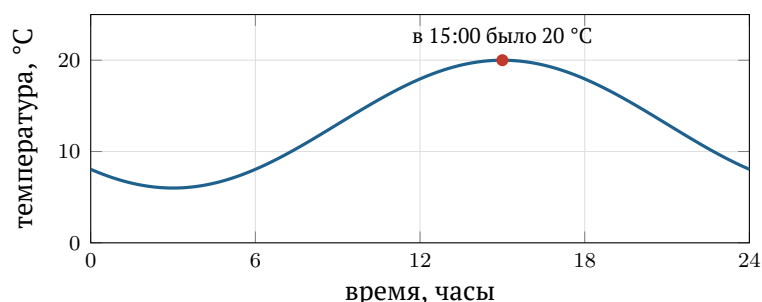


Рис. 4: Температура на улице в течение суток. Это тоже сигнал: величина, которая меняется со временем

2.2 Аналоговый сигнал

Аналоговый сигнал меняется *плавно* и может принимать *любое* значение: не только 1 или 2 вольта, но и 1,5, и 1,537, и 1,53712... Как температура: она не прыгает с 15 °C сразу на 16 °C, а проходит все промежуточные значения.

Представь горку и лестницу. По горке можно съехать и остановиться на любой высоте — это аналоговый сигнал. По лестнице можно стоять только на ступеньках — это цифровой сигнал, о нём чуть ниже.

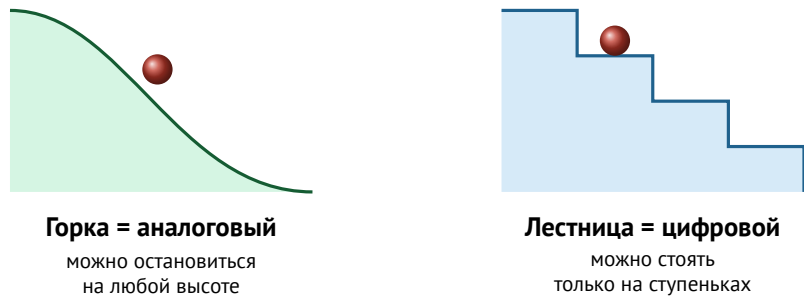


Рис. 5: Аналоговый и цифровой сигнал — как горка и лестница

Самый известный аналоговый сигнал — **синусоида**. Такой сигнал выдаёт микрофон, когда рядом звучит чистая нота. У неё есть два главных параметра:

- **амплитуда** U_m — насколько высоко поднимается волна (у звука — громкость);
- **период** T — через сколько времени волна повторяется. Сколько раз волна повторяется за секунду, называют **частотой**: $f = 1/T$. Частоту измеряют в герцах (Гц): 1 Гц = одно повторение в секунду.

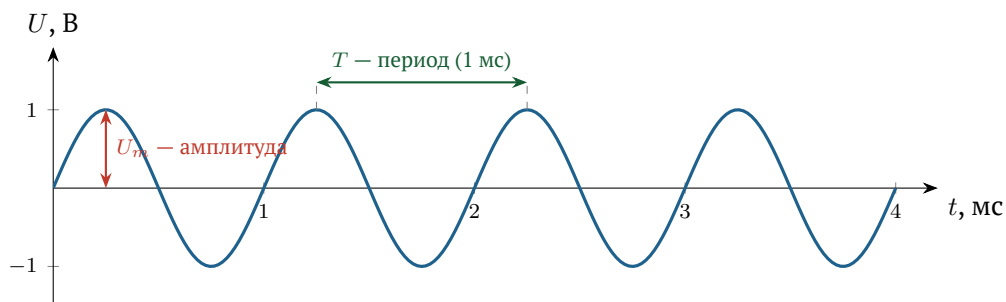


Рис. 6: Синусоида. Период 1 мс значит, что за секунду волна повторяется 1000 раз: частота $f = 1000$ Гц = 1 кГц. Такой звук похож на писк старого телефона

Это интересно

Человек слышит звуки с частотой примерно от 20 Гц (очень низкий гул) до 20 000 Гц (тонкий писк). С возрастом верхняя граница снижается, поэтому некоторые высокие звуки слышат только дети и подростки.

2.3 Цифровой сигнал

Цифровой сигнал может принимать только несколько заранее договорённых значений. Чаще всего значений всего два:

- **0** — «нет», низкий уровень, LOW, напряжение около 0 В;
- **1** — «да», высокий уровень, HIGH, для ESP32-S3 это около 3,3 В.

Одна такая «ячейка» со значением 0 или 1 называется **бит**. Выключатель света — цифровой: свет либо горит, либо нет. А регулятор громкости у колонки — аналоговый.

Чтобы передать много информации, биты посылают по очереди, один за другим. Каждому биту отводится одинаковое время — *длительность бита*:

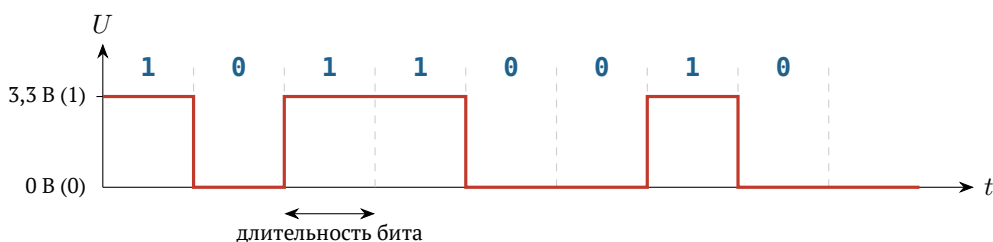


Рис. 7: Цифровой сигнал: восемь бит 10110010 передаются по одному проводу друг за другом

Аналоговые источники сигнала	Цифровые источники сигнала
микрофон (звук)	кнопка (нажата / не нажата)
фоторезистор (освещённость)	датчик движения (есть движение / нет)
термистор (температура)	геркон, концевой выключатель
потенциометр — «крутилка» (урок 11)	линия связи с компьютером (урок 9)

Таблица 1: Примеры аналоговых и цифровых сигналов, с которыми мы встретимся

2.4 Логические уровни: где кончается ноль и начинается единица

Настоящее напряжение никогда не бывает ровно 0 или ровно 3,3 В: провода длинные, детали неидеальные. Поэтому микросхема договаривается: всё, что *достаточно низко*, — это 0, всё, что *достаточно высоко*, — это 1. Посередине остаётся «серая зона», где микросхема может ошибиться. Хорошая схема в эту зону никогда не попадает.

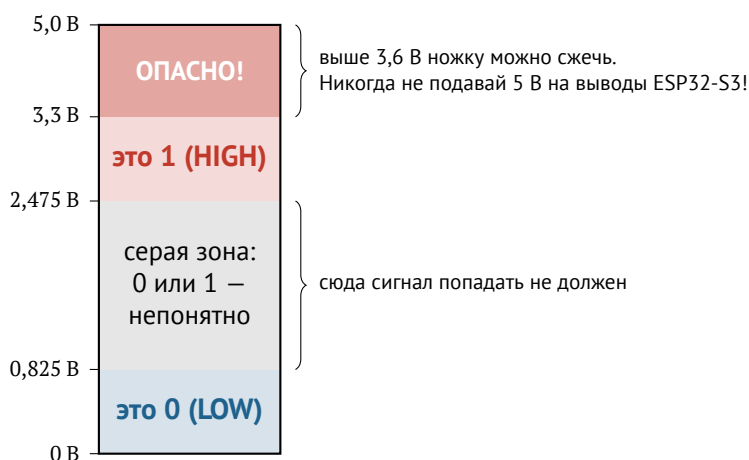


Рис. 8: Как ESP32-S3 (питание 3,3 В) понимает напряжение на входе. Граница нуля — четверть напряжения питания, граница единицы — три четверти

2.5 Почему цифровой сигнал надёжнее

По дороге к приёмнику любой сигнал обрастает **помехами**: рядом работает телефон, мигает лампа, моторчик создаёт искры. Помехи — это маленькие случайные добавки к напряжению.

Аналоговому сигналу от них не спастись: приёмник не знает, какая часть напряжения — полезный сигнал, а какая — помеха. А у цифрового сигнала всего два значения, и достаточно спросить: «напряжение выше середины или ниже?» Если помеха не слишком большая, ответ будет правильным, и сигнал восстановится *в точности*.

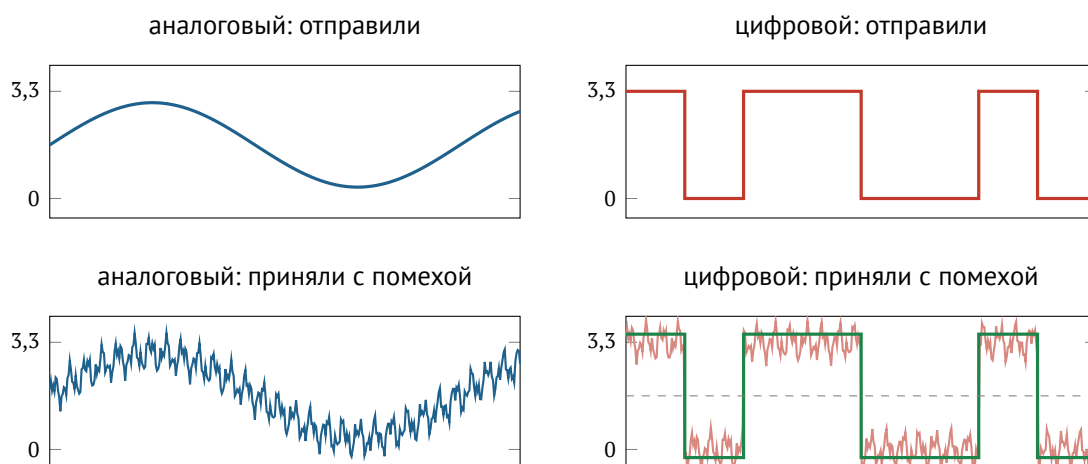


Рис. 9: Одна и та же помеха портит аналоговый сигнал навсегда, а цифровой легко восстановить: всё, что выше пунктирной линии, — единица, ниже — ноль (зелёная линия)

Именно поэтому музыку, фотографии и фильмы сегодня хранят и передают в цифровом виде: копия цифрового файла ничем не отличается от оригинала, сколько бы раз его ни копировали.

2.6 Двоичные числа

Один бит хранит всего два варианта: 0 или 1. Два бита — уже четыре варианта (00, 01, 10, 11), три бита — восемь. Каждый новый бит удваивает количество вариантов:

$$\text{количество вариантов} = 2^N, \quad N — \text{число бит.}$$

Восемь бит называют **байтом**. Байт хранит числа от 0 до 255. Чтобы перевести двоичное число в обычное, подпишем над каждым битом его «вес» — степень двойки — и сложим веса тех битов, где стоит единица:

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
вес бита:	128	64	32	16	8	4	2	1
бит:	1	0	1	1	0	0	1	0
	+128		+32	+16			+2	
								= 178

Рис. 10: Двоичное число 10110010 равно $128 + 32 + 16 + 2 = 178$

Число бит	Вариантов	Диапазон	Число бит	Вариантов	Диапазон
1	2	0–1	8 (байт)	256	0–255
2	4	0–3	10	1024	0–1023
3	8	0–7	12	4096	0–4095
4	16	0–15	16	65 536	0–65 535

Таблица 2: Сколько разных чисел помещается в N бит. Эти числа ещё не раз встретятся нам в курсе

2.7 Из аналогового в цифровой: АЦП

Процессор умеет работать только с числами. Чтобы он «услышал» микрофон или «узнал» положение ручки, аналоговый сигнал нужно превратить в числа. Этим занимается **АЦП** — *аналого-цифровой преобразователь*. Он делает два шага.

Шаг 1. Дискретизация — измеряем сигнал не всё время, а через одинаковые промежутки времени. Так работает кино: плавное движение снимают отдельными кадрами, 24 кадра в секунду, и глаз видит непрерывное движение. Сколько измерений делается за секунду, называют **частотой дискретизации**.

Шаг 2. Квантование — каждое измерение округляем до ближайшей «ступеньки». Это как измерять рост линейкой, на которой есть только сантиметровые деления: 152,7 см запишем как 153 см. Число ступенек зависит от числа бит АЦП: 3 бита — 8 ступенек, 12 бит — 4096 ступенек.

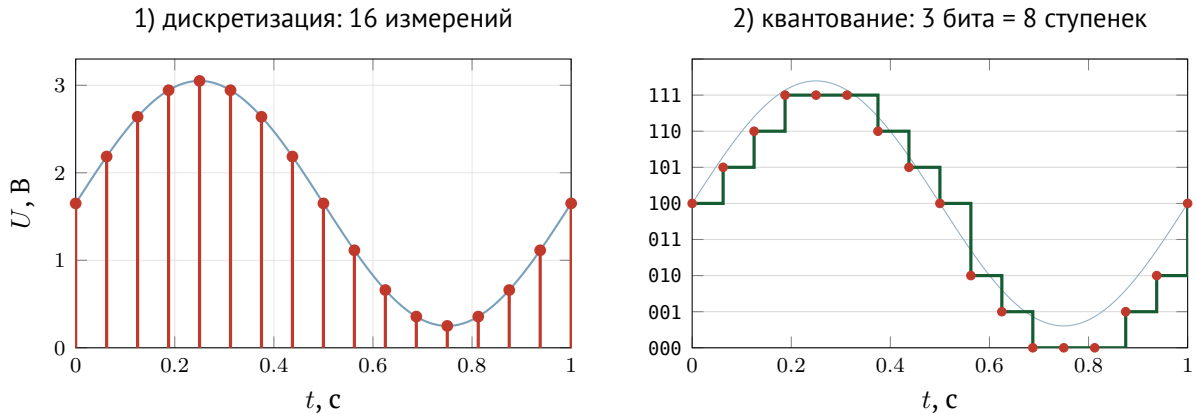


Рис. 11: Как АЦП превращает плавную волну в числа. Слева: измеряем 16 раз за период. Справа: каждое измерение округляем до одной из 8 ступенек и записываем номер ступеньки двоичным числом

Высота одной ступеньки называется **шагом квантования**:

$$q = \frac{U_{\max}}{2^N}, \quad \text{для 12-битного АЦП и 3,3 В: } q = \frac{3,3 \text{ В}}{4096} \approx 0,8 \text{ мВ.}$$

Чем больше бит, тем мельче ступеньки и тем точнее цифровая копия. В ESP32-S3 стоит 12-битный АЦП — с ним мы поработаем в уроке 11.

Как часто нужно измерять?

Если измерять слишком редко, можно получить совсем не тот сигнал. Посмотри на рисунок: быстрая волна (9 колебаний в секунду) измерена всего 10 раз в секунду, и точки измерений легли на медленную волну, которой на самом деле нет!

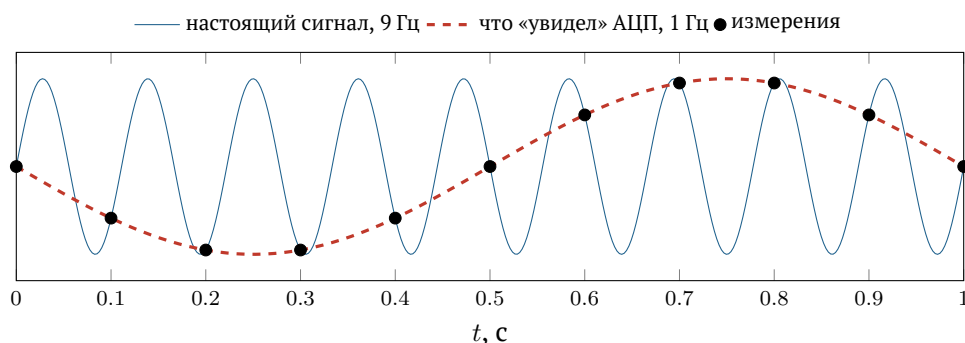


Рис. 12: Слишком редкие измерения обманывают. Поэтому в кино колёса машин иногда кажутся крутящимися медленно или даже назад

Правило (его называют **теоремой Котельникова**): измерять нужно *больше чем в два раза чаще*, чем самая высокая частота в сигнале. Человек слышит до 20 000 Гц, поэтому музыку записывают с частотой дискретизации 44 100 измерений в секунду.

2.8 Из цифрового в аналоговый: ЦАП и ШИМ

Обратная задача — из чисел получить плавное напряжение, например чтобы динамик заиграл музыку. Это делает **ЦАП** — цифро-аналоговый преобразователь.

В ESP32-S3 ЦАП нет, но есть хитрый приём — **ШИМ** (широтно-импульсная модуляция). Вывод очень быстро включается и выключается, тысячи раз в секунду. Если он половину времени включён, а половину выключен, то «в среднем» на нём половина напряжения. Светодиод при этом светит в половину яркости: глаз не успевает заметить мигание. Подробно ШИМ разберём в уроке 6.

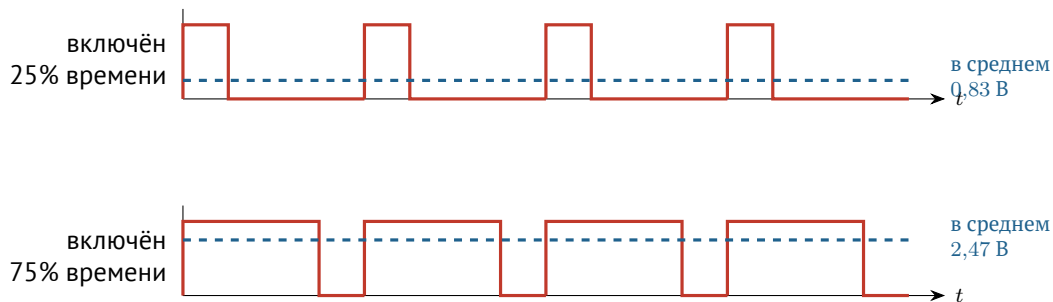


Рис. 13: ШИМ: чем дольше вывод включён, тем больше среднее напряжение (пунктир)

2.9 Путь сигнала внутри устройства

Почти любое «умное» устройство устроено одинаково: *аналоговый мир* → *цифровая обработка* → *аналоговый мир*. Вот, например, как работает цифровой диктофон:

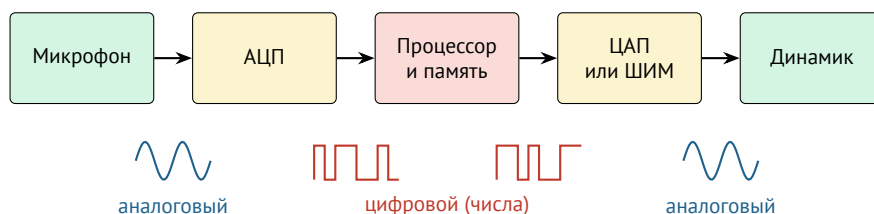


Рис. 14: Путь звука в цифровом диктофоне

В нашем курсе ESP32-S3 будет работать с сигналами обоих видов:

Что делаем	Какой сигнал	Урок
Включаем и выключаем светодиод	цифровой выход (0 или 1)	5
Разговариваем с компьютером	цифровой, биты друг за другом	9
Читаем кнопку	цифровой вход	10
Плавно меняем яркость	цифровой ШИМ, который «притворяется» аналоговым	6
Измеряем положение ручки	аналоговый вход → АЦП → число	11
Выводим картинку на экран	цифровой, два провода	12
Передаём данные по Wi-Fi	цифровые данные внутри радиоволны	7

Таблица 3: Какие сигналы встретятся в следующих уроках

Итоги урока

- Сигнал — это величина, которая меняется со временем и несёт информацию. В электронике это напряжение.
- Аналоговый сигнал плавный и принимает любые значения; цифровой — только договорённые (обычно 0 и 1).
- Цифровой сигнал устойчив к помехам: его легко восстановить по порогу.
- АЦП превращает аналоговый сигнал в числа: измеряет (дискретизация) и округляет (квантование).
- N бит дают 2^N вариантов: 8 бит — 256, 12 бит — 4096.
- ESP32-S3 понимает уровни 0 и 3,3 В. Напряжение 5 В для его выводов опасно.

Проверь себя

1. Какой сигнал у обычного выключателя света, а какой — у ручки громкости? Почему?
2. На вход ESP32-S3 подали 1,5 В. Это 0, 1 или «серая зона»?
3. Сколько вариантов можно закодировать 5 битами?
4. Переведи в обычное число двоичное 00101101.

Задания для самостоятельной работы

1. Найди дома по три аналоговых и три цифровых источника сигнала. Запиши, какую величину каждый из них передаёт.
2. Нарисуй график цифрового сигнала для буквы «А» (её код — 65). Сначала переведи 65 в двоичное число из 8 бит.
3. Посчитай шаг квантования 10-битного АЦП при напряжении 3,3 В. Во сколько раз он крупнее, чем у 12-битного?
4. Сколько измерений в секунду должен делать АЦП, чтобы правильно записать свист с частотой 3000 Гц?

Урок 3. Выводы платы: что есть что

Цель	узнать назначение выводов платы: питание, земля, цифровые и аналоговые входы, особые выводы
Опирается на	урок 1 (как выглядит плата), урок 2 (аналоговые и цифровые сигналы)
Практика	находим на плате выводы, которые понадобятся в курсе, и запоминаем, какие трогать нельзя
Понадобится	плата ESP32-S3-DevKitC-1

Теперь, когда мы знаем, чем аналоговый сигнал отличается от цифрового, вернёмся к выводам платы. Вот полная карта — её стоит держать под рукой все уроки:

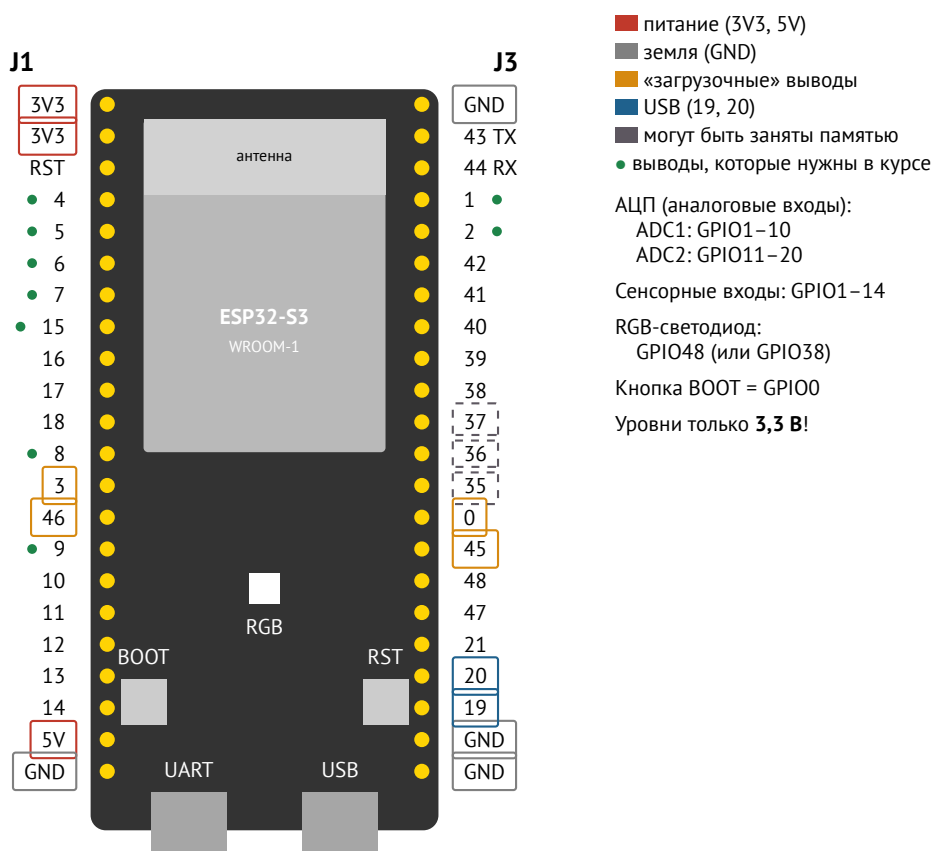


Рис. 15: Карта выводов ESP32-S3-DevKitC-1 (вид сверху). Числа — номера GPIO. У разных версий платы бывают отличия — сверяйся с документацией своей платы

3.1 Питание и земля

Электрический ток течёт только по **замкнутому кругу**: от «плюса» источника через нагрузку (например, светодиод) и обратно к «минусу». Если круг где-то разорван — тока нет. Это похоже на воду в трубах: насос гонит воду по кругу, а если перекрыть трубу, течение прекратится.

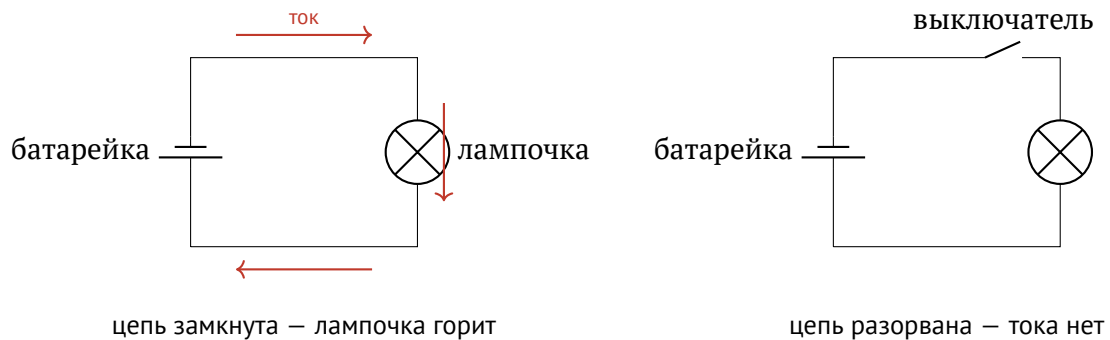


Рис. 16: Электрическая цепь: ток течёт только по замкнутому кругу

На плате для этого есть выводы питания:

- **GND** (англ. *ground* — земля) — общий «минус». От него отсчитывают все напряжения, как высоту гор отсчитывают от уровня моря. На плате несколько выводов GND, все они соединены.
- **3V3** — «плюс» 3,3 В. Отсюда будем питать датчики и экраны.
- **5V** — 5 В прямо от USB. Подходит для питания моторчиков и модулей на 5 В, но *никогда* не подключай 5 В к GPIO!

Осторожно

Никогда не соединяй проводом 3V3 или 5V с GND напрямую. Это **короткое замыкание**: ток станет огромным, плата нагреется и может сгореть, а USB-порт компьютера — отключиться.

3.2 Цифровые выводы GPIO

Любой пронумерованный вывод может быть **цифровым**:

- **выходом** — программа выдаёт на нём 0 (0 В) или 1 (3,3 В). Так мы будем зажигать светодиоды (урок 5);
- **входом** — программа узнаёт, что на выводе: 0 или 1. Так мы будем читать кнопки (урок 10).

Вывод-выход похож на переключатель, который подсоединяет ножку либо к 3,3 В, либо к земле:

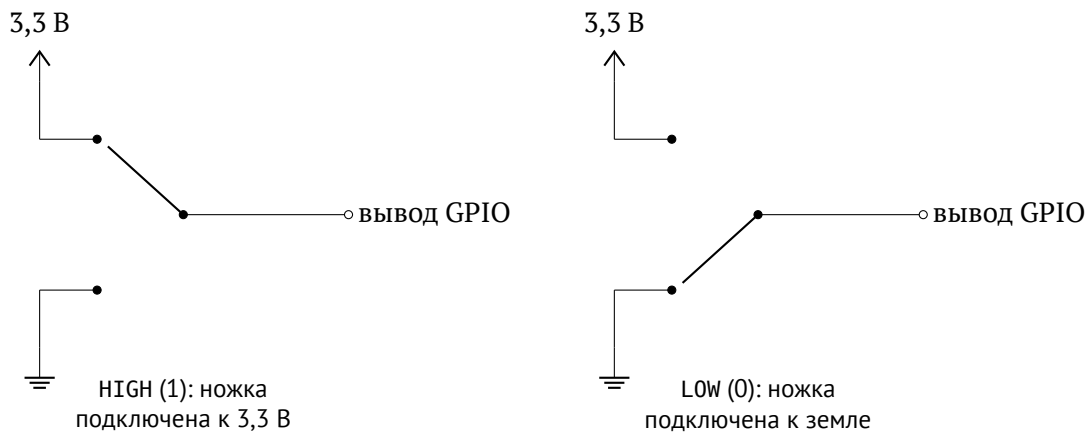


Рис. 17: Цифровой выход можно представить как переключатель. На самом деле внутри чипа стоят два крошечных транзистора, и переключаются они миллионы раз в секунду

Осторожно

Один вывод может отдать ток не больше ≈ 20 mA (максимум 40 mA). Этого хватает для светодиода, но мотор, реле или мощную лампу подключать напрямую **нельзя** — только через транзистор или специальный модуль-драйвер.

3.3 Аналоговые входы (АЦП)

Выводы **GPIO1–GPIO20** умеют не только отличать 0 от 1, но и **измерять напряжение**: внутри чипа к ним можно подключить АЦП, о котором мы говорили в уроке 2. АЦП в ESP32-S3 12-битный: он превращает напряжение от 0 до $\approx 3,1$ В в число от 0 до 4095.

АЦП в чипе два:

- **ADC1** — выводы **GPIO1–GPIO10**. Работает всегда;
- **ADC2** — выводы **GPIO11–GPIO20**. *Не работает, когда включён Wi-Fi*, потому что его использует радиомодуль.

Поэтому для аналоговых датчиков лучше сразу брать выводы ADC1. Мы будем использовать **GPIO1** и **GPIO2**.

3.4 Сенсорные входы

Выводы **GPIO1–GPIO14** умеют чувствовать **прикосновение пальца**. Человеческое тело немного проводит ток и работает как маленький конденсатор, поэтому, если коснуться провода, подключённого к такому выводу, чип это заметит. Так устроены сенсорные кнопки на плитках и лампах.

3.5 Выводы для связи и особые выводы

Выводы	Для чего нужны
GPIO43 (TX), GPIO44 (RX)	связь с компьютером через разъём UART (урок 9)
GPIO19, GPIO20	разъём USB — лучше не занимать
GPIO0, GPIO3, GPIO45, GPIO46	«загрузочные»: чип смотрит на них при включении, чтобы понять, запускать программу или ждать прошивку. GPIO0 — это кнопка BOOT. Не подключай к ним кнопки и датчики
GPIO35, GPIO36, GPIO37	на многих модулях заняты памятью PSRAM — не использовать
GPIO48 (или GPIO38)	встроенный RGB-светодиод
GPIO26–GPIO32	заняты памятью программ, на гребёнку не выведены

Таблица 4: Выводы с особыми обязанностями

А где же выводы для экранов (I²C) и других модулей? У ESP32-S3 есть удобная особенность — **матрица GPIO**. Это как старинный телефонный коммутатор: внутренний блок связи можно «соединить» почти с любым выводом. Поэтому мы сами выбираем, какие выводы использовать, и просто сообщаем об этом программе.

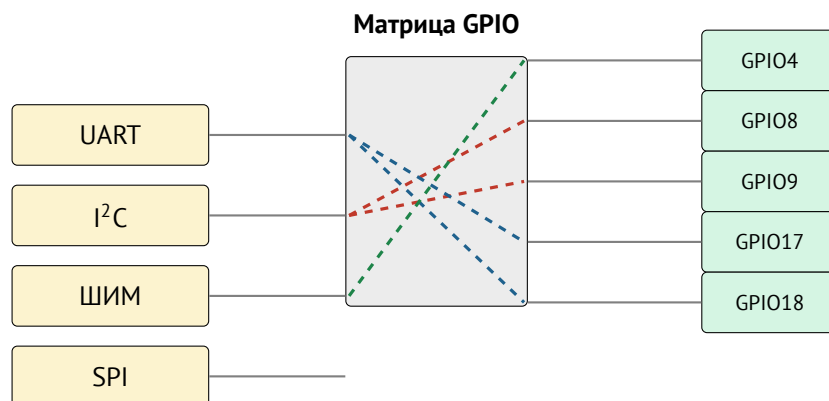


Рис. 18: Матрица GPIO соединяет внутренние блоки с выводами, которые мы выбрали

3.6 Выводы, которые мы будем использовать

Чтобы не путаться, во всех уроках одни и те же детали подключаются к одним и тем же выводам:

Вывод	Что подключаем	Уроки
GPIO4	светодиод	5, 6, 8, 9, 10, 14
GPIO5, GPIO6, GPIO7	ещё три светодиода для «бегущего огня»	5, 6
GPIO5	кнопка	10, 14
GPIO6	сервопривод	11
GPIO15	пьезопищалка	6
GPIO1	потенциометр (аналоговый вход)	11, 12, 14
GPIO2	делитель напряжения (аналоговый вход)	11
GPIO8, GPIO9	экран: SDA и SCL	12
GPIO17, GPIO18	второй UART	9

Таблица 5: Кто куда подключается в этом курсе

Итоги урока

- Ток течёт по замкнутой цепи. GND — общий «минус», 3V3 — «плюс» 3,3 В, 5V — питание от USB.
- Любой GPIO может быть цифровым входом или выходом; **GPIO1–GPIO20** ещё и измеряют напряжение (АЦП).
- При работе Wi-Fi аналоговые входы ADC2 (**GPIO11–GPIO20**) не работают.
- Загрузочные выводы, выводы USB и памяти лучше не занимать.

Проверь себя

1. Почему нельзя соединять 3V3 и GND проводом?
2. Какие выводы подойдут для измерения напряжения, если в проекте нужен Wi-Fi?
3. Можно ли подключить мотор прямо к выводу GPIO? Почему?

Урок 4. Arduino IDE и первая программа

Цель	установить среду программирования, загрузить в плату первую программу, узнать правила записи программ и что такое переменные
Опирается на	урок 1 (разъёмы и кнопки платы)
Практика	плата присылает на компьютер приветствие и рассказывает о себе
Понадобится	плата, USB-кабель с передачей данных (некоторые кабели умеют только заряжать!), компьютер

4.1 Как программа попадает в плату

Микроконтроллер не понимает текст на человеческом языке. Мы пишем программу на языке C++, а специальная программа — **компилятор** — переводит её в машинный код: длинную цепочку нулей и единиц. Затем этот код по USB записывается во Flash-память платы. Это называется **прошивкой**.

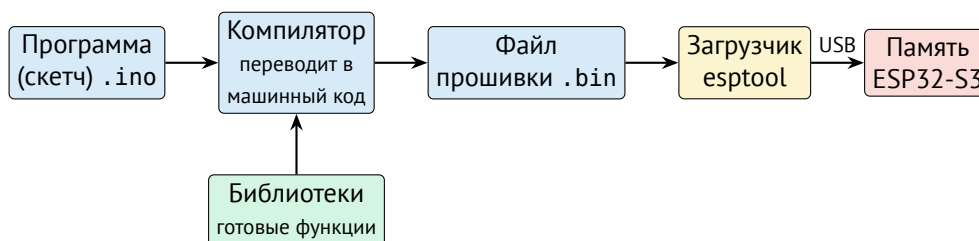


Рис. 19: Путь от текста программы до платы. Всё это Arduino IDE делает по одной кнопке

Программы для Arduino называют **скетчами** (англ. *sketch* — набросок).

4.2 Установка Arduino IDE

Мы будем писать программы в Arduino IDE 2.x — бесплатной и простой среде.

1. Скачай Arduino IDE 2 с сайта <https://www.arduino.cc/en/software> и установи.
2. Открой File → Preferences (Файл → Настройки) и в поле Additional boards manager URLs вставь адрес:
`https://espressif.github.io/arduino-esp32/package_esp32_index.json`
3. Открой Tools → Board → Boards Manager (Инструменты → Плата → Менеджер плат), найди **esp32 by Espressif Systems** и установи версию 3.x. Скачивание займёт несколько минут.
4. Подключи плату кабелем. В Tools → Board выбери **ESP32S3 Dev Module**, а в Tools → Port — появившийся порт (COM3, /dev/ttyUSB0 или похожий).
5. Если плата подключена через разъём **USB**, включи Tools → USB CDC On Boot: Enabled. Иначе сообщения от платы не дойдут до компьютера. Для разъёма UART эта настройка не нужна.

Совет

Если порт не появился, попробуй другой кабель: очень часто виноват кабель «только для зарядки». Для разъёма UART на Windows может понадобиться драйвер CP210x или CH343 — его название обычно написано на маленькой микросхеме рядом с разъёмом.

4.3 Из чего состоит скетч

Любой скетч состоит из двух главных частей — **функций**:

- `setup()` (англ. «настройка») выполняется *один раз* после включения. Здесь мы настраиваем выводы и готовимся к работе;
- `loop()` (англ. «петля») повторяется *бесконечно*, пока есть питание. Здесь находится основная работа.

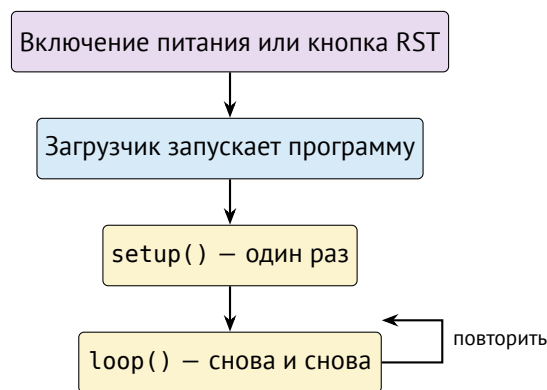


Рис. 20: Порядок работы программы

4.4 Первая программа

Первая программа ничего не подключает: она только сообщает о себе через USB. Набери её (лучше именно набрать, а не скопировать — так быстрее запоминается):

Листинг 1: Первый скетч: «Привет, ESP32-S3»

```

void setup() {
  Serial.begin(115200);           // открыть канал связи с компьютером
  delay(500);                     // дать USB время подняться
  Serial.println("Привет, ESP32-S3!");
  Serial.printf("Частота CPU: %lu МГц\n", getCpuFrequencyMhz());
  Serial.printf("Свободно SRAM: %lu байт\n", ESP.getFreeHeap());
  Serial.printf("PSRAM: %lu байт\n", ESP.getPsramSize());
  Serial.printf("Flash: %lu байт\n", ESP.getFlashChipSize());
}

void loop() {
  Serial.printf("Прошло %lu мс\n", millis());
  delay(1000);
}

```

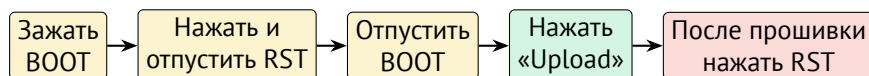
Что здесь написано:

- `Serial.begin(115200)` — включить связь с компьютером на скорости 115 200 бит в секунду;
- `Serial.println("...")` — отправить строку текста на компьютер;
- `Serial.printf(...)` — отправить текст, подставив в него числа (вместо `%lu`);
- `delay(1000)` — подождать 1000 миллисекунд, то есть одну секунду;
- `millis()` — сколько миллисекунд прошло с момента включения.

Нажми кнопку Upload (стрелка вправо). Когда внизу появится Done uploading, открой Tools → Serial Monitor, выбери скорость 115200 — и увидишь сообщения от платы. Как именно устроена эта связь, узнаем в уроке 9.

Если прошивка не начинается

Чтобы принять новую прошивку, чип должен войти в специальный режим. Обычно Arduino IDE делает это сама, но если ты видишь ошибку Failed to connect, помоги ей кнопками:



4.5 Учимся программировать: как записывается программа

Программа похожа на рецепт: список команд, которые выполняются *по порядку*, сверху вниз. Компьютер выполняет их очень точно и ничего не додумывает. Если в рецепте написано «положить 100 кг соли», компьютер положит 100 кг. Поэтому у языка C++ (читается «си-плюс-плюс»), на котором пишут программы для Arduino, есть строгие правила записи. Посмотри на них на примере строчки из следующей программы:

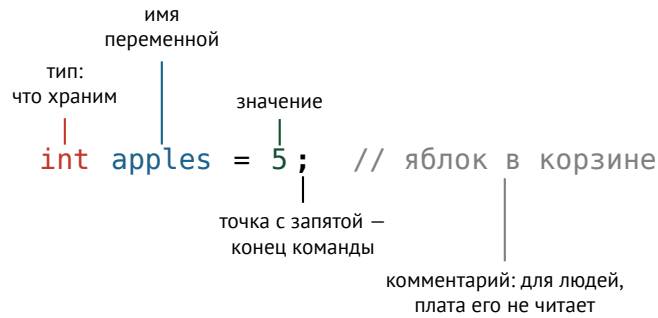


Рис. 21: Из чего состоит строчка программы

- каждая команда заканчивается **точкой с запятой** `;`;
- после `//` до конца строки идёт **комментарий** — пояснение для людей;
- фигурные скобки `{ }` объединяют несколько команд в **блок**, как абзац в тексте. Всё, что между скобками после `void setup()`, — это содержимое функции `setup()`;
- большие и маленькие буквы различаются: `apples` и `Apples` — разные имена;
- в дробных числах пишут **точку**, а не запятую: `23.5`;
- текст пишут в двойных кавычках: `"Привет"`.

Совет

Если в программе ошибка, при компиляции внизу Arduino IDE появится красное сообщение с номером строки. Чаще всего виноваты забытая `;`, незакрытая скобка или опечатка в имени. Ошибки — это нормально: их делают все программисты, даже самые опытные.

4.6 Учимся программировать: переменные

Программе нужно что-то помнить: сколько раз нажали кнопку, какая яркость у светодиода, какая температура. Для этого есть **переменные**. Переменная — это коробка с названием, в которой хранится значение. Значение можно положить, посмотреть и заменить другим. На коробке написано не только имя, но и **тип** — что в неё можно класть: целые числа, дробные, текст.

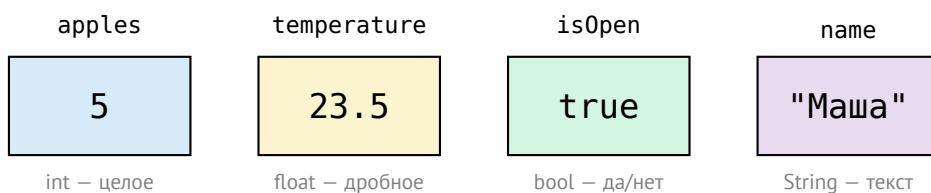


Рис. 22: Переменные — подписанные коробки. Тип говорит, что в коробку можно положить

Тип	Что хранит	Пример
int	целое число от $-2\,147\,483\,648$ до $2\,147\,483\,647$ (32 бита)	int score = 10;
float	дробное число (около 7 точных цифр)	float t = 36.6;
bool	логическое значение: true (истина) или false (ложь)	bool on = false;
String	строку текста	String s = "Привет";
uint8_t, byte	целое от 0 до 255 (8 бит, как в уроке 2)	byte r = 200;
uint32_t	целое от 0 до $4\,294\,967\,295$ — для времени millis()	uint32_t t = millis();

Таблица 6: Типы переменных, которые встретятся в курсе

Знак = в программировании означает не «равно», а «**положить в коробку**». Запись apples = apples + 3 читается так: «возьми, что лежит в apples, прибавь 3 и положи результат обратно в apples». В математике это бессмыслица, а для программы — обычное дело.

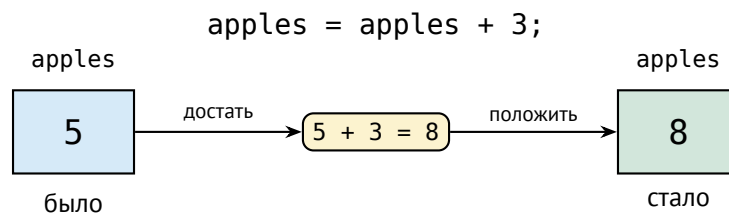


Рис. 23: Присваивание: сначала считается правая часть, потом результат кладётся в переменную

Запись	Что делает	Пример ($x = 7$)
+ - * /	сложение, вычитание, умножение, деление	$x * 2 \rightarrow 14$
%	остаток от деления	$x \% 2 \rightarrow 1$
x++, x--	увеличить или уменьшить на 1	x станет 8 или 6
x += 5	то же, что $x = x + 5$	x станет 12

Таблица 7: Арифметические операции

Листинг 2: Переменные и операции

```

// Переменные: коробки с названиями
int apples = 5;           // целое число
float temperature = 23.5; // дробное число (точка вместо запятой!)
bool isOpen = true;      // логическое: true (да) или false (нет)
String name = "Маша";    // строка текста
const int DAYS_IN_WEEK = 7; // константа: менять нельзя

void setup() {
  Serial.begin(115200);
  delay(500);

  Serial.print("Яблоко: ");
  Serial.println(apples); // 5

  apples = apples + 3;     // взять старое значение, прибавить 3, положить
  ↪ обратно
  Serial.print("Стало яблоко: ");
  Serial.println(apples); // 8

  apples++;               // то же, что apples = apples + 1
  Serial.println(apples); // 9
  
```

```
Serial.println(7 / 2);           // 3    -- целые числа делятся нацело!  
Serial.println(7 % 2);           // 1    -- остаток от деления  
Serial.println(7.0 / 2);         // 3.50 -- а дробные -- как обычно  
  
Serial.println("Привет, " + name + "!");  
Serial.printf("На улице %.1f градусов\n", temperature);  
}  
  
void loop() {}
```

Осторожно

Если оба числа целые, деление тоже будет целым: $7 / 2$ даёт **3**, а не $3,5$ — дробная часть просто отбрасывается. Чтобы получить $3,5$, хотя бы одно число должно быть дробным: $7.0 / 2$.

Переменная, объявленная *вне* функций (в начале программы), — **глобальная**: её видят все части программы. Переменная, объявленная *внутри* фигурных скобок, — **локальная**: она существует только внутри этого блока. Как записка на доске в классе (видят все) и записка в твоём кармане (видишь только ты).

Теперь понятно, что было в первой программе: `setup()` и `loop()` — это **функции**, наборы команд под одним именем; `Serial.println()` и `delay()` — готовые функции, которые мы *вызываем*, передавая им в скобках данные; `millis()` возвращает число — его можно положить в переменную.

Итоги урока

- Программа (скетч) пишется в Arduino IDE, переводится компилятором в машинный код и загружается по USB.
- `setup()` выполняется один раз, `loop()` — бесконечно.
- `Serial` позволяет плате отправлять сообщения на компьютер.
- Если прошивка не идёт — кнопки BOOT и RST.
- Команды заканчиваются `;`, блоки — в `{ }`, комментарии — после `//`.
- Переменная — коробка с именем и типом; `=` кладёт в неё значение; целые числа делятся нацело.

Задания для самостоятельной работы

1. Добавь в программу вывод своего имени и возраста.
2. Сделай так, чтобы плата печатала время работы не в миллисекундах, а в секундах.
3. Узнай, сколько памяти (Flash и PSRAM) у твоей платы, и сравни с надписью на модуле.
4. Заведи переменные `a = 17` и `b = 5` и напечатай их сумму, разность, произведение, частное и остаток. Почему частное получилось таким?

Урок 5. Светодиод: закон Ома и мигание

Цель	правильно подключить светодиод, рассчитать резистор по закону Ома, заставить светодиод мигать; научиться использовать условие <code>if</code> , массивы и цикл <code>for</code>
Опирается на	урок 2 (цифровой сигнал), урок 3 (выводы GPIO), урок 4 (переменные)
Практика	мигающий светодиод и «бегущий огонь» из четырёх светодиодов
Понадобится	макетная плата, провода, 4 светодиода, 4 резистора $220\ \Omega$

5.1 Светодиод

Светодиод (англ. LED — *light-emitting diode*) — это деталь, которая светится, когда через неё течёт ток. Он пропускает ток *только в одну сторону*, как дверь, которая открывается только наружу. Поэтому у светодиода есть «плюс» и «минус»:

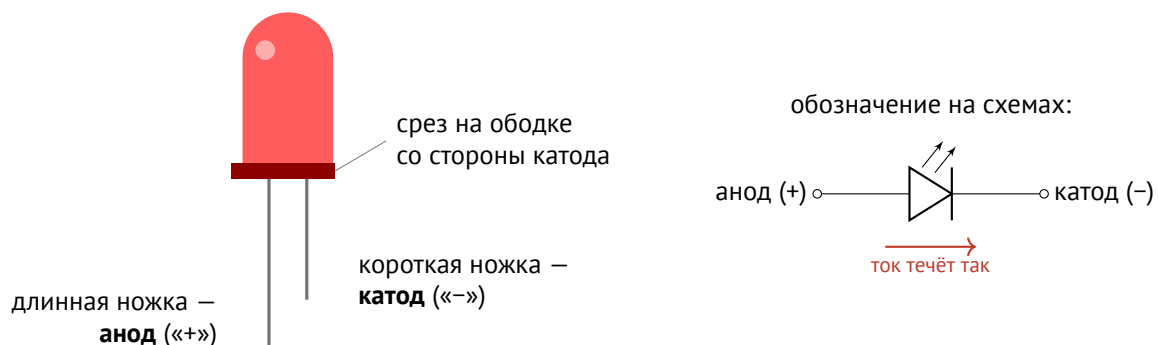


Рис. 24: Светодиод и его условное обозначение. Треугольник показывает, в какую сторону течёт ток

Если перепутать ножки, светодиод просто не загорится — ничего страшного. А вот если подключить его без резистора, он сгорит. Разберёмся почему.

5.2 Зачем нужен резистор: закон Ома

Электричество удобно сравнивать с водой в трубах:

- **напряжение** U (вольты, В) — это «давление», которое толкает воду;
- **ток** I (амперы, А) — сколько воды протекает за секунду;
- **сопротивление** R (омы, Ω) — насколько узкая труба: чем уже, тем меньше воды пройдёт.

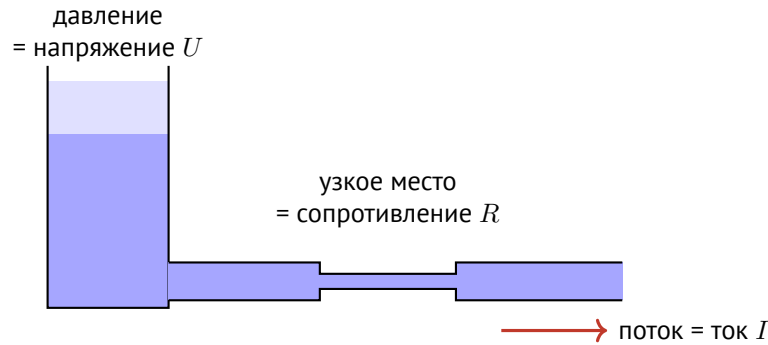


Рис. 25: «Водяная» модель электрической цепи

Эти три величины связаны **законом Ома** — главным законом электроники:

$$I = \frac{U}{R} \quad \text{или} \quad R = \frac{U}{I}.$$

Больше давление — больше поток. Уже труба — меньше поток.

Почему светодиод сгорает без резистора

Светодиод ведёт себя необычно: пока напряжение на нём маленькое, ток почти не течёт. Но стоит напряжению превысить определённый порог — **прямое напряжение** U_F , для красного светодиода около 2 В, — ток начинает расти *очень* быстро:

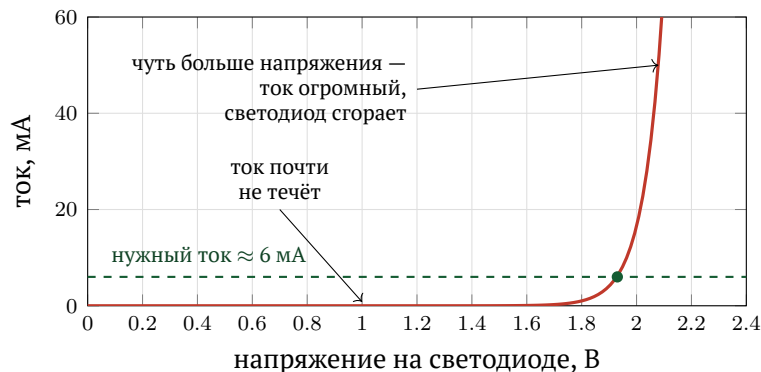


Рис. 26: Как ток через красный светодиод зависит от напряжения на нём (вольт-амперная характеристика)

Вывод платы выдаёт 3,3 В — намного больше, чем нужно светодиоду. Если подключить его напрямую, ток окажется огромным, и светодиод (а может, и вывод платы) сгорит. **Резистор** забирает на себя «лишнее» напряжение и ограничивает ток. Поэтому правило:

Светодиод подключается к выводу только через резистор!

Рассчитываем резистор

Из 3,3 В на светодиод придётся $U_F \approx 2,0$ В (для красного), значит, на резистор — оставшиеся $3,3 - 2,0 = 1,3$ В. Светодиоду достаточно тока около $6 \text{ мА} = 0,006 \text{ А}$. По закону Ома:

$$R = \frac{U_{\text{питания}} - U_F}{I} = \frac{3,3 \text{ В} - 2,0 \text{ В}}{0,006 \text{ А}} \approx 217 \Omega.$$

Резисторы продаются только определённых номиналов, поэтому берём ближайший: 220Ω .

Разные светодиоды требуют разного напряжения: красному хватает около 2 В, а синему нужно почти 3 В.¹ Поэтому и резистор для них нужен разный:

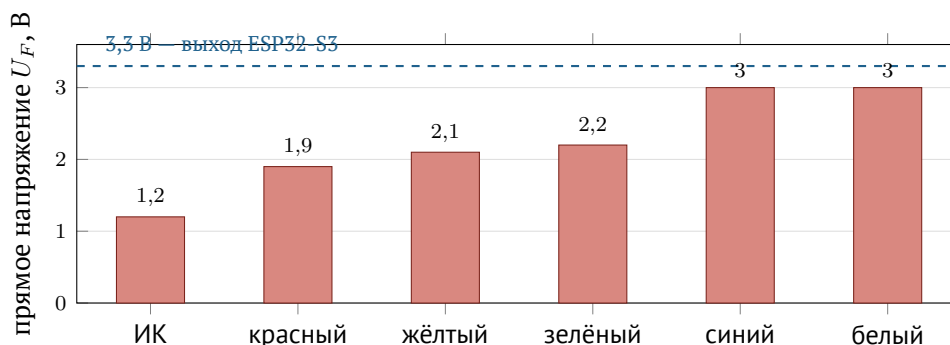


Рис. 27: Прямое напряжение светодиодов разных цветов (примерные значения)

Цвет светодиода	U_F , В	На резисторе, В	Резистор для 6 мА
Красный	1,9	$3,3 - 1,9 = 1,4$	$1,4/0,006 \approx 230 \Omega \rightarrow 220 \Omega$
Жёлтый, зелёный	2,1–2,2	1,1–1,2	$\approx 190 \Omega \rightarrow 180 \Omega - 220 \Omega$
Синий, белый	3,0	0,3	$0,3/0,006 = 50 \Omega \rightarrow 47 \Omega$

Таблица 8: Пример расчёта резисторов при питании от вывода 3,3 В

Синему и белому светодиоду от 3,3 В достаётся совсем небольшой запас, поэтому они светят чуть слабее. Для опытов в курсе лучше всего подходят красные, жёлтые и зелёные светодиоды с резистором 220 Ω .

Как узнать сопротивление резистора

Номинал резистора закодирован цветными полосками. Первые две — цифры, третья — сколько нулей дописать, а золотая полоска с краю — точность:

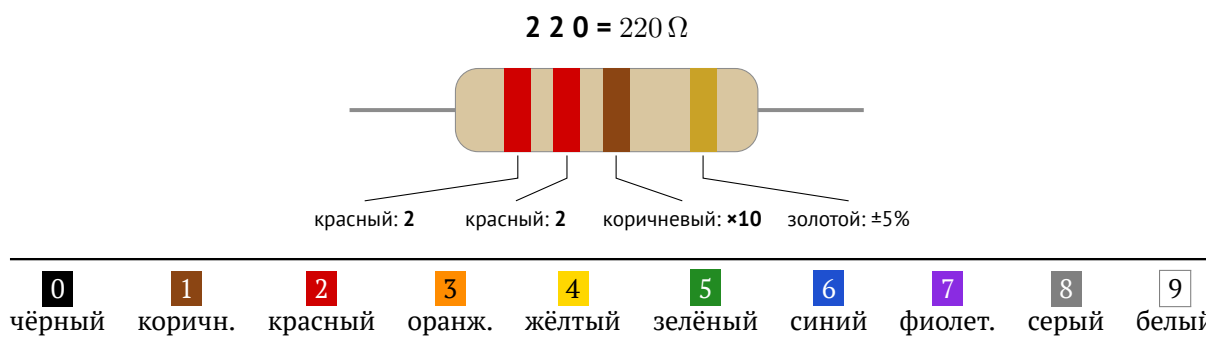


Рис. 28: Цветовая маркировка резисторов. Например, коричневый–чёрный–оранжевый = 1, 0 и три нуля = 10 000 Ω = 10 k Ω

¹Всё дело в энергии света. Свет состоит из крошечных порций — фотонов, и у синего света каждый фотон несёт больше энергии, чем у красного. Чтобы «выбить» из светодиода синий фотон, электрону нужно больше энергии — значит, нужно и большее напряжение. Поэтому ультрафиолетовые светодиоды требуют ещё больше, чем синие, а инфракрасные (в пульте от телевизора) — меньше всего, около 1,2 В.

5.3 Макетная плата и два вида схем

Чтобы собирать схемы без паяльника, используют **макетную плату** (англ. *breadboard*). Внутри под рядами отверстий спрятаны металлические полоски-зажимы. Всё, что воткнуто в одну полосу, соединено между собой:

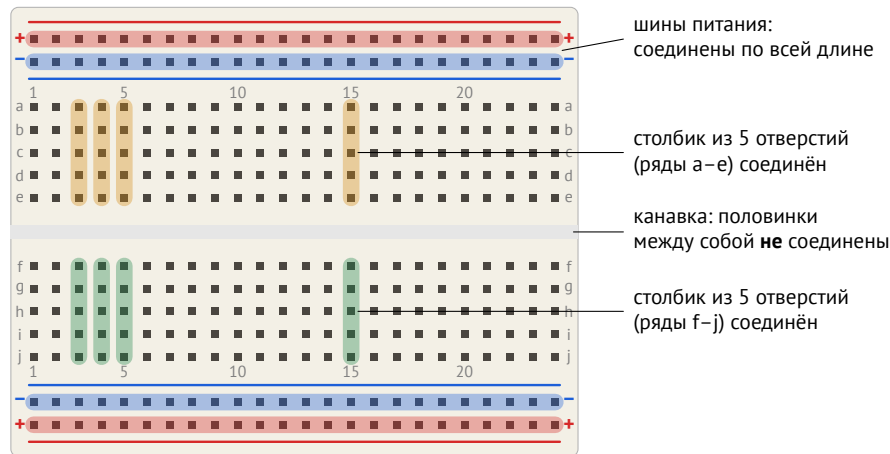


Рис. 29: Как соединены отверстия макетной платы (подсвечены соединённые группы)

Осторожно

Обе ножки детали нельзя втыкать в один столбик: тогда они замкнутся друг на друга, и ток пойдёт мимо детали.

Каждую схему в этой книге мы рисуем **двумя способами**:

- **монтажная схема** — как детали и провода выглядят на макетной плате. По ней удобно собирать;
- **принципиальная электрическая схема** — детали заменены условными значками, а провода — линиями. По ней удобно понимать, *как схема работает*. Такие схемы рисуют инженеры всего мира, и их стоит научиться читать.

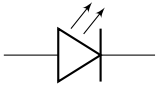
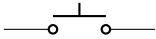
Деталь	На макетной плате	На принципиальной схеме
Светодиод		
Резистор		
Кнопка		
Потенциометр		
Земля (GND)	черный провод к шине «-»	 3,3 В
Питание 3,3 В	красный провод к шине «+»	
Вывод платы	цветной провод к гребёнке	GPIO4 

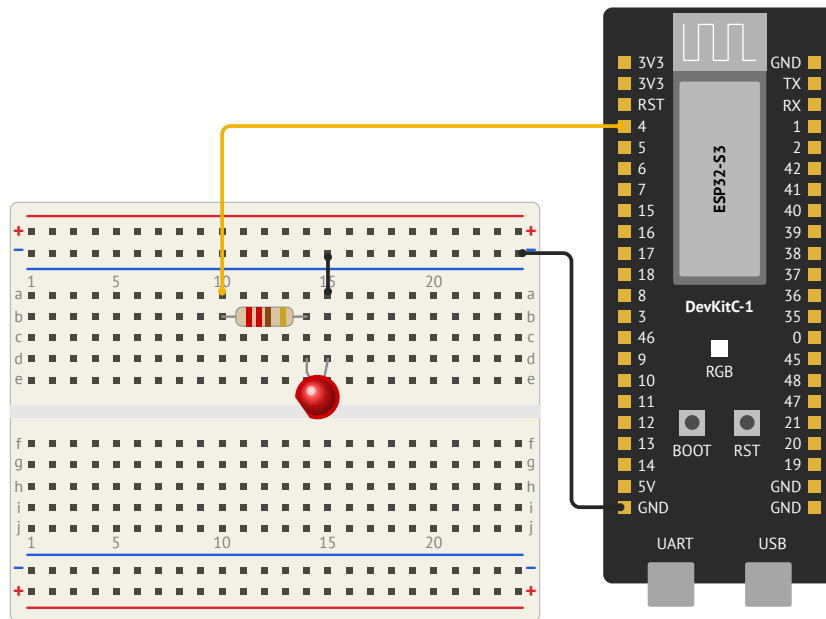
Таблица 9: Как одни и те же детали выглядят на двух видах схем

Совет

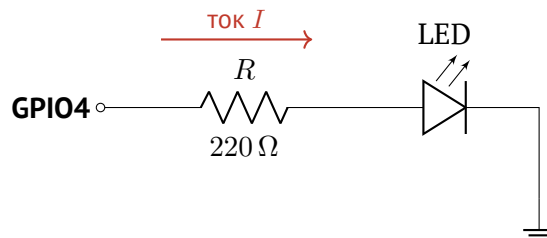
Договоримся о цветах проводов: **красный** — питание 3,3 В, **чёрный** — земля GND, остальные цвета — сигналы. В настоящих устройствах инженеры тоже соблюдают такие правила: так проще искать ошибки.

5.4 Подключаем светодиод

Соберём схему: вывод **GPIO4** → резистор 220 Ω → анод светодиода, катод → земля (GND).



а) Монтажная схема: как собрать на макетной плате



б) Принципиальная электрическая схема

Рис. 30: Подключение светодиода к выводу GPIO4

Сравни две картинки: жёлтый провод на макетной плате — это линия от **GPIO4** на схеме, чёрные провода — значок земли. Проследи путь тока пальцем: от вывода платы через резистор и светодиод к земле и обратно в плату — цепь замкнута.

5.5 Программа Blink: мигаем светодиодом

Листинг 3: Мигание светодиодом

```
const int LED_PIN = 4;

void setup() {
  pinMode(LED_PIN, OUTPUT);    // настроить вывод как выход
}

void loop() {
  digitalWrite(LED_PIN, HIGH); // 3,3 В -> светодиод горит
  delay(500);
  digitalWrite(LED_PIN, LOW);  // 0 В -> гаснет
  delay(500);
}
```

- `pinMode(LED_PIN, OUTPUT)` — сделать вывод выходом;
- `digitalWrite(LED_PIN, HIGH)` — выдать логическую 1 (3,3 В), светодиод загорается;

- `digitalWrite(LED_PIN, LOW)` — выдать 0 (0 В), светодиод гаснет;
- `delay(500)` — подождать полсекунды.

Учимся программировать: константы и вызов функций

Первая строка `const int LED_PIN = 4;` создаёт **константу** — переменную, которую нельзя изменить. Зачем она, если можно просто писать 4? Если завтра светодиод переедет на другой вывод, исправить придётся одну строчку, а не искать все четвёрки по программе. К тому же `LED_PIN` понятнее, чем просто число. Константы принято называть **БОЛЬШИМИ БУКВАМИ**. Строка `digitalWrite(LED_PIN, HIGH);` — это **вызов функции**. Имя функции говорит, что сделать («записать цифровое значение»), а в скобках через запятую перечислены **аргументы** — с чем сделать: на каком выводе и какое значение. `HIGH` и `LOW` — встроенные константы, равные 1 и 0.

`digitalWrite ( LED_PIN , HIGH );`

Рис. 31: Вызов функции: имя и аргументы в скобках

Посмотрим, какое напряжение при этом на выводе. Это цифровой сигнал из урока 2: у него всего два уровня, а переходы между ними резкие. Такой сигнал называют **прямоугольным** или **меандром**:

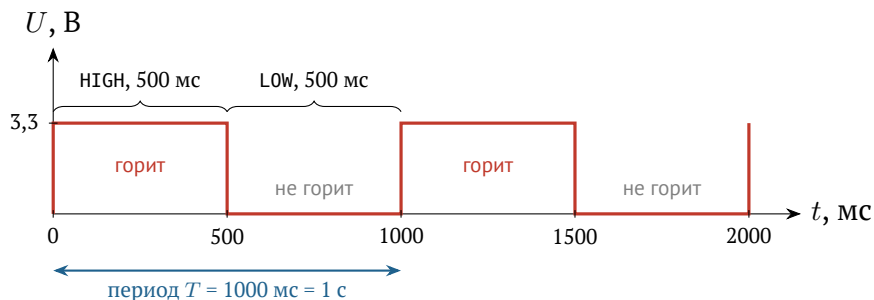


Рис. 32: Напряжение на выводе **GPIO4** при работе Blink — прямоугольный сигнал

Время одного полного цикла «горит + не горит» называется **периодом** T . Здесь $T = 500 + 500 = 1000 \text{ мс} = 1 \text{ с}$. Сколько периодов умещается в секунду, называется **частотой**:

$$f = \frac{1}{T} = \frac{1}{1 \text{ с}} = 1 \text{ Гц}.$$

Светодиод мигает один раз в секунду — частота 1 Гц.

Что видит глаз

При частоте 1 Гц глаз прекрасно видит каждую вспышку: светодиод полсекунды горит, полсекунды не горит. Глаз успевает заметить изменения, потому что они медленные. А что будет, если уменьшать `delay()` и мигать всё быстрее? Светодиод будет мигать чаще и чаще... а потом случится кое-что удивительное. Этот опыт мы проведём в уроке 6, и он приведёт нас к способу менять яркость светодиода.

5.6 Мигание без delay()

У `delay()` есть недостаток: пока идёт ожидание, программа *ничего больше не делает*. Она не заметит нажатия кнопки и не ответит компьютеру. Представь, что ты варишь яйцо и полторы минуты смотришь на часы, не отходя от плиты. Удобнее иногда поглядывать на часы, а в промежутках заниматься другими делами. Так же поступает программа ниже: она постоянно сравнивает текущее время `millis()` с моментом последнего переключения.

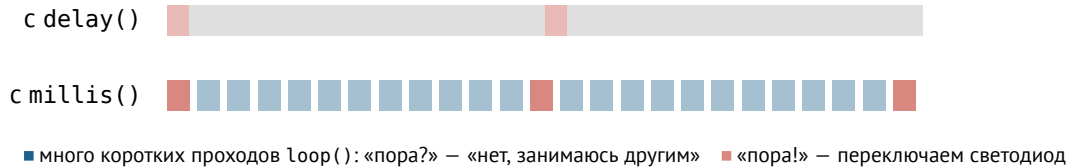


Рис. 33: Слева направо идёт время. Программа с `millis()` постоянно свободна

5.7 Учимся программировать: условие if

Часто нужно сделать что-то *только если* выполняется условие: «если прошло полсекунды — переключи светодиод». Для этого есть команда `if` («если»). Команды внутри фигурных скобок выполняются, только когда условие в круглых скобках верно. А после `else` («иначе») пишут, что делать, если условие неверно.

Алгоритмы удобно рисовать **блок-схемами**. Овал — начало и конец, прямоугольник — действие, ромб — вопрос с ответами «да» и «нет». Вот блок-схема функции `loop()` из программы ниже:

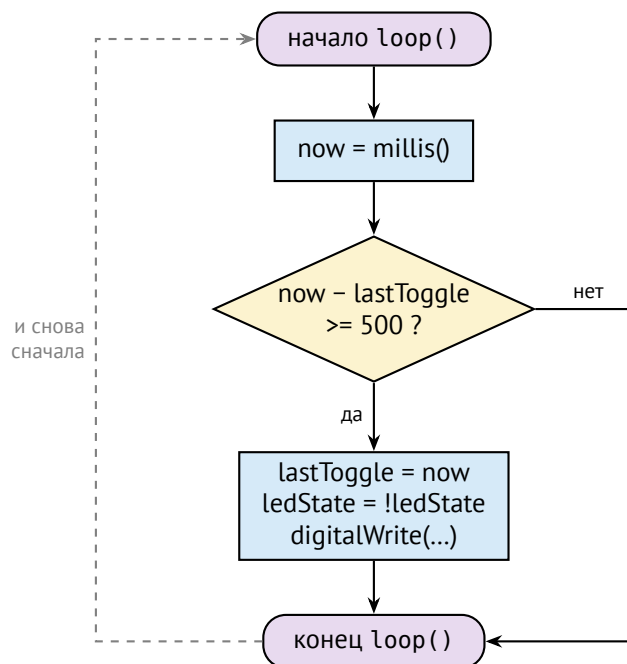


Рис. 34: Блок-схема неблокирующего мигания: `loop()` крутится тысячи раз в секунду, но светодиод переключается, только когда ответ на вопрос — «да»

Для сравнения чисел используются такие знаки:

Знак	Смысл	Пример	Знак	Смысл	Пример
==	равно	x == 5	!=	не равно	x != 0
<	меньше	t < 0	>	больше	t > 30
<=	меньше или равно	n <= 10	>=	больше или равно	now - last >= 500

Таблица 10: Операции сравнения. Результат сравнения — true (верно) или false (неверно)

Осторожно

Не путай = и ==! Одно равно — «положить в коробку», два равно — «сравнить». Запись `if (x = 5)` не проверит, равен ли x пяти, а *положит* в него 5. Это одна из самых частых ошибок.

В программе есть ещё два новых приёма. Переменная `ledState` типа `bool` помнит, горит ли светодиод. Восклицательный знак `!` означает «наоборот»: `ledState = !ledState` превращает `true` в `false` и обратно. А время хранится в `uint32_t` — `millis()` быстро вырастает до огромных чисел.

Листинг 4: Мигание без остановки программы

```
const int LED_PIN = 4;
const uint32_t INTERVAL = 500; // мс
uint32_t lastToggle = 0;
bool ledState = false;

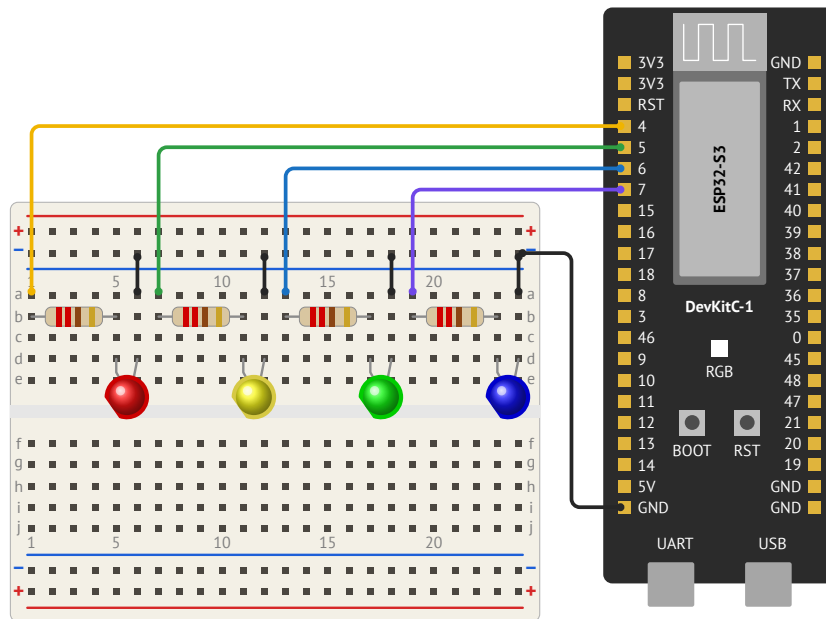
void setup() {
    pinMode(LED_PIN, OUTPUT);
}

void loop() {
    uint32_t now = millis();
    if (now - lastToggle >= INTERVAL) { // корректно и при переполнении millis()
        lastToggle = now;
        ledState = !ledState;
        digitalWrite(LED_PIN, ledState);
    }
    // здесь можно делать что-то ещё, не дожидаясь светодиода
}
```

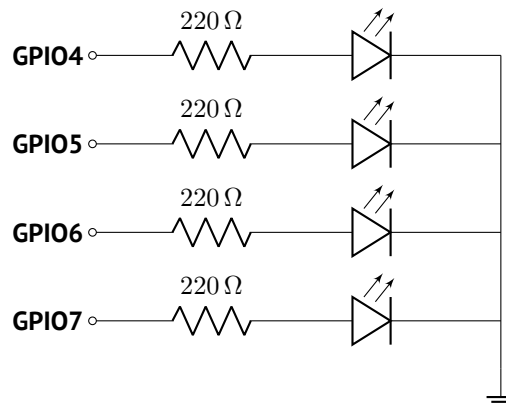
Запомни этот приём: `if (millis() - last >= INTERVAL)`. Он будет встречаться почти в каждом уроке.

5.8 Практика: бегущий огонь

Подключим четыре светодиода к выводам **GPIO4–GPIO7**, каждый со своим резистором, и будем зажигать их по очереди. Получится огонёк, который бежит по кругу, как на новогодней гирлянде.



а) Монтажная схема: как собрать на макетной плате



б) Принципиальная электрическая схема

Рис. 35: Четыре светодиода на выводах GPIO4–GPIO7

Обрати внимание: на принципиальной схеме земли всех светодиодов соединены одной линией. На макетной плате это сделано с помощью шины «-»: все катоды подключены к ней, а она — к GND платы.

5.9 Учимся программировать: массивы и цикл for

В бегущем огне четыре светодиода, и для каждого нужно вызвать `pinMode()`. Можно написать четыре одинаковые строчки, но что, если светодиодов будет двадцать? На помощь приходят два инструмента.

Массив — это ряд пронумерованных коробок под одним именем. Номер коробки называется **индексом**, и счёт начинается с нуля!

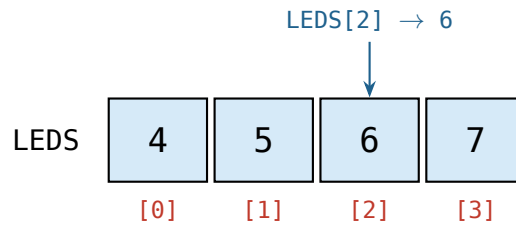


Рис. 36: Массив `const int LEDS[] = {4, 5, 6, 7};` — четыре ячейки с индексами от 0 до 3

Цикл `for` повторяет команды нужное число раз. У него четыре части:

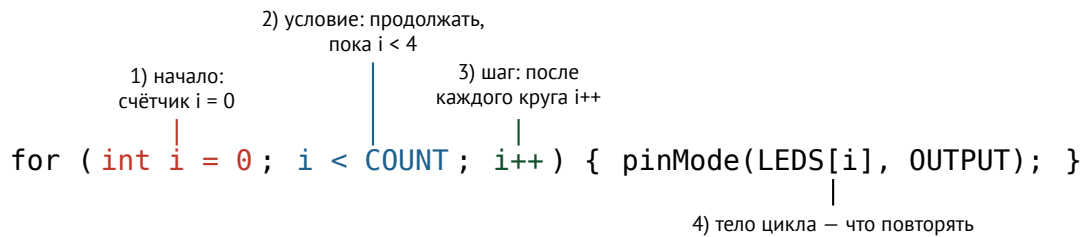
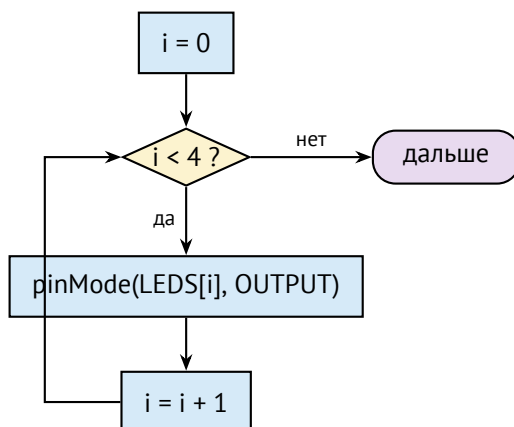


Рис. 37: Четыре части цикла `for`



Круг	i	LEDs[i]	Что выполнилось
1	0	4	<code>pinMode(4, OUTPUT)</code>
2	1	5	<code>pinMode(5, OUTPUT)</code>
3	2	6	<code>pinMode(6, OUTPUT)</code>
4	3	7	<code>pinMode(7, OUTPUT)</code>
—	4	—	<code>4 < 4</code> неверно — выход

Рис. 38: Блок-схема цикла и «трассировка» — как меняется счётчик на каждом круге

Листинг 5: Бегущий огонь

```
// Бегущий огонь: четыре светодиода загорятся по очереди
const int LEDS[] = {4, 5, 6, 7};           // выводы светодиодов
const int COUNT = 4;                       // сколько их
const uint32_t STEP_MS = 150;              // время горения одного, мс

int current = 0;                           // какой светодиод горит сейчас
uint32_t lastStep = 0;                     // когда было последнее переключение

void setup() {
  for (int i = 0; i < COUNT; i++) {
    pinMode(LEDs[i], OUTPUT);
  }
}

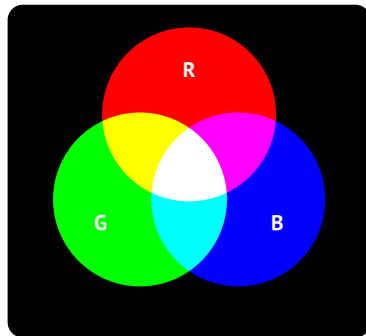
void loop() {
  if (millis() - lastStep >= STEP_MS) {    // пора переключаться?
```

```
lastStep = millis();  
digitalWrite(LEDs[current], LOW);           // гасим текущий  
current = (current + 1) % COUNT;             // следующий: 0,1,2,3,0,1...  
digitalWrite(LEDs[current], HIGH);          // зажигаем его  
}  
}
```

Цикл `for` в `setup()` настраивает все четыре вывода. В `loop()` переменная `current` хранит индекс горящего светодиода. Выражение `(current + 1) % COUNT` даёт по очереди 1, 2, 3, 0, 1, 2...: знак `%` означает остаток от деления, и после последнего светодиода счёт начинается снова с нулевого.

5.10 Встроенный RGB-светодиод

На плате уже есть светодиод, который светится любым цветом. Внутри него три крошечных светодиода: красный (R), зелёный (G) и синий (B). Смешивая их свет в разных пропорциях, можно получить любой цвет. Точно так же устроен каждый пиксель экрана телефона и телевизора!



красный + зелёный = **жёлтый**
зелёный + синий = **голубой**
красный + синий = **пурпурный**
все три = **белый**

Рис. 39: Смешение цветов света. Это не то же самое, что смешивать краски: краски вместе дают тёмный цвет, а свет — белый

Встроенный светодиод подключён к **GPIO48** (на некоторых платах — к **GPIO38**). Для него есть готовая функция `rgbLedWrite(pin, r, g, b)`, где каждая яркость — от 0 до 255:

Листинг 6: Смена цветов встроенного RGB-светодиода

```
const int RGB_PIN = 48;    // на ревизии v1.1 -- 38

void setup() {}

void loop() {
  rgbLedWrite(RGB_PIN, 40, 0, 0); delay(400); // красный (яркость 0..255)
  rgbLedWrite(RGB_PIN, 0, 40, 0); delay(400); // зелёный
  rgbLedWrite(RGB_PIN, 0, 0, 40); delay(400); // синий
  rgbLedWrite(RGB_PIN, 0, 0, 0);  delay(400); // выключен
}
```

Итоги урока

- Светодиод пропускает ток в одну сторону: длинная ножка — анод (+), короткая — катод (-).
- Светодиод подключается только через резистор. Резистор считаем по закону Ома: $R = (U - U_F)/I$.
- Чем «синее» светодиод, тем больше напряжения он требует.
- Blink создаёт на выводе прямоугольный сигнал. Период T — время одного цикла, частота $f = 1/T$.
- Чтобы программа не «засыпала», время отсчитываем через `millis()`, а не `delay()`.
- `const` — константа; `if` выполняет команды, только если условие верно; `==` — сравнение.
- Массив хранит ряд значений с индексами от 0; цикл `for` перебирает их по очереди.

Проверь себя

1. Что случится, если подключить светодиод к выводу без резистора?
2. Какой резистор нужен зелёному светодиоду ($U_F = 2,2$ В) при токе 5 мА и питании 3,3 В?
3. Светодиод горит 200 мс и не горит 300 мс. Чему равны период и частота?
4. Какие цвета полосок у резистора 4,7 кΩ (4700 Ω)?
5. Сколько раз выполнится тело цикла `for (int i = 0; i < 10; i++)`? Какой индекс у последнего элемента массива из 10 ячеек?

Задания для самостоятельной работы

1. Собери «светофор» из красного, жёлтого и зелёного светодиодов с правильной последовательностью и длительностями.
2. Сделай два светодиода, мигающих с разной частотой (1 Гц и 3 Гц), без `delay()`.
3. Заставь «бегущий огонь» бегать туда и обратно, как фары автомобиля из старого фильма.
4. Сделай RGB-светодиод «радугой»: красный → жёлтый → зелёный → голубой → синий → пурпурный.

Урок 6. ШИМ: яркость светодиода и частота

Цель	понять, почему быстрое мигание кажется ровным светом, освоить ШИМ и плавно менять яркость светодиода; научиться писать свои функции
Опирается на	урок 2 (ШИМ, двоичные числа), урок 5 (светодиод, период и частота)
Практика	«дышащий» светодиод, плавная волна на четырёх светодиодах, мелодия на пищалке
Понадобится	схема из урока 5; пассивная пьезопищалка (по желанию)

6.1 Можно ли светить вполсилы?

Команда `digitalWrite()` умеет только две вещи: включить вывод (3,3 В) или выключить (0 В). Светодиод либо горит в полную силу, либо не горит совсем. Как же сделать ночник, который светит тускло? У ESP32-S3 нет выхода, который выдавал бы, скажем, 1,5 В. Зато есть глаз, который легко обмануть. Проведём опыт.

6.2 Опыт: мигаем всё быстрее

Возьми схему со светодиодом на **GPIO4** из прошлого урока и загрузи программу, которая каждые 6 вспышек сокращает время вдвое:

Листинг 7: Опыт с частотой мигания

```
// Опыт: с какой частоты глаз перестаёт видеть мигание?
const int LED_PIN = 4;
int halfPeriod = 500;                                // половина периода, мс

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  for (int i = 0; i < 6; i++) {                        // мигнём 6 раз с текущей частотой
    digitalWrite(LED_PIN, HIGH);
    delay(halfPeriod);
    digitalWrite(LED_PIN, LOW);
```

```

    delay(halfPeriod);
}
halfPeriod = halfPeriod / 2;           // в два раза быстрее
if (halfPeriod < 2) halfPeriod = 500;   // и всё сначала
Serial.printf("Частота: %.1f Гц\n", 1000.0 / (2 * halfPeriod));
}

```

Наблюдай за светодиодом и за сообщениями в Serial Monitor:

delay(), мс	Период T , мс	Частота f , Гц	Что видно
500	1000	1	спокойное мигание
125	250	4	быстрое мигание
31	62	16	дрожащий, мерцающий свет
15	30	33	лёгкое мерцание, особенно краем глаза
7	14	71	ровный свет , но слабее обычного
3	6	167	ровный свет

Таблица 11: Что видит глаз при разной частоте мигания

Удивительно: светодиод продолжает включаться и выключаться, но мы видим ровный свет — примерно в половину яркости! Почему так происходит?

6.3 Инерция зрения

Глаз — не фотоаппарат с мгновенным затвором. Светочувствительные клетки сетчатки реагируют на свет не сразу и «остывают» тоже не сразу: каждое впечатление держится примерно $1/20$ – $1/50$ секунды. Это называется **инерцией зрения**. Если вспышки следуют чаще, они сливаются, и мозг видит *среднюю* яркость.

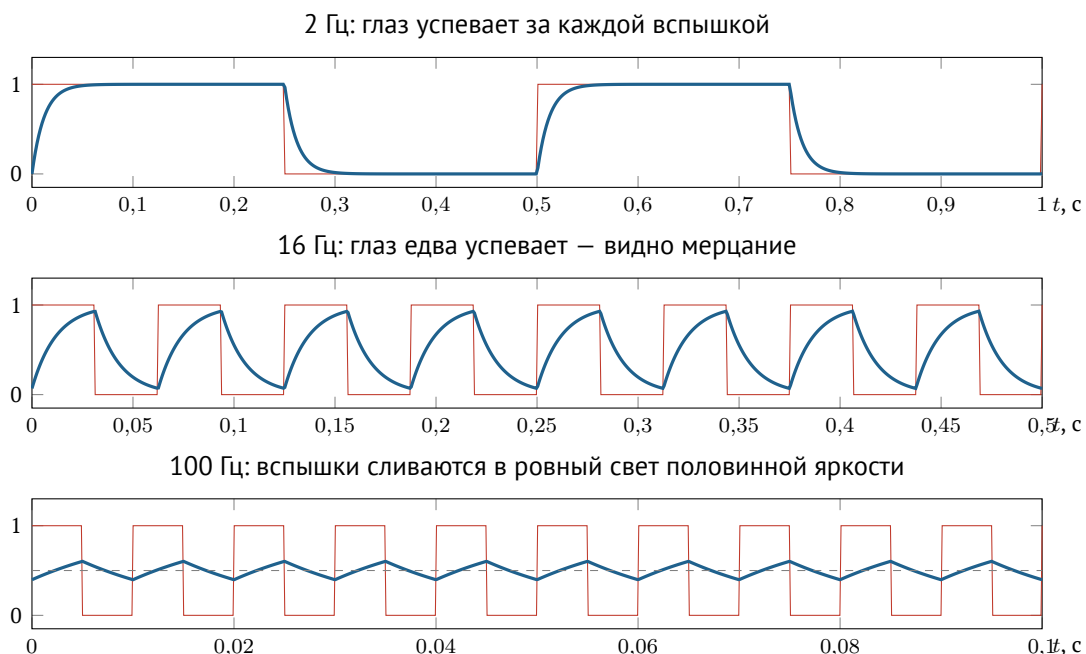


Рис. 40: Один и тот же прямоугольный сигнал (красная линия) с разной частотой. Синяя линия — упрощённая модель того, что «видит» глаз

Это интересно

Инерция зрения — основа кино и мультфильмов. Кинокадры сменяются 24 раза в секунду, а каждый кадр проецируется два-три раза, чтобы вспышки шли с частотой 48–72 Гц. Экран телефона обновляется 60–120 раз в секунду. А обычные лампы в розетке мерцают 100 раз в секунду — это можно увидеть, если посмотреть на них через камеру телефона в режиме замедленной съёмки.

6.4 ШИМ — широтно-импульсная модуляция

Теперь понятна идея: будем мигать *так быстро*, что глаз не видит мигания, а яркость менять, изменяя *долю времени*, пока светодиод горит. Это и есть **ШИМ** — широтно-импульсная модуляция (по-английски PWM, *pulse-width modulation*): мы меняем («модулируем») ширину импульсов.

Доля времени во включённом состоянии называется **коэффициентом заполнения** D (англ. *duty cycle*):

$$D = \frac{t_{\text{вкл}}}{T} \cdot 100\%, \quad U_{\text{среднее}} = D \cdot 3,3 \text{ В.}$$

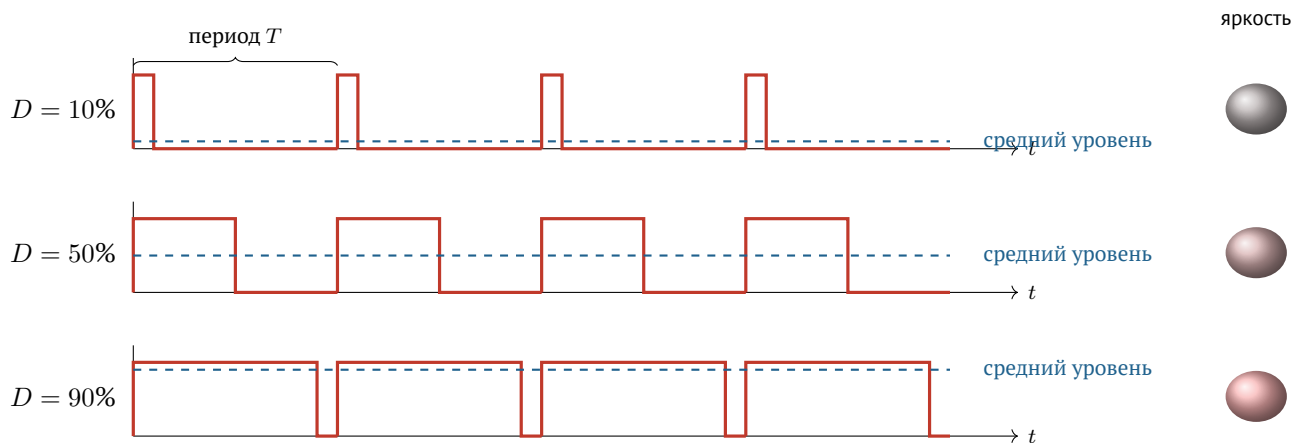


Рис. 41: ШИМ с разным коэффициентом заполнения: чем шире импульсы, тем ярче светодиод

6.5 ШИМ в ESP32-S3: модуль LEDC

В ESP32-S3 ШИМ создаёт специальный блок **LEDC** (от англ. *LED control* — управление светодиодами). Он переключает вывод сам, аппаратно, и процессору не нужно этим заниматься. Нужно задать всего три вещи:

- **вывод;**
- **частоту** ШИМ — сколько раз в секунду вывод включается. Для светодиодов берут 5000 Гц: это в 50 раз быстрее, чем может заметить глаз;
- **разрешение** в битах — на сколько ступенек делится период. 8 бит дают $2^8 = 256$ ступенек: значения от 0 (всегда выключен) до 255 (всегда включён). Помнишь таблицу двоичных чисел из урока 2?

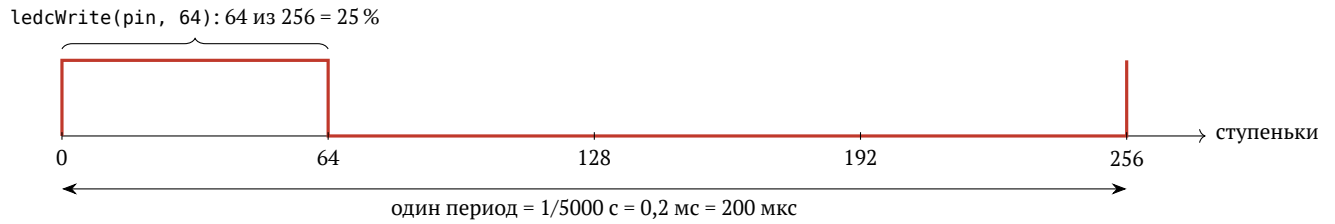


Рис. 42: 8-битный ШИМ: период разделён на 256 ступенек. Значение duty говорит, сколько из них вывод включён

В Arduino-ESP32 для этого есть две функции: `ledcAttach(pin, частота, биты)` — вместо `pinMode()`, и `ledcWrite(pin, duty)` — вместо `digitalWrite()`.

6.6 Светодиод с линейно меняющейся яркостью

Сделаем «дыхание»: пусть значение duty равномерно растёт от 0 до 255, а потом так же равномерно падает.

Листинг 8: «Дыхание» светодиода: duty меняется линейно

```
const int LED_PIN = 4;
const uint32_t FREQ = 5000; // Гц
const uint8_t RES = 8; // бит -> значения 0..255

void setup() {
  ledcAttach(LED_PIN, FREQ, RES);
}

void loop() {
  for (int d = 0; d <= 255; d++) { ledcWrite(LED_PIN, d); delay(4); }
  for (int d = 255; d >= 0; d--) { ledcWrite(LED_PIN, d); delay(4); }
}
```

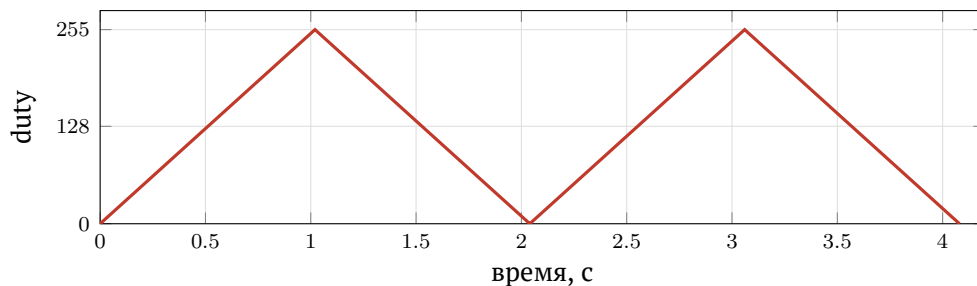


Рис. 43: Как меняется duty в программе: 256 шагов по 4 мс вверх, столько же вниз — «пила» с периодом около 2 с

Здесь два цикла `for`: первый считает вверх (`d++`), второй — вниз (`d--`), начиная с 255 и пока `d >= 0`. Каждые 4 мс duty меняется на единицу. За это время ШИМ успевает сделать $5000 \cdot 0,004 = 20$ периодов — глаз видит не мигание, а плавное изменение яркости.

6.7 Почему «линейно» выглядит нелинейно

Запусти пример и присмотри. Светодиод очень быстро становится ярким, а потом долго почти не меняется — хотя duty растёт равномерно! Дело снова в глазе.

Глаз замечает не саму яркость, а *во сколько раз* она изменилась. Разница между 1 и 2 лампочками огромна (вдвое!), а между 100 и 101 — незаметна. Поэтому в тёмной части нужны очень маленькие шаги, а в яркой — большие. Исправляется это **гамма-коррекцией**: вместо равномерного роста duty возводим «желаемую яркость» в степень 2,2:

$$\text{duty} = 255 \cdot \left(\frac{\text{яркость в \%}}{100} \right)^{2,2}.$$

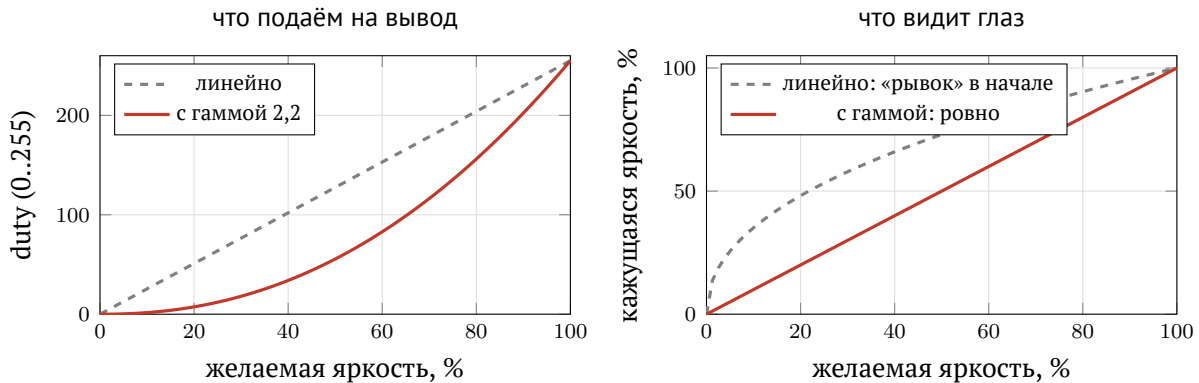


Рис. 44: Гамма-коррекция: на вывод подаём «изогнутую» кривую, чтобы глаз увидел ровное изменение яркости

6.8 Учимся программировать: свои функции

Формулу гамма-коррекции придётся использовать много раз: для каждого значения яркости. Чтобы не переписывать её снова и снова, оформим её как **свою функцию**. Функция — это «машинка»: у неё есть имя, на вход ей передают данные (**параметры**), а на выходе она **возвращает** результат. Написал один раз — пользуешься сколько угодно, как новым словом в языке.

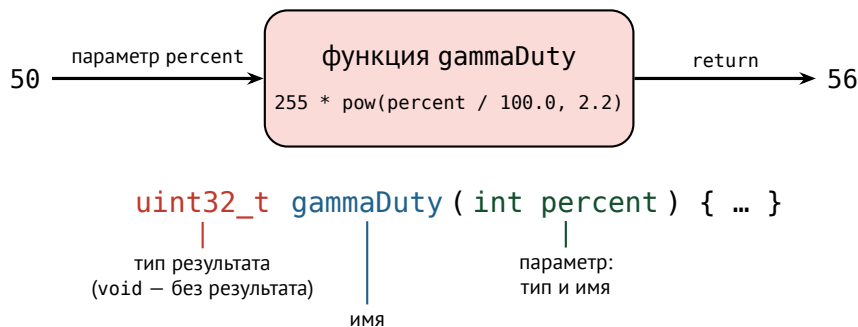


Рис. 45: Функция получает яркость в процентах и возвращает значение duty: 50 % → 56

- Слово `return` («вернуть») отдаёт результат туда, откуда функцию вызвали. Поэтому `ledcWrite(LED_PIN, gammaDuty(p))` сначала вычислит `gammaDuty(p)`, а потом подставит результат.
- Функции `setup()` и `loop()` имеют тип `void` («пустота»): они ничего не возвращают.
- Почему в формуле `100.0`, а не `100`? Помнишь целочисленное деление из урока 4? `50 / 100` даст 0, а `50 / 100.0` — правильные 0,5.

Совет

Хорошая функция делает *одно* дело, и её имя говорит, какое: `gammaDuty()`, `blinkTwice()`, `isButtonPressed()`. Тогда программа читается почти как обычный текст.

Листинг 9: «Дыхание» с гамма-коррекцией

```
// «Дыхание» с поправкой на глаз (гамма-коррекция)
const int LED_PIN = 4;

uint32_t gammaDuty(int percent) {           // 0..100 % -> 0..255
    return (uint32_t)(255 * pow(percent / 100.0, 2.2) + 0.5);
}

void setup() {
    ledcAttach(LED_PIN, 5000, 8);
}

void loop() {
    for (int p = 0; p <= 100; p++) { ledcWrite(LED_PIN, gammaDuty(p)); delay(10); }
    for (int p = 100; p >= 0; p--) { ledcWrite(LED_PIN, gammaDuty(p)); delay(10); }
}
```

Функция `pow(x, y)` возводит x в степень y . Сравни два примера: теперь светодиод разгорается и гаснет плавно по всей длине.

Это интересно

Глаз способен видеть и при свете луны, и на солнечном пляже, где света в миллион раз больше. Такой огромный диапазон возможен как раз потому, что зрение реагирует на *отношения*, а не на разницу. Слух устроен так же, поэтому громкость звука измеряют в децибелах — логарифмической шкале. Этот закон восприятия называют законом Вебера–Фехнера.

6.9 Какую частоту ШИМ выбрать

Частота и разрешение ШИМ связаны: блок LEDC работает от внутреннего «метронома» 80 МГц, и за один период ШИМ он должен успеть отсчитать все ступеньки. Чем выше частота, тем меньше ступенек помещается в период.

Применение	Частота	Разрешение
Светодиоды	5 kHz	8–13 бит
Сервопривод (урок 11)	50 Hz	14 бит
Моторчик через драйвер	10 – 20 kHz	10 бит
Звук (пищалка)	100 – 5000 Hz	8 бит

Таблица 12: Типичные настройки ШИМ

Совет

Если выбрать для светодиода частоту всего 50–100 Гц, глаз не заметит мерцания, если смотреть прямо. Но стоит быстро перевести взгляд или снять светодиод на камеру телефона — появятся полосы и «пунктир». Поэтому для света берут тысячи герц.

6.10 ШИМ и звук

Звук — это колебания воздуха. Если подключить к выводу **пьезопищалку** и включать-выключать его, например, 440 раз в секунду, пластинка внутри пищалки будет колебаться с той же частотой, и мы услышим ноту ля первой октавы. Частота определяет **высоту звука**: чем больше герц, тем выше нота.

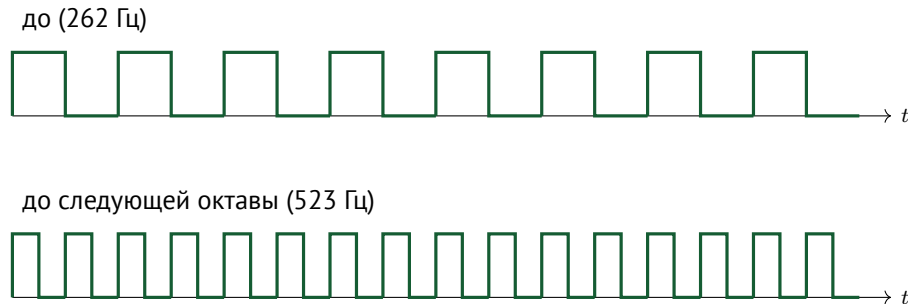
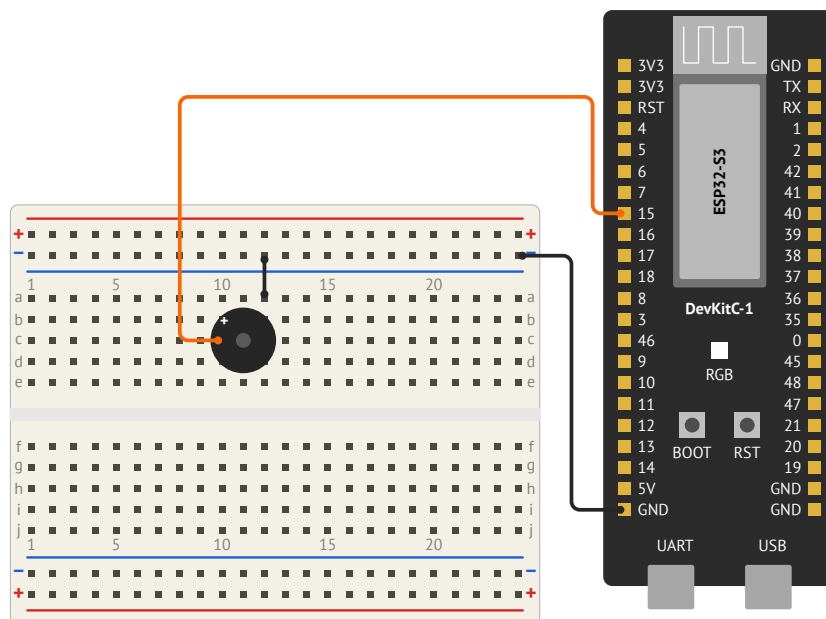
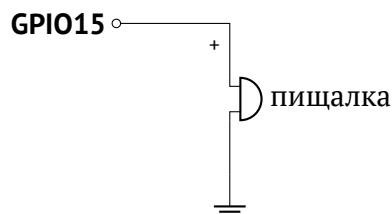


Рис. 46: Нота на октаву выше — это ровно вдвое большая частота

Для звука используют функцию `ledcWriteTone(pin, частота)`: она сама настраивает ШИМ с коэффициентом заполнения 50 % и нужной частотой. Подключи *пассивную* пищалку (без встроенного генератора) к **GPIO15**:



а) Монтажная схема: как собрать на макетной плате



б) Принципиальная электрическая схема

Рис. 47: Пьезопищалка на выводе GPIO15

Листинг 10: Гамма «до-ре-ми» на пищалке

```
// Пищалка играет гамму «до-ре-ми-фа-соль-ля-си-до»
const int BUZZER_PIN = 15;
const int NOTES[] = {262, 294, 330, 349, 392, 440, 494, 523}; // частоты, Гц

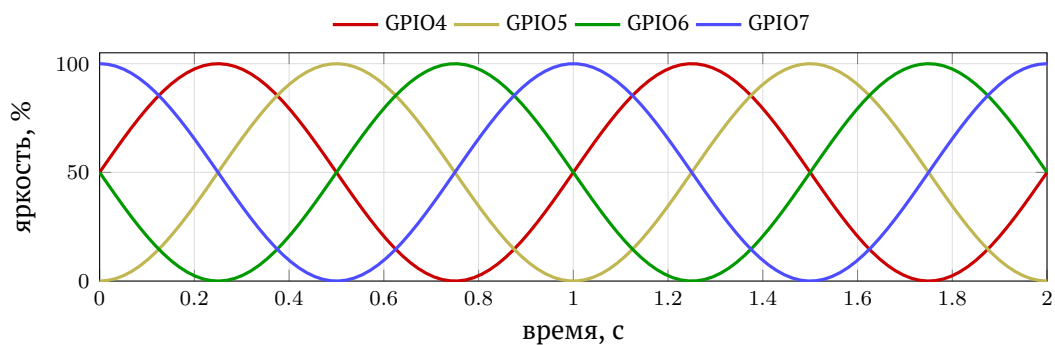
void setup() {
  ledcAttach(BUZZER_PIN, 1000, 8);
}

void loop() {
  for (int i = 0; i < 8; i++) {
    ledcWriteTone(BUZZER_PIN, NOTES[i]); // квадратная волна нужной частоты
    delay(300);
  }
  ledcWriteTone(BUZZER_PIN, 0); // 0 Гц -- тишина
  delay(1500);
}
```

до	ре	ми	фа	соль	ля	си	до
262	294	330	349	392	440	494	523

Таблица 13: Частоты нот первой октавы, Гц. Каждая следующая октава — вдвое выше**6.11 Практика: плавная волна**

Вернёмся к схеме из четырёх светодиодов (рис. 35). Теперь у каждого будет свой ШИМ, и яркость будет меняться по синусоиде — той самой волне из урока 2. Если сдвинуть волну каждого следующего светодиода на четверть периода, по ряду побежит мягкая световая волна.

**Рис. 48:** Яркость четырех светодиодов: одна и та же волна, сдвинутая на четверть периода**Листинг 11:** Световая волна

```
// Плавная «волна» на четырех светодиодах
const int LEDS[] = {4, 5, 6, 7};
const int COUNT = 4;

uint32_t gammaDuty(int percent) {
  return (uint32_t)(255 * pow(percent / 100.0, 2.2) + 0.5);
}
```

```
void setup() {  
  for (int i = 0; i < COUNT; i++) {  
    ledcAttach(LEDS[i], 5000, 8);           // каждому светодиоду -- свой ШИМ  
  }  
}  
  
void loop() {  
  float t = millis() / 1000.0;             // время в секундах  
  for (int i = 0; i < COUNT; i++) {  
    // синусоида, сдвинутая для каждого светодиода на четверть периода  
    float s = sin(2 * PI * (t - i * 0.25)); // от -1 до 1  
    int percent = (s + 1) * 50;             // от 0 до 100  
    ledcWrite(LEDS[i], gammaDuty(percent));  
  }  
  delay(10);  
}
```

Итоги урока

- Из-за инерции зрения вспышки чаще ≈ 50 раз в секунду сливаются в ровный свет.
- ШИМ меняет долю времени, пока вывод включён (коэффициент заполнения), и этим — среднюю яркость.
- `ledcAttach(pin, freq, bits) + ledcWrite(pin, duty)`; при 8 битах duty от 0 до 255.
- Глаз воспринимает яркость нелинейно, поэтому для плавности нужна гамма-коррекция.
- Частота определяет, заметно ли мерцание, а для звука — высоту ноты.
- Своя функция: тип результата, имя, параметры; `return` возвращает результат.

Проверь себя

1. Почему при частоте мигания 100 Гц мы видим ровный свет?
2. Чему равен коэффициент заполнения, если вывод включён 150 мкс из периода 200 мкс?
3. Какое значение duty соответствует 50 % при 10-битном разрешении?
4. Какая частота у ноты «ля» второй октавы, если у «ля» первой — 440 Гц?

Задания для самостоятельной работы

1. Сделай «дыхание» в два раза медленнее, не меняя `delay()`. Подсказка: шаг.
2. Напиши функцию `void blinkTimes(int n)`, которая мигает светодиодом `n` раз, и вызови её с разными числами.
3. Попробуй функцию `ledcFade(pin, от, до, мс)`: она меняет яркость плавно и полностью сама.
4. Сыграй на пищалке «В траве сидел кузнечик» или любую другую мелодию.
5. Установи частоту ШИМ 30 Гц и посмотри на «дыхание». Что изменилось и почему?

Урок 7. Радио: Wi-Fi и Bluetooth

Цель	понять, что такое радиоволны, как плата подключается к Wi-Fi и что такое MAC- и IP-адреса
Опирается на	урок 2 (частота, биты), урок 4 (сообщения в Serial Monitor)
Практика	плата ищет сети, подключается к роутеру, сама становится точкой доступа и получает имя esp32-led.local
Понадобится	Wi-Fi сеть на 2,4 ГГц (или телефон в режиме точки доступа)

Светодиоды мигают, пищалка пищит — пора перейти к самому интересному. Главная «суперсила» ESP32-S3, за которую его выбирают инженеры по всему миру, — **встроенное радио**. Чип умеет работать с **Wi-Fi** и **Bluetooth** без дополнительных модулей. Значит, любое наше устройство может управляться с телефона, отправлять данные в интернет и общаться с другими платами.

7.1 Что такое радиоволны

Радиоволны — это **электромагнитные волны**, как и свет. Отличаются они только *частотой*: сколько раз в секунду колеблется волна. Мы уже знаем, что частоту измеряют в герцах. Для радио получаются огромные числа, поэтому используют приставки: 1 клогерц (кГц) = 1000 Гц, 1 мегагерц (МГц) = миллион герц, 1 гигагерц (ГГц) = миллиард герц.

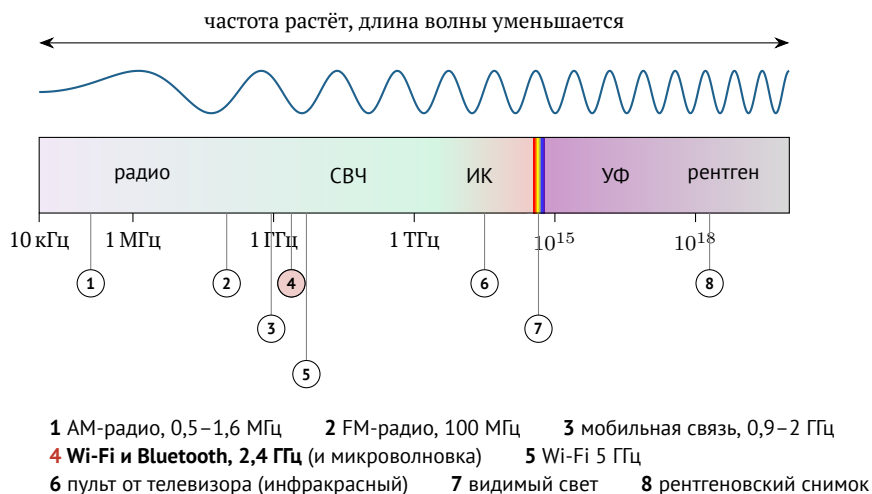


Рис. 49: Шкала электромагнитных волн (частоты отложены так, что каждое деление больше предыдущего в тысячу раз). Видимый свет занимает крошечную полоску, а Wi-Fi — соседи микроволновки

Все электромагнитные волны летят со скоростью света $c = 300\,000$ км/с. Зная частоту, можно найти **длину волны** — расстояние между соседними «гребнями»:

$$\lambda = \frac{c}{f} = \frac{300\,000\,000\text{ м/с}}{2\,400\,000\,000\text{ Гц}} = 0,125\text{ м} = 12,5\text{ см.}$$

Антенна обычно делается длиной в четверть волны — около 3 см. Посмотри на зигзаг на модуле ESP32-S3: если его «распрямить», получится примерно такая длина!

Это интересно

Микроволновая печь греет еду волнами почти той же частоты, что и Wi-Fi, — 2,45 ГГц. Только мощность у неё около 800 Вт, а у Wi-Fi платы — меньше 0,1 Вт, в десять тысяч раз слабее. Поэтому работающая микроволновка может мешать Wi-Fi, но Wi-Fi никого не нагреет.

7.2 Wi-Fi и его каналы

Wi-Fi — это способ передавать цифровые данные по радио. Биты «прячутся» в радиоволне: передатчик очень быстро меняет её параметры, а приёмник расшифровывает изменения обратно в нули и единицы.

Диапазон 2,4 ГГц поделён на **каналы**, как дорога — на полосы. Если две сети работают на одном канале, они мешают друг другу. Каналы шириной 20 МГц частично перекрываются, поэтому не мешают друг другу только каналы 1, 6 и 11:

номера каналов — под осью; не перекрываются 1, 6 и 11

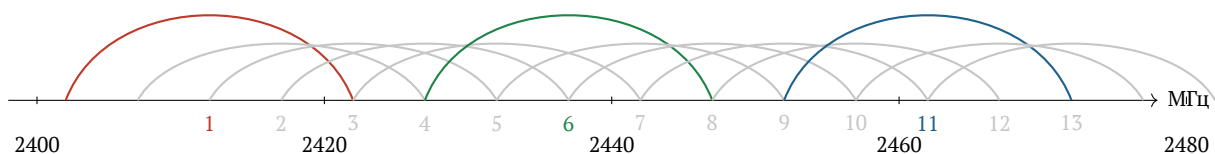


Рис. 50: Каналы Wi-Fi в диапазоне 2,4 ГГц

Начнём с того, что посмотрим, какие сети «видит» плата. Эта программа выводит имя каждой сети, её канал и силу сигнала:

Листинг 12: Поиск Wi-Fi сетей

```
// Какие Wi-Fi сети видит плата?
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA); // режим «клиент»
  Serial.print("MAC-адрес платы: ");
  Serial.println(WiFi.macAddress());
}

void loop() {
  Serial.println("Ищу сети...");
  int n = WiFi.scanNetworks(); // занимает пару секунд
  for (int i = 0; i < n; i++) {
    Serial.printf("%2d) %-20s канал %2d, сигнал %4d дБм, %s\n",
                  i + 1, WiFi.SSID(i).c_str(), WiFi.channel(i), WiFi.RSSI(i),
                  WiFi.encryptionType(i) == WIFI_AUTH_OPEN ? "открытая" : "с паролем");
  }
  WiFi.scanDelete(); // освободить память
  delay(10000);
}
```

Сила сигнала (RSSI) измеряется в **дБм** (децибел-милливаттах) и всегда отрицательна: чем ближе к нулю, тем сильнее сигнал.

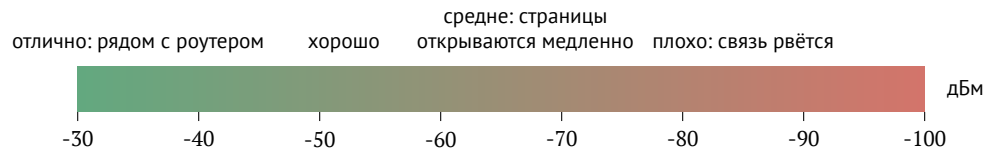


Рис. 51: Как понимать силу сигнала RSSI

7.3 Bluetooth

Bluetooth работает в том же диапазоне 2,4 ГГц, но создан для другого: связывать *близкие* устройства (наушники, часы, телефон) и при этом тратить очень мало энергии. ESP32-S3 поддерживает **Bluetooth Low Energy** (BLE, «Bluetooth с низким энергопотреблением»).

	Wi-Fi	Bluetooth LE
Дальность	десятки метров	10–30 метров
Скорость	миллионы бит в секунду	сотни тысяч бит в секунду
Потребление	большое (до 300 мА в пике)	маленькое (можно годами работать от батареек)
Нужен роутер?	обычно да	нет, устройства связываются напрямую
Для чего	интернет, веб-страницы, видео	датчики, фитнес-браслеты, умные замки, наушники

Таблица 14: Wi-Fi и Bluetooth LE: одна частота — разные задачи

Устройства BLE постоянно «кричат» в эфир короткие сообщения: «Я тут, меня зовут так-то». Послушаем их:

Листинг 13: Поиск Bluetooth-устройств поблизости

```
// Какие Bluetooth-устройства рядом?
#include <BLEDevice.h>

void setup() {
  Serial.begin(115200);
  BLEDevice::init(""); // включить Bluetooth
}

void loop() {
  Serial.println("Слушаю эфир 5 секунд...");
  BLEScan* scan = BLEDevice::getScan();
  scan->setActiveScan(true); // просить устройства назвать себя
  BLEScanResults* found = scan->start(5, false);
  for (int i = 0; i < found->getCount(); i++) {
    BLEAdvertisedDevice d = found->getDevice(i);
    Serial.printf("%5s сигнал %4d дБм %s\n", d.getAddress().toString().c_str(),
                  d.getRSSI(), d.getName().c_str());
  }
  scan->clearResults();
  delay(5000);
}
```

Скорее всего, ты увидишь телефоны, наушники и часы соседей — а у некоторых даже имена. Дальше в курсе мы будем работать с Wi-Fi, а Bluetooth можно изучить самостоятельно по примерам в File → Examples → BLE.

7.4 Два режима Wi-Fi: клиент и точка доступа

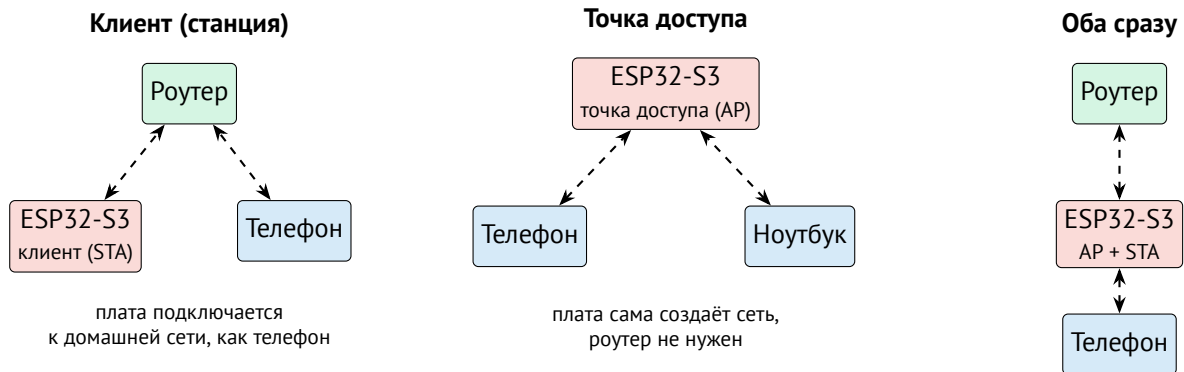


Рис. 52: Режимы Wi-Fi. STA — от англ. *station* (станция), AP — *access point* (точка доступа)

Пример 1: плата — клиент

В этом режиме ESP32-S3 подключается к твоему роутеру, как телефон или ноутбук. Впиши в программу имя своей сети и пароль:

Листинг 14: Подключение к Wi-Fi сети

```
// Подключаемся к домашней сети как клиент (станция)
#include <WiFi.h>

const char* SSID = "MyNetwork";           // имя твоей сети
const char* PASS = "MyPassword";         // пароль

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(SSID, PASS);
  Serial.print("Подключаюсь");
  uint32_t start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < 15000) {
    delay(300);
    Serial.print(".");
  }
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("\nНе получилось. Проверь имя и пароль сети.");
    return;
  }
  Serial.println("\nГотово!");
  Serial.printf("MAC-адрес: %s\n", WiFi.macAddress().c_str());
  Serial.printf("IP-адрес: %s\n", WiFi.localIP().toString().c_str());
  Serial.printf("Маска: %s\n", WiFi.subnetMask().toString().c_str());
  Serial.printf("Роутер: %s\n", WiFi.gatewayIP().toString().c_str());
  Serial.printf("Сигнал: %d дБм\n", WiFi.RSSI());
}

void loop() {}
```

Учимся программировать: цикл `while` и логическое «и»

Подключение к сети занимает несколько секунд, и заранее неизвестно, сколько именно. Цикл `for` здесь не подходит — он для случаев, когда число повторений известно. Нужен цикл

while («пока»): он повторяет команды, *пока* условие верно.

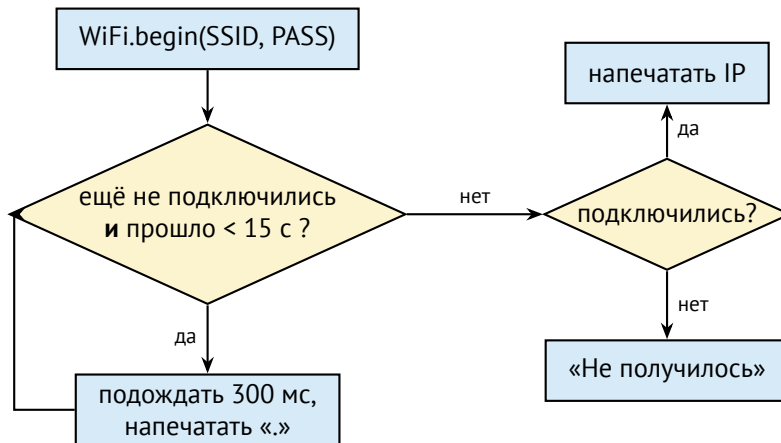


Рис. 53: Блок-схема подключения: ждём в цикле, но не дольше 15 секунд

В условии цикла два вопроса соединены знаком && — логическим «и»: цикл продолжается, только если верны *оба* условия. Как только плата подключилась *или* время вышло, цикл заканчивается. Без второго условия плата могла бы ждать вечно, если пароль неверный.

Осторожно

Если условие while никогда не станет ложным, программа застрянет в цикле навсегда. Проверь, что рано или поздно цикл закончится. Выйти из любого цикла досрочно можно командой `break`, а из функции — командой `return` (так сделано в программе, если подключиться не удалось).

Осторожно

ESP32-S3 работает только в диапазоне 2,4 ГГц. Сеть 5 ГГц он не увидит. Если роутер раздаёт обе сети с одинаковым именем, иногда помогает включить на телефоне точку доступа и подключить плату к нему.

Пример 2: плата — точка доступа

Если роутера нет (например, в походе или на соревнованиях роботов), плата может сама создать Wi-Fi сеть. Найди в списке сетей телефона ESP32-S3, введи пароль 12345678 — и в Serial Monitor появится сообщение о подключении:

Листинг 15: Своя точка доступа

```

// Плата сама создаёт Wi-Fi сеть (точка доступа)
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_AP); // режим «точка доступа»
  WiFi.softAP("ESP32-S3", "12345678"); // имя сети и пароль (не короче 8 символов)
  Serial.print("Сеть создана. Адрес платы: ");
  Serial.println(WiFi.softAPIP()); // обычно 192.168.4.1
  Serial.printf("MAC точки доступа: %s\n", WiFi.softAPmacAddress().c_str());
}
  
```

```

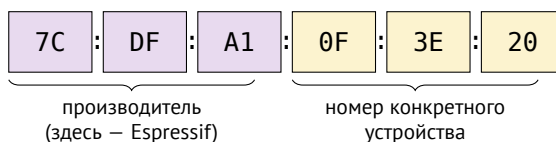
void loop() {
    static int last = -1;
    int n = WiFi.softAPgetStationNum();           // сколько устройств подключилось
    if (n != last) {
        last = n;
        Serial.printf("Подключено устройств: %d\n", n);
    }
    delay(500);
}

```

7.5 MAC-адрес и IP-адрес

Когда в сети много устройств, данные нужно доставлять точно по адресу. Для этого у каждого устройства есть *два* адреса.

MAC-адрес — «номер паспорта»



6 байт, записаны в шестнадцатеричной системе. Дается на заводе, у всех разный.

IP-адрес — «почтовый адрес»



4 числа от 0 до 255. Выдает роутер, когда плата подключается к сети.

Рис. 54: Два адреса каждого сетевого устройства

- **MAC-адрес** — постоянный «номер паспорта» сетевого устройства. Его записывают на заводе, и он не меняется при переезде в другую сеть. Роутер по MAC-адресу узнаёт устройство «в лицо»: например, можно запретить подключение незнакомым устройствам или выдать своему всегда один и тот же IP.
- **IP-адрес** — «почтовый адрес» в конкретной сети. В домашней сети он обычно выглядит как 192.168.1.xx или 192.168.0.xx. Переедешь в другую сеть (например, в школьную) — адрес будет другим, как у человека при переезде.

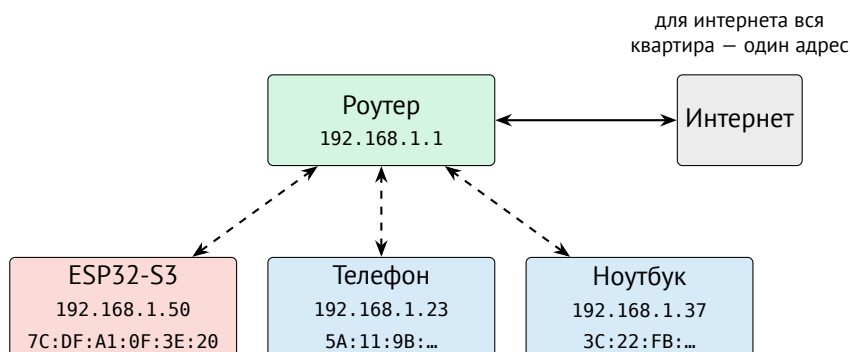


Рис. 55: Домашняя сеть: у каждого устройства свой IP-адрес внутри сети и свой MAC-адрес

Кто раздаёт IP-адреса

Когда плата подключается, она не знает своего IP. Она громко спрашивает всю сеть, и роутер выдаёт ей свободный адрес. Этот механизм называется **DHCP**:

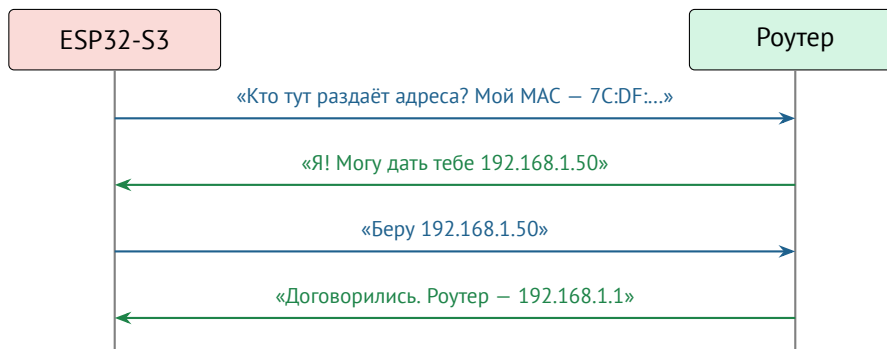


Рис. 56: Как плата получает IP-адрес (DHCP)

Неудобство в том, что при следующем включении роутер может выдать *другой* адрес. Решений три:

1. попросить роутер всегда выдавать этому MAC-адресу один и тот же IP (в настройках роутера);
2. задать **постоянный (статический) IP** прямо в программе;
3. вообще не запоминать IP, а обращаться к плате **по имени** — об этом ниже.

Как задать IP, MAC и имя в программе

Листинг 16: Постоянный IP-адрес, свой MAC и имя в сети

```

// Постоянный IP-адрес, свой MAC-адрес и имя esp32-led.local
#include <WiFi.h>
#include <ESPmDNS.h>
#include <esp_wifi.h>

const char* SSID = "MyNetwork";
const char* PASS = "MyPassword";

IPAddress ip(192, 168, 1, 50);           // адрес, который мы хотим получить
IPAddress gateway(192, 168, 1, 1);       // адрес роутера
IPAddress subnet(255, 255, 255, 0);     // маска: «своя» сеть -- 192.168.1.*
IPAddress dns(192, 168, 1, 1);          // кто переводит имена сайтов в адреса

uint8_t myMac[6] = {0x02, 0x12, 0x34, 0x56, 0x78, 0x9A}; // «самодельный» MAC

void setup() {
  Serial.begin(115200);
  WiFi.setHostname("esp32-led");        // имя в списке устройств роутера
  WiFi.mode(WIFI_STA);
  esp_wifi_set_mac(WIFI_IF_STA, myMac); // по желанию: сменить MAC
  WiFi.config(ip, gateway, subnet, dns); // постоянный IP вместо выданного роутером
  WiFi.begin(SSID, PASS);
  while (WiFi.status() != WL_CONNECTED) {
    delay(300);
    Serial.print(".");
  }
}
  
```

```

Serial.printf("\nMAC: %s, IP: %s\n",
              WiFi.macAddress().c_str(), WiFi.localIP().toString().c_str());

if (MDNS.begin("esp32-led")) {           // включить имя esp32-led.local
    Serial.println("Плата отвечает на имя esp32-led.local");
}
}

void loop() {}

```

- `WiFi.config(ip, gateway, subnet, dns)` задаёт IP вручную. Выбирай адрес из своей сети (первые три числа — как у роутера) и такой, который точно никем не занят. Если адрес совпадёт с чужим, оба устройства будут работать с ошибками.
- `esp_wifi_set_mac()` меняет MAC-адрес. Обычно этого делать не нужно; первый байт `0x02` означает «адрес назначен вручную», так он не совпадёт ни с одним заводским.
- `WiFi.setHostname()` — имя, которое будет видно в списке устройств на странице роутера.

7.6 Имя вместо адреса: mDNS

Запоминать `192.168.1.50` неудобно. В интернете эту задачу решают имена сайтов: мы пишем `wikipedia.org`, а специальный сервер DNS переводит имя в IP-адрес. В домашней сети есть похожий механизм — **mDNS** (англ. *multicast DNS*). Устройство само отвечает на своё имя, которое всегда оканчивается на `.local`. Достаточно одной строки `MDNS.begin("esp32-led")`, и плата станет доступна как `esp32-led.local`.

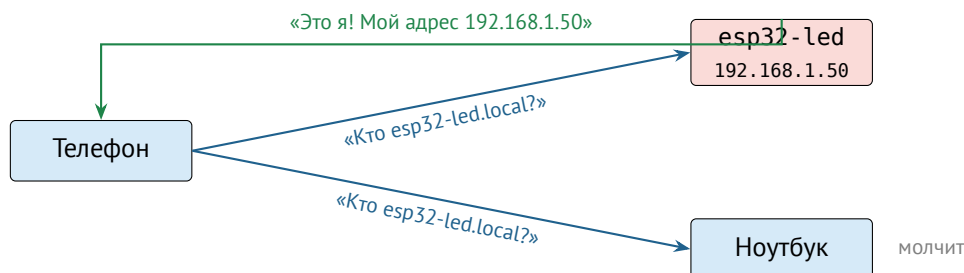


Рис. 57: mDNS: вопрос отправляется всем устройствам сразу, отвечает только то, чьё это имя

Совет

Имена `.local` понимают Windows 10/11, macOS, iPhone и Linux. Некоторые старые Android-телефоны mDNS не поддерживают — тогда придётся ввести IP-адрес.

Итоги урока

- Wi-Fi и Bluetooth — радиоволны частотой 2,4 ГГц, длина волны около 12 см.
- Wi-Fi — быстрый и «дальнобойный», Bluetooth LE — экономный, для близких устройств.
- Плата может быть клиентом (подключаться к роутеру) или точкой доступа (создавать свою сеть).
- MAC-адрес — постоянный заводской номер, IP-адрес — адрес в конкретной сети (выдаёт роутер через DHCP).
- mDNS позволяет обращаться к плате по имени `.local` вместо IP-адреса.
- Цикл `while` повторяет команды, пока условие верно; `&&` — «и», оба условия должны быть верны.

Проверь себя

1. Чем радиоволна Wi-Fi отличается от видимого света?
2. Какова длина волны Wi-Fi 5 ГГц?
3. Что изменится — MAC или IP, — если принести плату в гости к другу и подключить к его сети?
4. Какой сигнал сильнее: -45 дБм или -80 дБм?

Задания для самостоятельной работы

1. Запусти поиск сетей и составь «карту» каналов в своём доме. Какой канал самый свободный?
2. Обойди квартиру с платой и ноутбуком: как меняется RSSI в разных комнатах?
3. Если за 15 секунд не удалось подключиться к роутеру, пусть плата переходит в режим точки доступа.
4. Мигай светодиодом на **GPIO4** тем быстрее, чем сильнее сигнал Wi-Fi.

Урок 8. Веб-сервер: HTTP, HTML и CSS

Цель	понять, как браузер разговаривает с сервером (запросы GET и POST), и сделать красивую веб-страницу для управления светодиодом
Опирается на	урок 6 (яркость через ШИМ), урок 7 (подключение к Wi-Fi, IP-адрес, mDNS)
Практика	панель управления светодиодом в браузере телефона: кнопка, ползунок и «лампочка» на экране
Понадобится	схема со светодиодом на GPIO4 (рис. 30), телефон или компьютер в той же Wi-Fi сети

8.1 Клиент и сервер

Когда ты открываешь сайт, твой браузер (**клиент**) отправляет **запрос** другому компьютеру (**серверу**), а тот присылает **ответ** — страницу. Это похоже на кафе: посетитель делает заказ, официант передаёт его на кухню и приносит блюдо. Сегодня кухней будет наша плата: ESP32-S3 станет **веб-сервером**, а браузер телефона — клиентом.

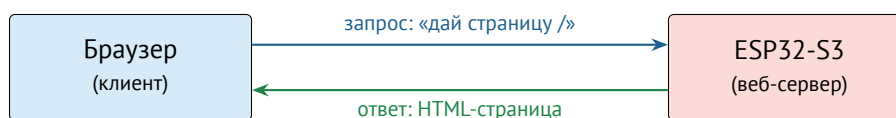


Рис. 58: Клиент спрашивает — сервер отвечает. Сам сервер никогда не начинает разговор первым

Язык, на котором они разговаривают, называется **HTTP** (англ. *HyperText Transfer Protocol* — протокол передачи гипертекста). Начнём с самой простой страницы:

Листинг 17: Первый веб-сервер

```

// Самый простой веб-сервер: страница «Привет!»
#include <WiFi.h>
#include <WebServer.h>
#include <ESPmDNS.h>

const char* SSID = "MyNetwork";
const char* PASS = "MyPassword";

WebServer server(80); // 80 -- стандартный «порт» для HTTP

void handleRoot() { // кто-то открыл адрес "/"
  String html = "<h1>Привет! Я ESP32-S3</h1>";
  html += "<p>Я работаю уже " + String(millis() / 1000) + " секунд.</p>";
  server.send(200, "text/html; charset=utf-8", html); // 200 = «всё хорошо»
}

void handleNotFound() { // адрес, которого у нас нет
  server.send(404, "text/plain; charset=utf-8", "Такой страницы нет");
}

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  
```

```

WiFi.begin(SSID, PASS);
while (WiFi.status() != WL_CONNECTED) { delay(300); Serial.print("."); }
Serial.printf("\nОткрой http://%s или http://esp32.local\n",
              WiFi.localIP().toString().c_str());
MDNS.begin("esp32");

server.on("/", handleRoot);           // адрес -> функция-обработчик
server.onNotFound(handleNotFound);
server.begin();
}

void loop() {
  server.handleClient();               // ответить браузеру, если он что-то спросил
}

```

Загрузи программу, дождись сообщения в Serial Monitor и открой адрес в браузере телефона, подключённого к той же сети. Обнови страницу несколько раз — число секунд будет расти: страницу каждый раз заново «готовит» плата.

- `WebServer server(80)` — создать сервер. 80 — номер «двери» (порта), в которую браузер стучится по умолчанию;
- `server.on("/", handleRoot)` — «если спросят адрес /, вызови `handleRoot`»;
- `server.send(200, тип, текст)` — отправить ответ. 200 — код «всё хорошо»;
- `server.handleClient()` — проверить, не пришёл ли запрос. Вызываем постоянно в `loop()`.

8.2 Из чего состоит адрес

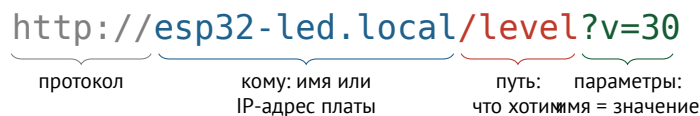


Рис. 59: Адрес (URL) состоит из частей. Параметров может быть несколько: `?v=30&color=red`

8.3 Методы GET и POST

Каждый запрос начинается со слова-**метода**, который говорит, чего хочет клиент. Самые важные:

- **GET** («получить») — «дай мне страницу». Параметры записываются прямо в адресе, после знака ?. Так работают обычные ссылки и адресная строка браузера;
- **POST** («отправить») — «прими от меня данные». Данные передаются в теле запроса, отдельно от адреса. Так отправляют формы: логин и пароль, сообщение, заказ.

GET — как открытка

```
GET /level?v=30 HTTP/1.1
Host: esp32-led.local
(тела нет)
```

данные видны прямо в адресе,
их можно сохранить в закладки

POST — как письмо в конверте

```
POST /note HTTP/1.1
Host: esp32.local
Content-Type: application/x-www-form-urlencoded
text=Привет, плата!
```

данные в теле запроса (красным),
в адресной строке их не видно

Рис. 60: Так выглядят запросы на самом деле — это обычный текст

	GET	POST
Для чего	получить страницу или данные	отправить данные, что-то изменить
Где данные	в адресе: /level?v=30	в теле запроса
Видно в адресной строке	да	нет
Можно добавить в закладки	да	нет
Повтор при обновлении страницы	безопасен	браузер спросит «отправить повторно?»
Объём данных	небольшой (адрес ограничен по длине)	большой (хоть целый файл)

Таблица 15: Сравнение GET и POST

Совет

Правило хорошего тона: **GET** — только чтобы что-то *узнать*, **POST** — чтобы что-то *изменить*. Иначе поисковый робот или «ускоритель» браузера, заранее открывающий ссылки, может случайно выключить тебе свет!

Сервер отвечает **кодом ответа** — трёхзначным числом:

Код	Значение	Когда встречается
200	OK — всё хорошо	страница найдена и отправлена
303	See Other — смотри другую	«я всё сделал, теперь открой главную»
404	Not Found — не найдено	адреса нет — знаменитая «ошибка 404»
500	Internal Server Error	ошибка в программе сервера

Таблица 16: Самые частые коды ответа HTTP

Пример: управление через GET

Самый простой способ управлять светодиодом — ссылки. Нажатие на ссылку /toggle переключает светодиод, а форма с ползунком отправляет /level?v=...

Листинг 18: Светодиод управляется GET-запросами

```
// Управляем светодиодом через GET-запросы: /toggle и /level?v=...
#include <WiFi.h>
#include <WebServer.h>
#include <ESPmDNS.h>

const char* SSID = "MyNetwork";           // имя твоей Wi-Fi сети
const char* PASS = "MyPassword";         // и пароль
```

```

const int LED_PIN = 4;

WebServer server(80);
bool ledOn = false;
int level = 50;                                // яркость, %

uint32_t gammaDuty(int percent) {
    return (uint32_t)(255 * pow(percent / 100.0, 2.2) + 0.5);
}

void applyLed() { ledcWrite(LED_PIN, ledOn ? gammaDuty(level) : 0); }

String page() {                                // собираем HTML-страницу
    String html = "<!DOCTYPE html><html><head><meta charset='utf-8'>"
        "<meta name='viewport' content='width=device-width'>"
        "<title>Светодиод</title></head><body style='font-family:sans-serif'>";
    html += "<h1>Светодиод: ";
    html += ledOn ? "ВКЛ" : "ВЫКЛ";
    html += "</h1><p><a href='/toggle'><button>Вкл/выкл</button></a></p>"
        "<form action='/level'>Яркость: <input type='range' name='v' "
        "min='0' max='100' value='" + String(level) + "' "
        "onchange='this.form.submit()'></form></body></html>";
    return html;
}

void goHome() {                                // «перейди на главную страницу»
    server.sendHeader("Location", "/");
    server.send(303);
}

void setup() {
    Serial.begin(115200);
    ledcAttach(LED_PIN, 5000, 8);
    applyLed();

    WiFi.mode(WIFI_STA);
    WiFi.begin(SSID, PASS);
    while (WiFi.status() != WL_CONNECTED) { delay(300); Serial.print("."); }
    Serial.printf("\nОткрой http://%s или http://esp32-led.local\n",
        WiFi.localIP().toString().c_str());
    MDNS.begin("esp32-led");

    server.on("/", []() { server.send(200, "text/html; charset=utf-8", page()); });
    server.on("/toggle", []() { ledOn = !ledOn; applyLed(); goHome(); });
    server.on("/level", []() {
        level = constrain(server.arg("v").toInt(), 0, 100);
        applyLed();
        goHome();
    });
    server.begin();
}

void loop() {
    server.handleClient();                      // ответить браузеру, если он что-то спросил
}

```

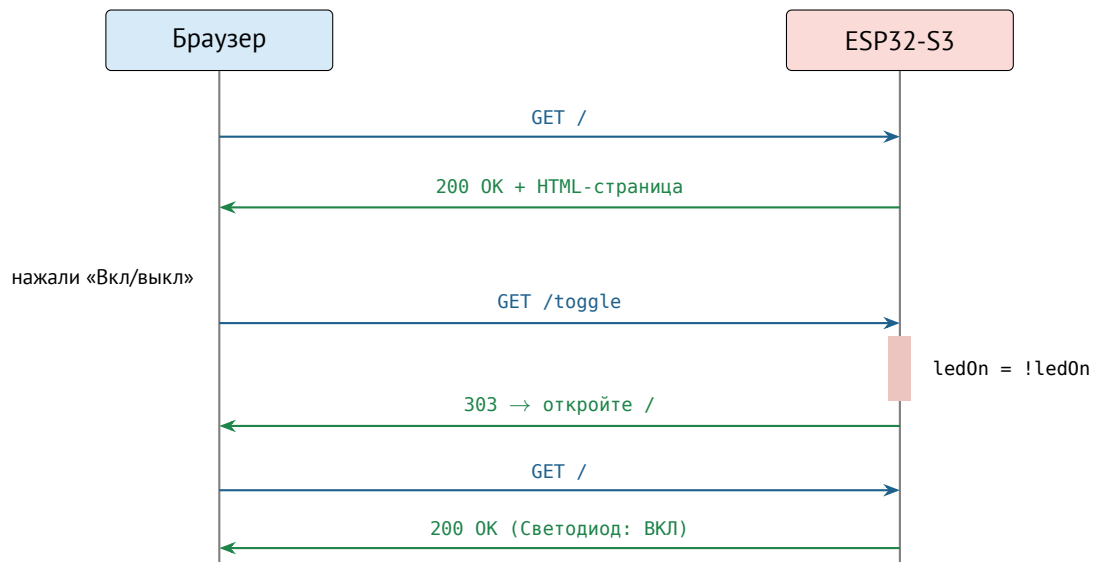


Рис. 61: Что происходит при нажатии кнопки на странице

Работает, но у этого способа два минуса: команды меняют состояние через GET (см. совет выше), а страница каждый раз полностью перезагружается и мигает. Исправим оба.

Пример: отправка данных через POST

Сделаем «записку для платы»: форма отправляет текст методом POST, плата печатает его в Serial Monitor и показывает на странице.

Листинг 19: Приём данных методом POST

```

// Записка для платы: форма отправляет текст методом POST
#include <WiFi.h>
#include <WebServer.h>
#include <ESPmDNS.h>

const char* SSID = "MyNetwork";
const char* PASS = "MyPassword";

WebServer server(80);
String note = "(записок пока нет)"; // последняя полученная записка

void handleRoot() { // GET / -- показать страницу с формой
    String html = "<!DOCTYPE html><html><head><meta charset='utf-8'></head><body>"
        "<h1>Записка для ESP32</h1>"
        "<p>Последняя записка: <b>" + note + "</b></p>"
        "<form method='POST' action='/note'>"
        "<input name='text' placeholder='Напиши что-нибудь'>"
        "<button>Отправить</button></form></body></html>";
    server.send(200, "text/html; charset=utf-8", html);
}

void handleNote() { // POST /note -- принять записку
    note = server.arg("text"); // данные из тела запроса
    Serial.println("Новая записка: " + note);
    server.setHeader("Location", "/"); // вернуться на главную
    server.send(303);
}
  
```

```

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(SSID, PASS);
  while (WiFi.status() != WL_CONNECTED) { delay(300); Serial.print("."); }
  Serial.printf("\nОткрой http://%s или http://esp32.local\n",
               WiFi.localIP().toString().c_str());
  MDNS.begin("esp32");

  server.on("/", HTTP_GET, handleRoot);
  server.on("/note", HTTP_POST, handleNote); // этот адрес принимает только POST
  server.begin();
}

void loop() {
  server.handleClient();
}

```

В форме указано `method='POST'`, а на плате обработчик зарегистрирован с `HTTP_POST`: на обычный GET-запрос по адресу `/note` он не ответит. Текст записки достаётся функцией `server.arg("text")` — так же, как параметры GET, только данные пришли из тела запроса.

8.4 HTML: из чего сделана страница

Страница, которую отправляет сервер, написана на языке **HTML** (англ. *HyperText Markup Language* — язык разметки гипертекста). Это не язык программирования: он описывает, *что* есть на странице. Каждый элемент обозначается **тегами** в угловых скобках: открывающим `<p>` и закрывающим `</p>`. Теги вкладываются друг в друга, как матрёшки:

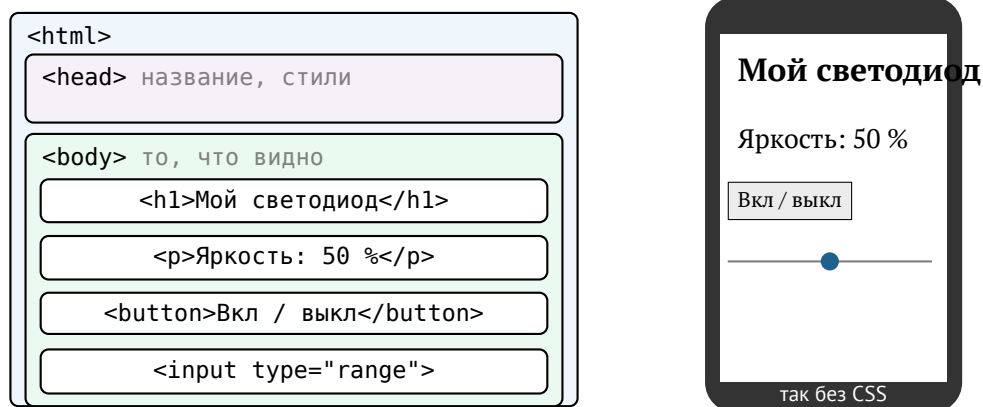


Рис. 62: Структура HTML-страницы (слева) и как браузер её показывает (справа)

Тег	Что означает
<code><h1>...</h1></code>	крупный заголовок (есть ещё h2–h6 поменьше)
<code><p>...</p></code>	абзац текста
<code>...</code>	жирный текст
<code>...</code>	ссылка
<code><button>...</button></code>	кнопка
<code><input type="range"></code>	ползунок (type="text" — поле для текста)
<code><form>...</form></code>	форма: собирает поля и отправляет на сервер
<code><div>...</div></code>	«коробка» для группировки элементов

Таблица 17: Теги, которых хватит для первых страниц

8.5 CSS: делаем красиво

HTML отвечает за содержание, а за *внешний вид* — цвета, шрифты, отступы, скругления, тени — отвечает язык **CSS** (англ. *Cascading Style Sheets* — каскадные таблицы стилей). Стили пишут внутри тега `<style>`. Каждое правило состоит из *селектора* (к чему применить) и списка свойств:

селектор: все кнопки свойство значение

```
button { background: #1f618d; border-radius: 12px; }
```

читается так: «у всех кнопок фон цвета #1f618d и скруглённые на 12 пикселей углы»

Рис. 63: Устройство правила CSS

Цвета в CSS часто записывают шестнадцатеричным кодом `#RRGGBB`: две цифры на красный, зелёный и синий — та же смесь трёх цветов, что в RGB-светодиоде из урока 5! Например, `#ff0000` — красный, `#ffffff` — белый.

Свойство CSS	Что меняет
color, background	цвет текста и фона
font-family, font-size	шрифт и его размер
padding, margin	внутренний и внешний отступ
border-radius	скругление углов (50% — круг)
box-shadow	тень или свечение вокруг элемента
width, height	ширина и высота
transition	плавная смена стиля (анимация)

Таблица 18: Свойства CSS для первых экспериментов

8.6 Практика: панель управления светодиодом

Соберём всё вместе: HTML-страницу с CSS-стилями и маленькой программой на **JavaScript** — языке, который выполняется прямо в браузере. Он будет отправлять POST-запросы без *перезагрузки страницы* и перерисовывать «лампочку» на экране.



Рис. 64: Как будет выглядеть панель на телефоне

Листинг 20: Панель управления: HTML + CSS + JavaScript

```
// Красивая панель управления светодиодом: HTML + CSS + немного JavaScript
#include <WiFi.h>
#include <WebServer.h>
#include <ESPmDNS.h>

const char* SSID = "MyNetwork";
const char* PASS = "MyPassword";
const int LED_PIN = 4;

WebServer server(80);
bool ledOn = false;
int level = 50;                                // яркость, %

// Вся страница -- одна большая строка. R"rawliteral( ... )rawliteral"
// позволяет писать кавычки и переносы строк без лишних значков.
const char PAGE[] = R"rawliteral(
<!DOCTYPE html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Светодиод</title>
<style>
  body { font-family: sans-serif; background: #eef2f7;
        display: flex; justify-content: center; padding-top: 40px; }
  .card { background: white; border-radius: 20px; padding: 24px 32px; width: 260px;
        box-shadow: 0 6px 20px rgba(0,0,0,.12); text-align: center; }
  h1    { color: #7b241c; font-size: 24px; }
  .led { width: 90px; height: 90px; border-radius: 50%; margin: 10px auto 20px;
        background: #ccc; transition: .3s; }
  .led.on { background: #ff5a4f; box-shadow: 0 0 30px #ff5a4f; }
  button { background: #1f618d; color: white; border: none; border-radius: 12px;
        font-size: 18px; padding: 10px 28px; }
  input { width: 100%; margin-top: 20px; }
```

```

</style>
</head>
<body>
<div class="card">
  <h1>Мой светодиод</h1>
  <div id="led" class="led"></div>
  <button onclick="send('toggle')">Вкл / выкл</button>
  <input id="lvl" type="range" min="0" max="100" onchange="send('level=' +
  ↪ this.value)">
  <p>Яркость: <b id="txt">?</b> %</p>
</div>
<script>
function show(s) {
  // нарисовать состояние светодиода
  document.getElementById('led').className = s.on ? 'led on' : 'led';
  document.getElementById('lvl').value = s.level;
  document.getElementById('txt').textContent = s.level;
}
function send(cmd) {
  // POST-запрос без перезагрузки страницы
  fetch('/api', { method: 'POST', body: new URLSearchParams(cmd) })
    .then(r => r.json()).then(show);
}
fetch('/api').then(r => r.json()).then(show); // GET: узнать состояние при открытии
</script>
</body>
</html>
)rawliteral";

uint32_t gammaDuty(int percent) {
  return (uint32_t)(255 * pow(percent / 100.0, 2.2) + 0.5);
}

void applyLed() { ledcWrite(LED_PIN, ledOn ? gammaDuty(level) : 0); }

void handleApi() {
  if (server.method() == HTTP_POST) {
    // POST -- команда изменить состояние
    if (server.hasArg("toggle")) ledOn = !ledOn;
    if (server.hasArg("level")) {
      level = constrain(server.arg("level").toInt(), 0, 100);
      ledOn = true;
    }
    applyLed();
  }
  // и на GET, и на POST отвечаем текущим состоянием в формате JSON
  String json = String("{\"on\":") + (ledOn ? "true" : "false") +
    ", \"level\": " + level + "}";
  server.send(200, "application/json", json);
}

void setup() {
  Serial.begin(115200);
  ledcAttach(LED_PIN, 5000, 8);
  applyLed();
  WiFi.mode(WIFI_STA);
  WiFi.begin(SSID, PASS);
  while (WiFi.status() != WL_CONNECTED) { delay(300); Serial.print("."); }
  Serial.printf("\nОткрой http://%s или http://esp32-led.local\n",
    WiFi.localIP().toString().c_str());
  MDNS.begin("esp32-led");
}

```

```

server.on("/", []) { server.send(200, "text/html; charset=utf-8", PAGE); };
server.on("/api", handleApi);
server.begin();
}

void loop() {
  server.handleClient();
}

```

Как это работает:

1. При открытии `http://esp32-led.local` браузер делает GET / и получает страницу PAGE.
2. Скрипт на странице сразу делает GET /api. Плата отвечает состоянием в формате JSON — `{"on":true,"level":60}`. Это текст, который легко читают и программы, и люди.
3. При нажатии кнопки функция `fetch()` отправляет POST /api с телом `toggle=`, а при движении ползунка — `level=60`. Плата меняет яркость и снова отвечает JSON.
4. Функция `show()` перекрашивает «лампочку» и обновляет число. Страница не перезагружается.

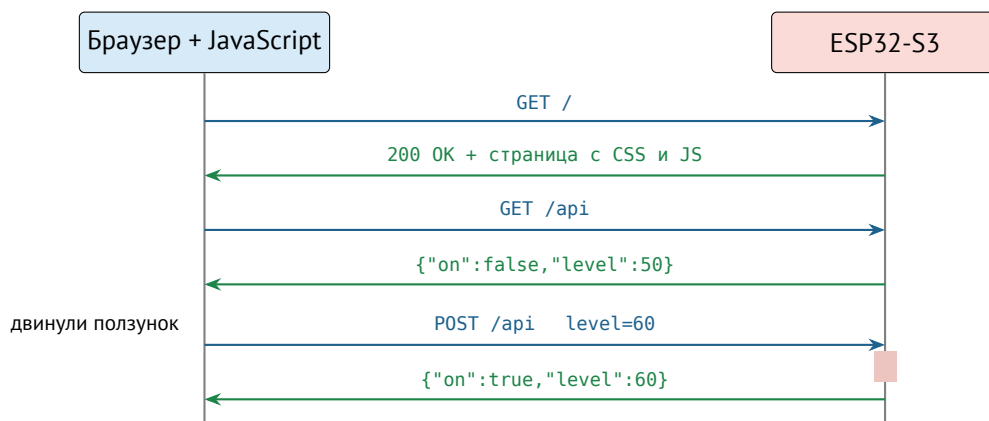


Рис. 65: Страница загружается один раз, дальше идут только маленькие запросы к /api

Совет

Хранить страницу в строке PAGE удобно, пока она небольшая. Для больших сайтов с картинками файлы записывают в файловую систему платы (LittleFS) — это хорошая тема для самостоятельного изучения.

Итоги урока

- Браузер (клиент) отправляет запросы, ESP32-S3 (сервер) отвечает. Язык разговора — HTTP.
- GET — получить (параметры в адресе), POST — отправить данные (в теле запроса).
- Ответ сервера содержит код: 200 — успех, 303 — перейди, 404 — не найдено.
- HTML описывает содержание страницы, CSS — её внешний вид, JavaScript — поведение.
- JSON — удобный формат для обмена данными между страницей и платой.

Проверь себя

1. Какой метод и какой путь в адресе `http://esp32.local/set?color=red`? Какой параметр?
2. Почему пароль нельзя отправлять методом GET?
3. Что означает код ответа 404?
4. Какое CSS-свойство делает квадрат кругом?

Задания для самостоятельной работы

1. Поменяй цвета панели на свои любимые. Сделай «лампочку» зелёной.
2. Добавь на панель кнопки «30 %», «60 %» и «100 %».
3. Добавь адрес `/api/uptime`, который возвращает JSON со временем работы платы, и покажи его на странице.
4. Запусти панель в режиме точки доступа (урок 7), чтобы она работала без роутера.

Урок 9. Последовательный порт: разговор с компьютером

Цель	понять, как биты путешествуют по одному проводу, и научиться принимать команды с компьютера
Опирается на	урок 2 (биты и байты), урок 4 (<code>Serial.print</code>), урок 6 (яркость)
Практика	светодиод слушается команд <code>on</code> , <code>off</code> и <code>level 30</code> ; игра «Угадай число»
Понадобится	схема со светодиодом на GPIO4 (рис. 30); для последнего раздела — модуль GPS или вторая плата (по желанию)

С самого первого урока мы пользуемся Serial Monitor: плата присылает сообщения, и мы их читаем. Пора разобраться, как это устроено, и научиться разговаривать в обратную сторону — отправлять плате команды.

9.1 Как передать букву по одному проводу

Внутри компьютера каждая буква — это число. Например, латинская «А» — это 65, а в двоичном виде 01000001. Эти 8 бит (1 байт) можно передать по одному проводу друг за другом, как в уроке 2. Такой способ называют **последовательной передачей**, а устройство внутри чипа, которое этим занимается, — **UART** (англ. *Universal Asynchronous Receiver-Transmitter*, универсальный асинхронный приёмопередатчик).

Слово «асинхронный» значит, что у передатчика и приёмника нет общего «метронома». Поэтому они заранее договариваются о **скорости**: сколько бит в секунду передавать. Эту скорость называют **бодами**; мы используем 115 200 бод.



Рис. 66: Передача буквы «А». В покое линия в состоянии 1. Старт-бит (0) говорит «внимание, начинаю!», стоп-бит (1) — «закончил»

Это интересно

При скорости 115 200 бод на один символ уходит 10 бит (8 бит данных + старт + стоп), значит, за секунду можно передать 11 520 символов — примерно 5 страниц этой книги.

9.2 Serial и Serial Monitor

В ESP32-S3 три UART. В Arduino им соответствуют объекты:

Объект Arduino	Блок	Выводы	Для чего
Serial	USB или UART0	разъём USB / 43, 44	связь с компьютером
Serial1	UART1	любые, задаются в <code>begin()</code>	свободен
Serial2	UART2	любые, задаются в <code>begin()</code>	свободен

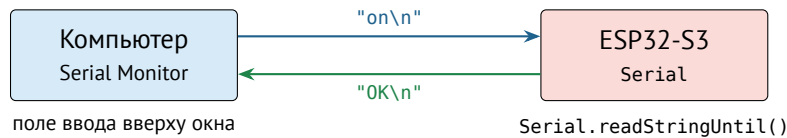


Рис. 67: Разговор в две стороны: команда от компьютера и ответ платы

Основные функции:

- `Serial.begin(115200)` — включить порт; скорость в Serial Monitor должна совпадать;
- `Serial.print()`, `println()`, `printf()` — отправить текст;
- `Serial.available()` — сколько байт пришло и ждёт чтения;
- `Serial.readStringUntil('\n')` — прочитать строку до символа «новая строка».

Совет

В Serial Monitor выбери окончание строки New Line: тогда каждая команда будет заканчиваться символом `\n`, и `readStringUntil()` сразу поймёт, где конец команды.

9.3 Практика: светодиод слушается команд

Используем схему со светодиодом на **GPIO4** (рис. 30) и ШИМ из урока 6. Функция `handleSerial()` ничего не ждёт: если команды нет, она сразу возвращается, и `loop()` может заниматься другими делами.

Листинг 21: Управление светодиодом командами

```

// Управляем светодиодом командами из Serial Monitor
const int LED_PIN = 4;

bool ledOn = false;
int level = 50; // яркость, 0..100 %

uint32_t gammaDuty(int percent) { // из урока про ШИМ
    return (uint32_t)(255 * pow(percent / 100.0, 2.2) + 0.5);
}

void applyLed() { // показать состояние на светодиоде
    ledcWrite(LED_PIN, ledOn ? gammaDuty(level) : 0);
}

void handleSerial() {
    if (!Serial.available()) return; // команд нет -- не ждём
    String cmd = Serial.readStringUntil('\n');
    cmd.trim(); // убрать лишние пробелы и \r
    if (cmd == "on") {
        ledOn = true;
    } else if (cmd == "off") {
        ledOn = false;
    } else if (cmd.startsWith("level ")) { // например, "level 30"
        level = constrain(cmd.substring(6).toInt(), 0, 100);
    } else if (cmd == "status") {
        Serial.printf("светодиод: %s, яркость %d%%\n", ledOn ? "вкл" : "выкл", level);
        return;
    } else {

```

```

    Serial.println("Команды: on, off, level N, status");
    return;
}
applyLed();
Serial.println("OK");
}

void setup() {
    Serial.begin(115200);
    ledcAttach(LED_PIN, 5000, 8);
    applyLed();
}

void loop() {
    handleSerial();
    // здесь может быть что-то ещё: handleSerial() ничего не ждёт
}

```

Попробуй: on, level 10, level 100, status, off.

9.4 Учимся программировать: строки и цепочка условий

Команда из Serial Monitor приходит как **строка** — переменная типа `String`. Со строками можно делать много полезного:

Запись	Что делает	Пример для cmd = "level 30"
cmd == "on"	сравнить строки	false
cmd.startsWith("level ")	начинается ли с этих букв	true
cmd.substring(6)	вырезать часть, начиная с символа № 6	"30"
cmd.substring(6).toInt()	превратить текст в число	30
cmd.length()	длина строки	8
cmd.trim()	убрать пробелы и переводы строки по краям	—
"уровень " + String(30)	склеить строки	"уровень 30"

Таблица 19: Полезные действия со строками `String`. Символы нумеруются с нуля, как ячейки массива

Чтобы выбрать одно действие из многих, условия выстраивают в **цепочку** `if — else if — else`. Они проверяются по очереди сверху вниз, и выполняется только *первая* подходящая ветка. Последний `else` ловит всё остальное — например, опечатки.

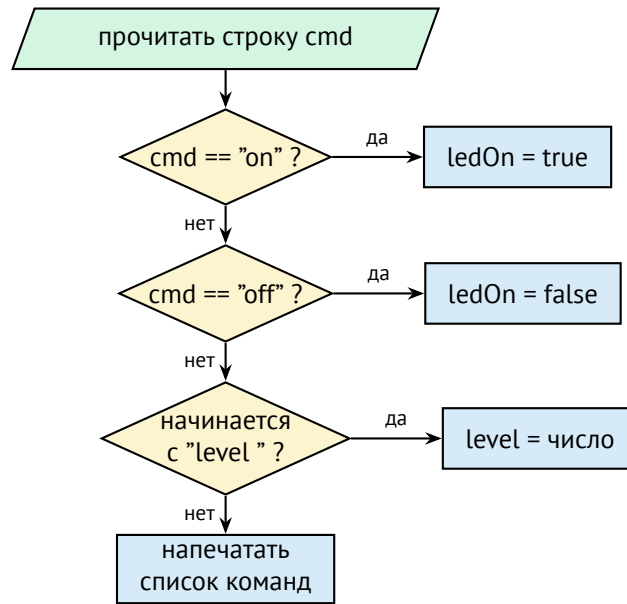


Рис. 68: Цепочка условий в функции `handleSerial()`: проверяем команды по очереди

Обрати внимание на `return` в начале функции: `if (!Serial.available()) return;` — «если данных нет, сразу выйти из функции». Такой ранний выход избавляет от лишних фигурных скобок.

9.5 Практика: игра «Угадай число»

Теперь, когда мы умеем читать ввод с компьютера, сделаем игру. Плата загадывает случайное число от 1 до 100, а ты угадываешь, вводя числа в поле Serial Monitor (окончание строки — New Line). Плата подсказывает «мало» или «много». Светодиод для игры не нужен.

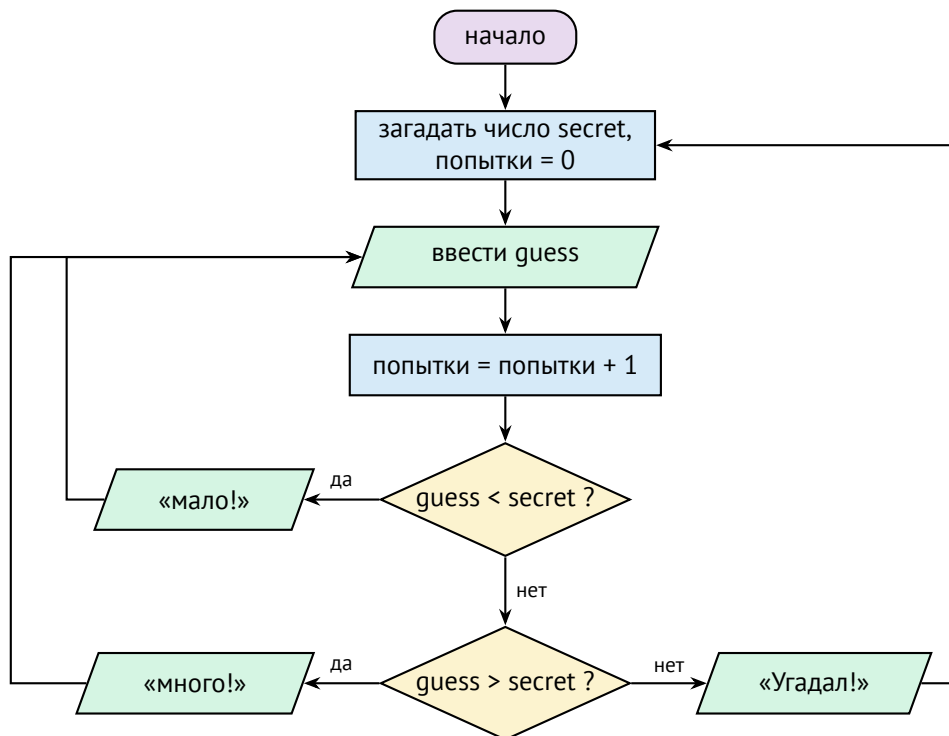


Рис. 69: Блок-схема игры «Угадай число»

Листинг 22: Игра «Угадай число»

```
// Игра «Угадай число»: плата загадывает число от 1 до 100
int secret; // загаданное число
int tries; // сколько было попыток

void newGame() {
    secret = random(1, 101); // случайное от 1 до 100
    tries = 0;
    Serial.println("Я загадала число от 1 до 100. Угадай!");
}

int readNumber() { // ждём, пока человек введёт число
    while (Serial.available() == 0) {
        delay(10);
    }
    int n = Serial.parseInt(); // прочитать число из строки
    while (Serial.available() > 0) Serial.read(); // выбросить остаток строки
    return n;
}

void setup() {
    Serial.begin(115200);
    delay(500);
    randomSeed(esp_random()); // чтобы числа каждый раз были разными
    newGame();
}

void loop() {
    int guess = readNumber();
    tries++;
    if (guess < secret) {
        Serial.printf("%d -- мало!\n", guess);
    } else if (guess > secret) {
        Serial.printf("%d -- много!\n", guess);
    } else {
        Serial.printf("Угадал за %d попыток! Сыграем ещё.\n", tries);
        newGame();
    }
}
```

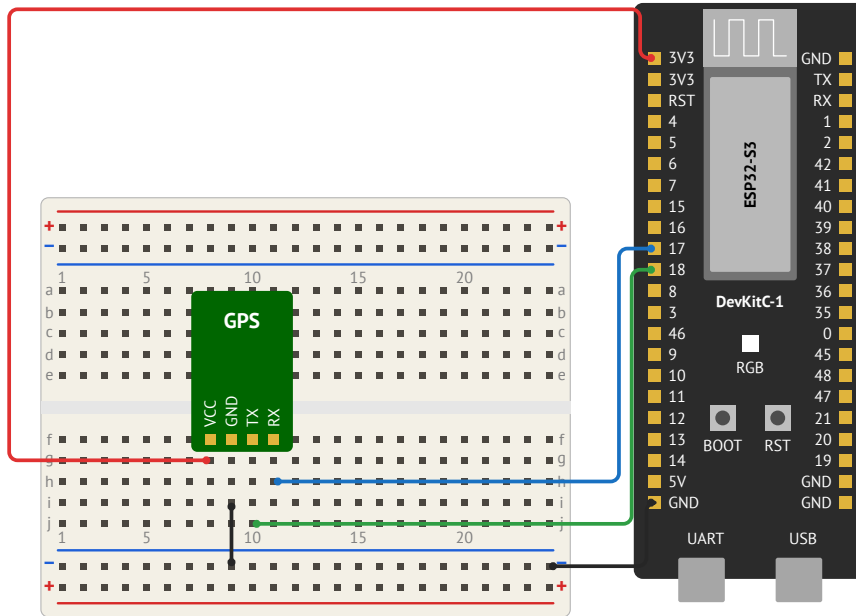
Здесь встретилось всё, что мы уже знаем: глобальные **переменные** `secret` и `tries`, **цепочка условий**, цикл `while`, который ждёт ввода, и **свои функции** `newGame()` и `readNumber()`. Новые готовые функции: `random(1, 101)` даёт случайное число от 1 до 100, а `Serial.parseInt()` читает из пришедших символов целое число.

Это интересно

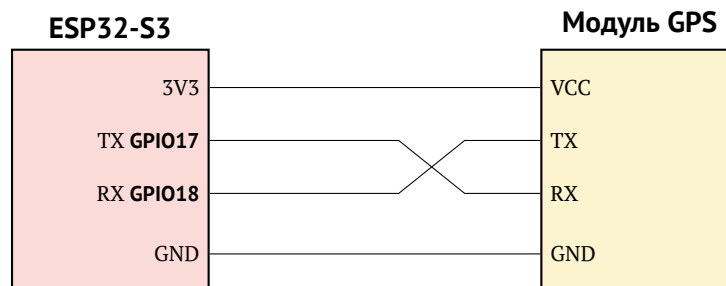
Если угадывать с умом — каждый раз называть середину оставшегося промежутка (50, потом 25 или 75...), — любое число от 1 до 100 угадывается не больше чем за 7 попыток, ведь $2^7 = 128 > 100$. Точно так же работает АЦП из урока 2: он «угадывает» напряжение за 12 шагов. Этот способ называется **двоичным поиском**.

9.6 Второй UART: связь с другими устройствами

На том же языке «разговаривают» GPS-приёмники, модули связи и другие микроконтроллеры. Благодаря матрице GPIO (урок 3) UART1 можно вывести на любые выводы, например **GPIO17** и **GPIO18**. Главное правило: **TX одного устройства соединяется с RX другого** (что один говорит — другой слушает), а земли объединяются.



а) Монтажная схема: как собрать на макетной плате



б) Принципиальная электрическая схема

Рис. 70: Подключение модуля GPS ко второму UART

Листинг 23: Мост: всё, что пришло с модуля, пересылается на компьютер, и наоборот

```
void setup() {  
  Serial.begin(115200);  
  Serial1.begin(9600, SERIAL_8N1, 18, 17);  
}  
  
void loop() {  
  while (Serial1.available()) Serial.write(Serial1.read());  
  while (Serial.available()) Serial1.write(Serial.read());  
}
```

Осторожно

Некоторые модули (часть Arduino, старые GPS) работают с логикой 5 В. Их TX нельзя подключать к RX ESP32-S3 напрямую — помнишь «опасную зону» из урока 2? Нужен делитель напряжения или преобразователь уровней.

Итоги урока

- UART передаёт байты по одному проводу: старт-бит, 8 бит данных, стоп-бит.
- Скорость (бод) у обеих сторон должна совпадать.
- Команды читаем, не останавливая программу: сначала `available()`, потом `readStringUntil()`.
- TX одного устройства — к RX другого, земли — вместе.
- Строки `String` можно сравнивать, резать и превращать в числа; цепочка `if/else if/else` выбирает одно действие из многих.

Проверь себя

1. Зачем нужен старт-бит?
2. Что будет, если в Serial Monitor выбрать скорость 9600, а в программе — 115 200?
3. Как соединить TX и RX двух плат?

Задания для самостоятельной работы

1. Добавь команду `blink N`: мигнуть светодиодом N раз (без `delay()`).
2. Раз в 10 секунд выводи время работы платы, не мешая приёму команд.
3. Соедини две платы по UART: команды, набранные на одной, пусть управляют светодиодом другой.
4. Измени игру: пусть плата отгадывает число, загаданное тобой, двоичным поиском. Ты отвечаешь «+», «-» или «=».

Урок 10. Цифровой вход: кнопки и прерывания

Цель	научиться читать кнопку, понять роль подтягивающего резистора, победитьдребезг, познакомиться с прерываниями
Опирается на	урок 2 (логические уровни), урок 5 (<code>millis()</code>), урок 6 (яркость)
Практика	кнопка переключает яркость светодиода: выкл → 30 % → 100 %
Понадобится	тактовая кнопка, светодиод с резистором 220 Ω

До сих пор выводы платы только *выдавали* сигналы. Теперь научимся их *читать*. Самое простое устройство ввода — кнопка: нажата или нет, 1 или 0. Идеальный цифровой сигнал!

10.1 Как устроена кнопка

Внутри тактовой кнопки две металлические пластинки. Когда кнопку нажимают, пластинки касаются, и цепь замыкается. У кнопки четыре ножки, но соединены они попарно: ножки на противоположных сторонах всегда связаны, а нажатие соединяет одну пару с другой.

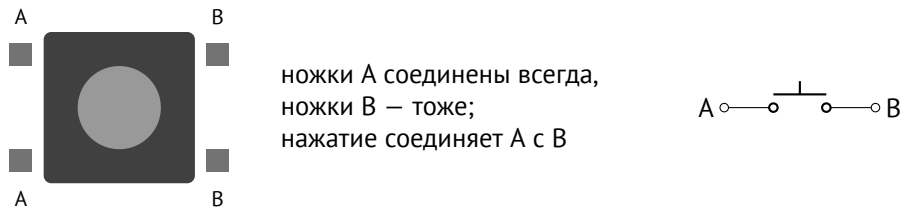


Рис. 71: Тактовая кнопка (вид снизу) и её обозначение на схеме

10.2 Проблема «висящего» входа

Кажется, всё просто: подключить кнопку между выводом и землёй. Нажали — на выводе 0. А если отпустили? Тогда вывод вообще ни к чему не подключён. Вход ESP32-S3 очень чувствительный: «висящий» провод работает как антенна и ловит наводки от рук, телефона, проводки. Программа будет читать то 0, то 1 — случайно.

Решение — **подтягивающий резистор**. Он слабо «тянет» вывод к 3,3 В, и в покое там уверенная 1. Нажатая кнопка соединяет вывод с землёй гораздо сильнее, чем резистор, — и получается 0.

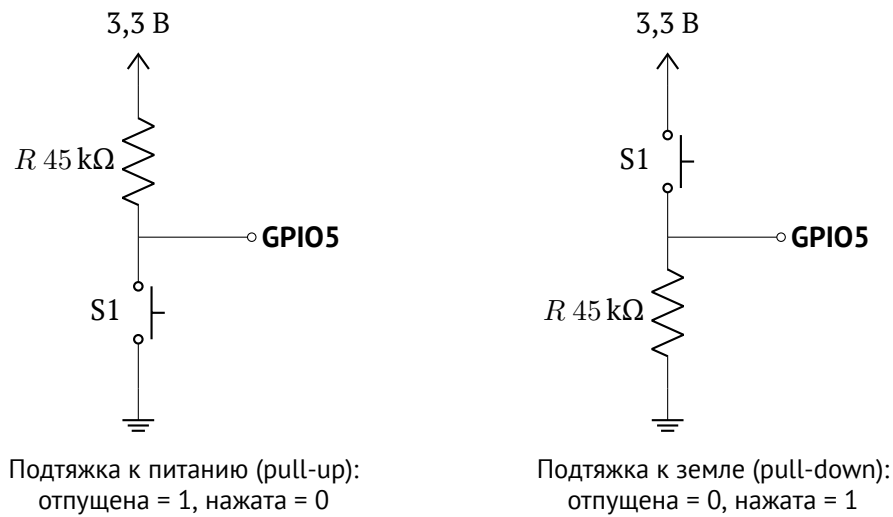
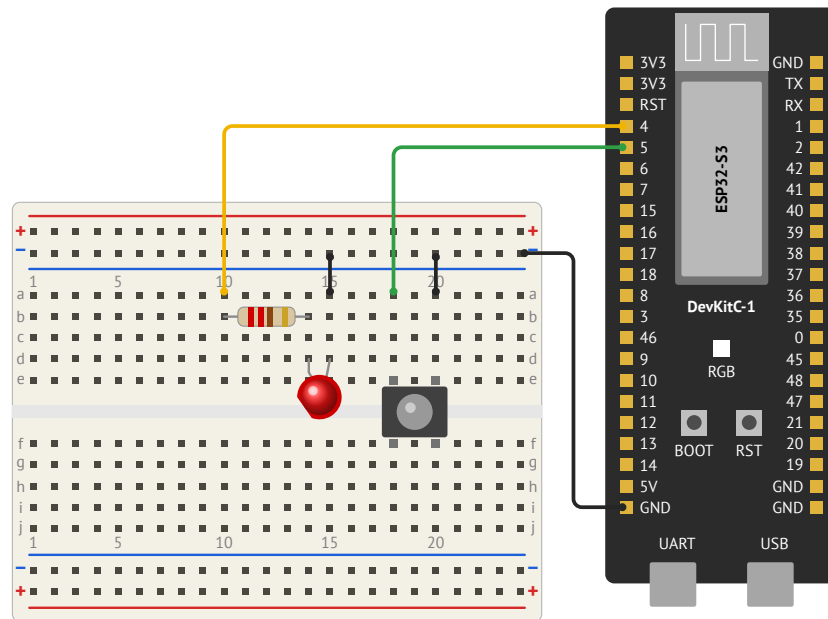


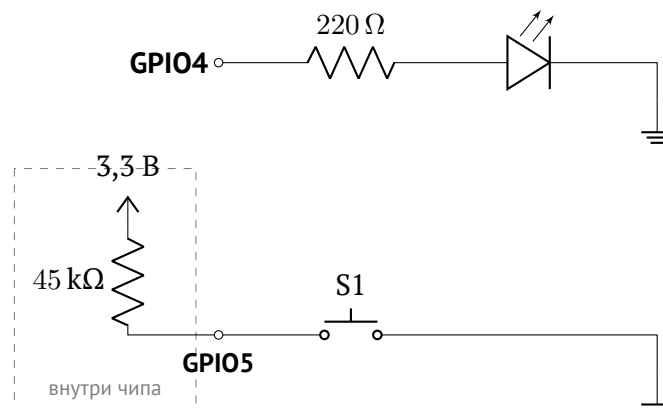
Рис. 72: Два способа подтяжки. У ESP32-S3 оба резистора уже есть внутри чипа, их включают программой. Мы будем использовать pull-up: кнопка замыкает вывод на землю

10.3 Схема подключения

Добавим к светодиоду на **GPIO4** кнопку на **GPIO5**. Внешний резистор для кнопки не нужен: включим встроенную подтяжку командой `pinMode(5, INPUT_PULLUP)`.



а) Монтажная схема: как собрать на макетной плате



б) Принципиальная электрическая схема

Рис. 73: Кнопка на GPIO5 и светодиод на GPIO4

На макетной плате кнопка стоит поперёк канавки. Зелёный провод от **GPIO5** подключён к одной её стороне, чёрный провод от другой стороны ведёт на шину земли.

10.4 Читаем кнопку

Листинг 24: Состояние кнопки в Serial Monitor

```
const int BTN_PIN = 5;

void setup() {
  Serial.begin(115200);
  pinMode(BTN_PIN, INPUT_PULLUP); // включаем встроенную подтяжку
}

void loop() {
  bool pressed = (digitalRead(BTN_PIN) == LOW); // pull-up: нажатие = LOW
  Serial.println(pressed ? "нажата" : "отпущена");
  delay(200);
}
```

```
}

```

Попробуй закомментировать INPUT_PULLUP и написать просто INPUT. Поднеси руку к проводу — увидишь, как «висящий» вход сходит с ума.

10.5 Дребезг контактов

Если просто написать «когда кнопка нажата — переключи светодиод», он будет переключаться много раз за одно нажатие. Виноват **дребезг**: металлические пластинки при ударе несколько раз «подпрыгивают», как мячик. Для человека это одно нажатие, а микроконтроллер, проверяющий вход миллионы раз в секунду, видит пачку нажатий.

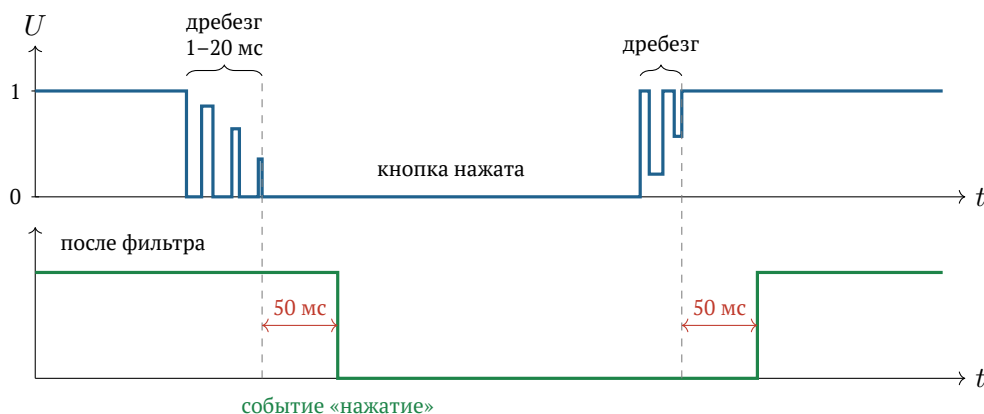


Рис. 74: Дребезг контактов (сверху) и сигнал после программного фильтра (снизу)

Программный фильтр верит новому уровню, только если тот продержался дольше 50 мс. Оформим его в функцию, которая возвращает true ровно один раз на каждое нажатие:

Листинг 25: Функция buttonPressed(): нажатие без дребезга

```
const int BTN_PIN = 5;
const uint32_t DEBOUNCE_MS = 50;

bool buttonPressed() {
    static bool stable = digitalRead(BTN_PIN); // подтверждённое состояние
    static bool last = stable;                 // последнее прочитанное
    static uint32_t changed = 0;               // когда уровень менялся
    bool r = digitalRead(BTN_PIN);
    if (r != last) {                          // уровень дёрнулся --
        last = r;                             // перезапускаем отсчёт
        changed = millis();
    }
    if (millis() - changed > DEBOUNCE_MS && r != stable) {
        stable = r;                          // уровень устоялся
        return stable == LOW;                // событие -- только нажатие
    }
    return false;
}

void setup() {
    Serial.begin(115200);
    pinMode(BTN_PIN, INPUT_PULLUP);
}
```

```
void loop() {
  if (buttonPressed()) Serial.println("Нажатие!");
}
```

10.6 Учимся программировать: логические операции и static

В функции `buttonPressed()` встретилось условие посложнее: `millis() - changed > DEBOUNCE_MS && r != stable`. Оно состоит из двух вопросов, соединённых **логической операцией**. Таких операций три:

Запись	Название	Когда верно
<code>A && B</code>	«и»	только если верны <i>оба</i> : «прошло 50 мс <i>и</i> уровень изменился»
<code>A B</code>	«или»	если верно <i>хотя бы одно</i> : «слишком холодно <i>или</i> слишком жарко»
<code>!A</code>	«не»	если A неверно: <code>!ledOn</code> — «светодиод не горит»

Таблица 20: Логические операции

A && B			A B		
A	B	результат	A	B	результат
false	false	false	false	false	false
false	true	false	false	true	true
true	false	false	true	false	true
true	true	true	true	true	true

Рис. 75: Таблицы истинности: результат «и» и «или» для всех сочетаний

Второе новое слово — `static`. Обычная локальная переменная создаётся заново при каждом вызове функции и забывает старое значение. А переменная со словом `static` создаётся один раз и *помнит* значение между вызовами — как блокнот, который функция носит с собой. Именно поэтому `buttonPressed()` помнит, каким был уровень кнопки в прошлый раз.

10.7 Практика: кнопка-переключатель яркости

Каждое нажатие переключает светодиод на следующую ступень: выключен → 30 % → 100 % → выключен. Так работают многие настольные лампы.

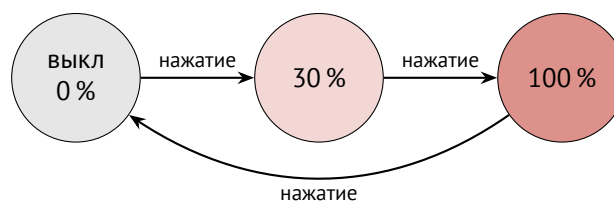


Рис. 76: Диаграмма состояний: кружки — состояния, стрелки — что их меняет

Листинг 26: Кнопка переключает яркость

```
// Кнопка переключает яркость: выкл -> 30 % -> 100 % -> выкл ...
const int LED_PIN = 4;
const int BTN_PIN = 5;
const int LEVELS[] = {0, 30, 100};           // ступени яркости, %

int mode = 0;                               // номер текущей ступени

uint32_t gammaDuty(int percent) {
    return (uint32_t)(255 * pow(percent / 100.0, 2.2) + 0.5);
}

bool buttonPressed() {                      // см. выше: с защитой от дребезга
    static bool stable = digitalRead(BTN_PIN);
    static bool last = stable;
    static uint32_t changed = 0;
    bool r = digitalRead(BTN_PIN);
    if (r != last) { last = r; changed = millis(); }
    if (millis() - changed > 50 && r != stable) {
        stable = r;
        return stable == LOW;
    }
    return false;
}

void setup() {
    Serial.begin(115200);
    pinMode(BTN_PIN, INPUT_PULLUP);
    ledcAttach(LED_PIN, 5000, 8);
    ledcWrite(LED_PIN, 0);
}

void loop() {
    if (buttonPressed()) {
        mode = (mode + 1) % 3;               // 0 -> 1 -> 2 -> 0
        ledcWrite(LED_PIN, gammaDuty(LEVELS[mode]));
        Serial.printf("Яркость: %d%%\n", LEVELS[mode]);
    }
}
```

10.8 Прерывания

Пока `loop()` крутится быстро, опрос кнопки работает отлично. Но если программа занята чем-то долгим, короткое нажатие можно пропустить. Выход — **прерывание**: попросить процессор *самого* вызвать нашу функцию в тот момент, когда уровень на выводе изменится. Как будильник: не нужно всё время смотреть на часы, он сам позовёт.



Рис. 77: Прерывание ненадолго приостанавливает основную программу

Листинг 27: Подсчёт нажатий через прерывание

```

const int BTN_PIN = 5;

volatile uint32_t pressCount = 0; // volatile: меняется в обработчике
volatile uint32_t lastIsrTime = 0;

void IRAM_ATTR onButton() { // IRAM_ATTR: код в быстрой памяти
    uint32_t now = millis();
    if (now - lastIsrTime > 50) { // грубое подавление дребезга
        pressCount++;
        lastIsrTime = now;
    }
}

void setup() {
    Serial.begin(115200);
    pinMode(BTN_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BTN_PIN), onButton, FALLING);
}

void loop() {
    static uint32_t shown = 0;
    if (pressCount != shown) { // печатаем здесь, а не в обработчике
        shown = pressCount;
        Serial.printf("Нажатий: %lu\n", shown);
    }
}

```

Осторожно

Внутри обработчика прерывания (ISR) нельзя вызывать `delay()`, `Serial.print()` и другие медленные функции. Правило: изменить переменную — и выйти. Всю остальную работу делает основная программа.

Совет

Кнопку не стоит подключать к «загрузочным» выводам **GPIO0**, **GPIO3**, **GPIO45**, **GPIO46** (урок 3): если при включении она окажется нажатой, плата может не запустить программу.

Итоги урока

- Вход без подтяжки «висит». Используем INPUT_PULLUP и кнопку на землю: нажата = LOW.
- Ловим *момент* нажатия и отфильтровываем дребезг (20–50 мс).
- Прерывание срабатывает мгновенно, но обработчик должен быть очень коротким.
- && — «и», || — «или», ! — «не»; static-переменная помнит значение между вызовами функции.

Проверь себя

1. Что прочитает программа с вывода с INPUT_PULLUP, если кнопка не нажата?
2. Почему одно нажатие может посчитаться несколько раз?
3. Чего нельзя делать внутри обработчика прерывания?

Задания для самостоятельной работы

1. Используй кнопку BOOT (**GPIO0**) как вторую кнопку — после того как программа уже запустилась.
2. Различай короткое (< 0,5 с) и длинное (> 1 с) нажатие: длинное выключает светодиод сразу.
3. Заменяй кнопку сенсорным входом: touchRead(2) растёт, если коснуться провода на **GPIO2**.
4. Сделай «реакциометр»: светодиод загорается в случайный момент, программа измеряет, как быстро ты нажмёшь кнопку.

Урок 11. АЦП: измеряем аналоговый сигнал

Цель	научиться измерять напряжение, понять ограничения АЦП ESP32-S3, управлять сервоприводом «крутилкой»
Опирается на	урок 2 (АЦП, квантование), урок 3 (выводы ADC1/ADC2), урок 6 (ШИМ)
Практика	потенциометр поворачивает сервопривод; вольтметр для батарейки
Понадобится	потенциометр 10 kΩ, сервопривод SG90, резисторы 100 kΩ и 33 kΩ, конденсатор 100 nF

Мы уже знаем, что АЦП превращает напряжение в число (урок 2). Настало время попробовать его в деле.

11.1 Как работает АЦП ESP32-S3

АЦП в ESP32-S3 12-битный: диапазон напряжений делится на $2^{12} = 4096$ ступенек, и результат — число от 0 до 4095:

$$N \approx \frac{U_{\text{ВХ}}}{U_{\text{макс}}} \cdot 4095.$$

Он работает методом **последовательного приближения**, как в игре «угадай число»: «больше половины? — да; больше трёх четвертей? — нет; ...» — 12 вопросов, и число найдено.

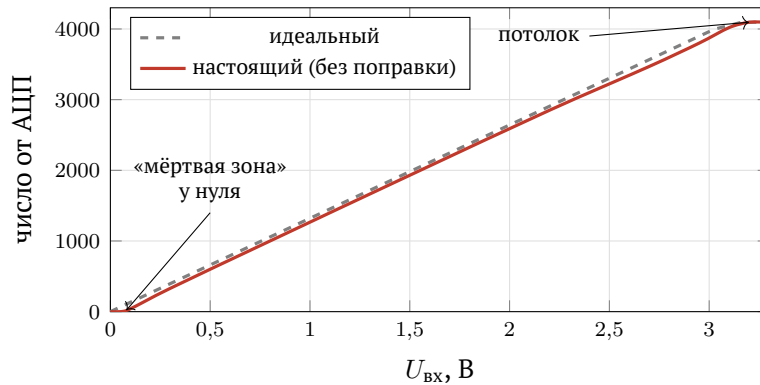


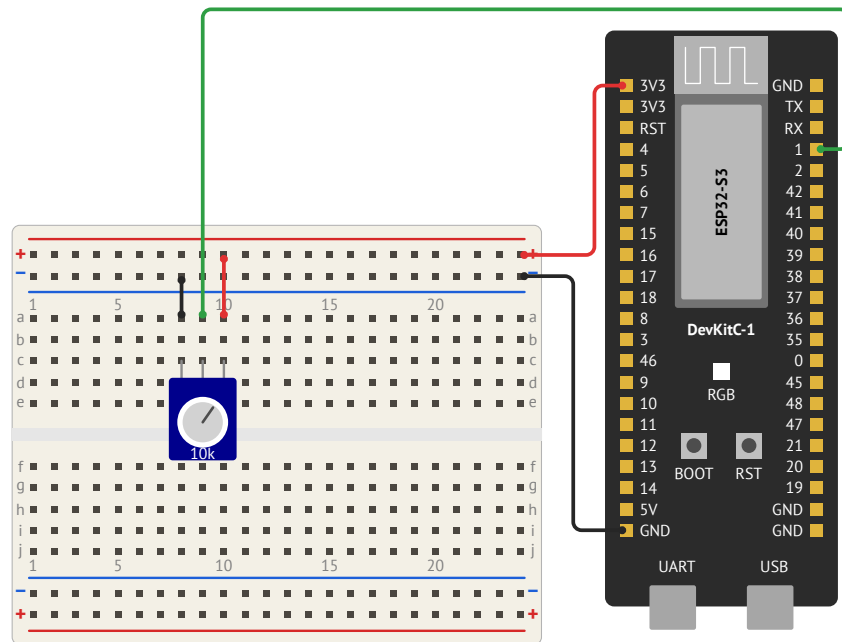
Рис. 78: Как число от АЦП зависит от напряжения (иллюстрация). Крайя диапазона неточные — поэтому удобнее функция с заводской поправкой

Особенности, которые нужно помнить:

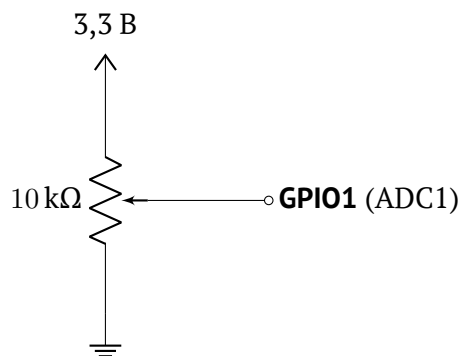
- Измерять можно от 0 до $\approx 3,1$ В. Больше 3,3 В подавать **нельзя!**
- **ADC2 не работает вместе с Wi-Fi** — поэтому берём выводы ADC1: **GPIO1–GPIO10**.
- Каждый чип на заводе измерили и записали поправки. Функция `analogReadMilliVolts()` учитывает их и сразу возвращает милливольты. Ею и будем пользоваться.

11.2 Потенциометр — «крутилка»

Потенциометр — это резистор с третьим, подвижным контактом (движком). Крайние выводы подключают к 0 и 3,3 В, а движок, скользя по резистивной дорожке, выдаёт любое напряжение между ними. Поворот ручки — самый наглядный аналоговый сигнал.



а) Монтажная схема: как собрать на макетной плате



б) Принципиальная электрическая схема

Рис. 79: Потенциометр на выводе GPIO1

Листинг 28: Чтение потенциометра

```

const int POT_PIN = 1;

void setup() {
  Serial.begin(115200);
  analogReadResolution(12); // 0..4095
}

void loop() {
  int raw = analogRead(POT_PIN);
  int mv = analogReadMilliVolts(POT_PIN); // с учётом калибровки
  Serial.printf("raw:%d mV:%d\n", raw, mv); // формат для Serial Plotter
  delay(100);
}

```

Совет

Открой Tools → Serial Plotter: пары «имя:значение» Arduino IDE нарисует графиками. Покрути ручку — увидишь свой аналоговый сигнал вживую!

11.3 Шум и усреднение

Даже если ручку не трогать, числа немного «прыгают» — это **шум**: помехи из урока 2. Самый простой способ с ним бороться — **усреднение**: измерить несколько раз подряд и взять среднее. Случайные отклонения вверх и вниз компенсируют друг друга.

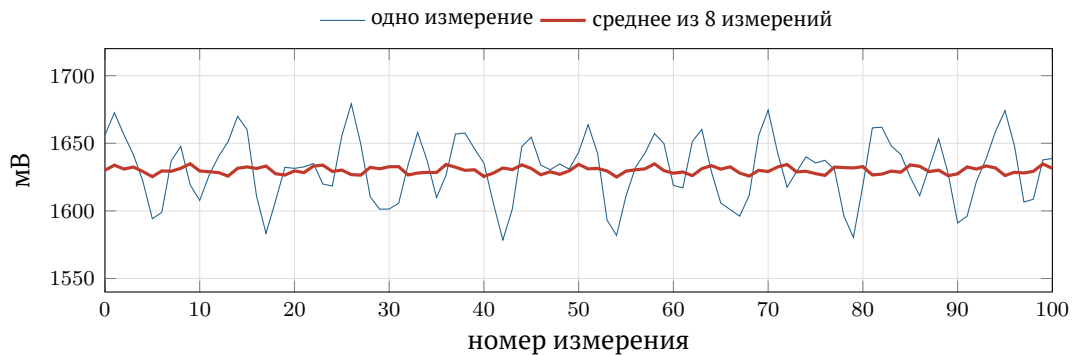


Рис. 80: Ручка стоит на месте, а измерения «дрожат». Среднее из нескольких измерений намного спокойнее

11.4 Сервопривод

Сервопривод — моторчик с редуктором и «мозгами», который поворачивается на заданный угол (0–180°) и держит его. Такие стоят в радиоуправляемых машинах и роботах. Угол задают ШИМ-сигналом частотой 50 Гц: *длительность* импульса говорит, куда повернуться.

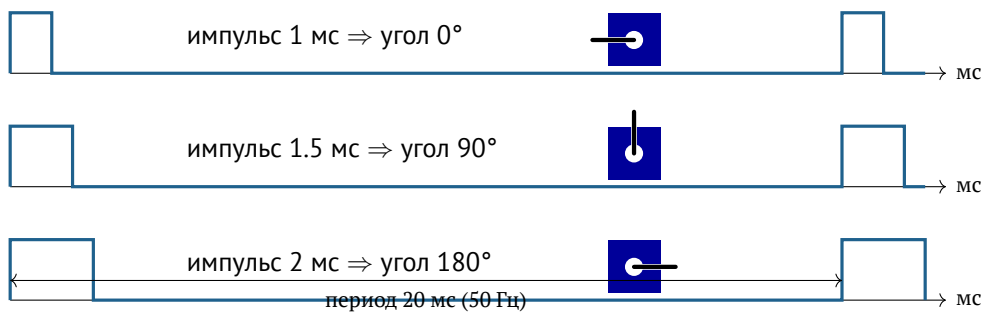


Рис. 81: Управление сервоприводом: чем длиннее импульс, тем больше угол

Листинг 29: Сервопривод качается туда-обратно

```
const int SERVO_PIN = 6;
const uint8_t RES = 14; // 0..16383 на период 20 мс

uint32_t usToDuty(uint32_t us) { // микросекунды -> значение duty
    return (uint32_t)((uint64_t)us * ((1 << RES) - 1) / 20000);
}

void servoWrite(int angle) { // 0..180 градусов
    uint32_t us = map(angle, 0, 180, 1000, 2000); // у многих серво 500..2500
    ledcWrite(SERVO_PIN, usToDuty(us));
}

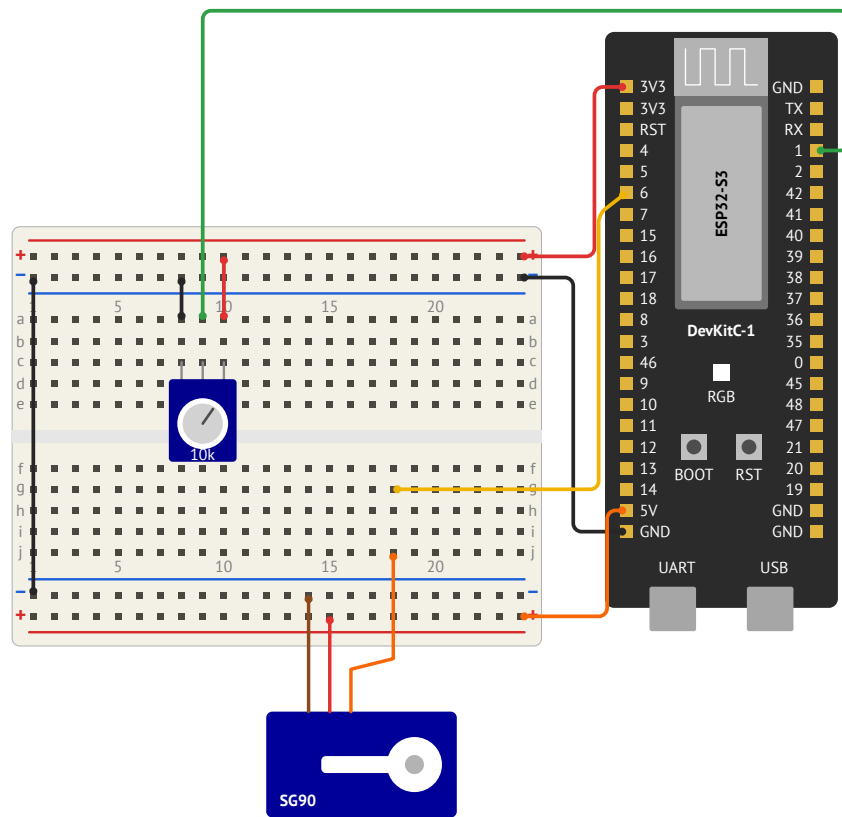
void setup() {
    ledcAttach(SERVO_PIN, 50, RES);
}
```

```
void loop() {
  for (int a = 0; a <= 180; a += 2) { servoWrite(a); delay(20); }
  for (int a = 180; a >= 0; a -= 2) { servoWrite(a); delay(20); }
}
```

11.5 Практика: ручка поворачивает сервопривод

Соединим «крутилку» и сервопривод: повернул ручку — повернулся моторчик. Так устроены многие манипуляторы роботов.

У сервопривода три провода: **коричневый** — земля, **красный** — питание 5 В, **оранжевый** — сигнал. Питание сервопривода возьмём с вывода 5V платы, поэтому на макетной плате у нас будут *две разные шины «+»*: сверху 3,3 В для потенциометра, снизу 5 В для мотора. Будь внимателен и не перепутай их! Шины «-» соединены перемычкой слева.



а) Монтажная схема: как собрать на макетной плате



б) Принципиальная электрическая схема

Рис. 82: Потенциометр управляет сервоприводом

Листинг 30: Ручка поворачивает сервопривод

```
// Ручка-потенциометр поворачивает сервопривод
const int POT_PIN = 1; // ADC1
const int SERVO_PIN = 6;
const uint8_t RES = 14; // 0..16383 на период 20 мс

void servoWrite(int angle) { // 0..180 градусов
    uint32_t us = map(angle, 0, 180, 1000, 2000); // длина импульса, мкс
    ledcWrite(SERVO_PIN, us * ((1 << RES) - 1) / 20000);
}

int readPot() { // среднее из 8 измерений, мВ
    uint32_t sum = 0;
    for (int i = 0; i < 8; i++) sum += analogReadMilliVolts(POT_PIN);
    return sum / 8;
}

int lastAngle = -100;

void setup() {
    Serial.begin(115200);
    ledcAttach(SERVO_PIN, 50, RES); // 50 Гц -- как любит серво
}

void loop() {
    int angle = constrain(map(readPot(), 0, 3100, 0, 180), 0, 180);
    if (abs(angle - lastAngle) >= 2) { // не дёргаемся из-за шума
        lastAngle = angle;
        servoWrite(angle);
        Serial.printf("угол: %d\n", angle);
    }
    delay(20);
}
```

Функция `map(x, 0, 3100, 0, 180)` пересчитывает число из одного диапазона в другой: 0 мВ → 0°, 3100 мВ → 180°. А проверка `abs(angle - lastAngle) >= 2` не даёт мотору дёргаться из-за шума.

Осторожно

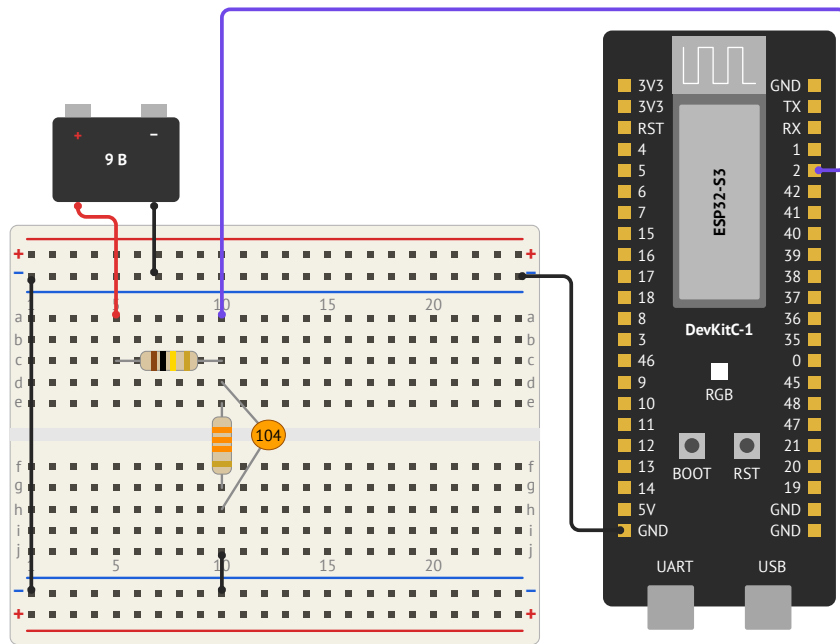
Под нагрузкой сервопривод потребляет сотни миллиампер. Если при его движении плата перезагружается, питай сервопривод от отдельного источника 5 В (например, 4 батарейки AA), соединив его «минус» с GND платы.

11.6 Измеряем напряжение больше 3,3 В

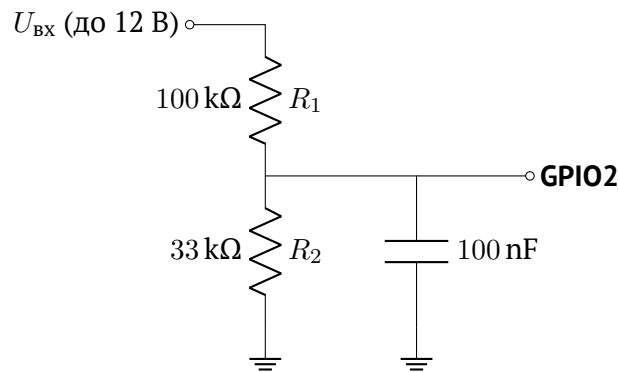
Как измерить, например, батарейку 9 В, если на вход можно подать максимум 3,3 В? Нужно уменьшить напряжение в известное число раз. Для этого есть **делитель напряжения** — два резистора подряд. Напряжение делится между ними пропорционально сопротивлениям:

$$U_{\text{АЦП}} = U_{\text{вх}} \cdot \frac{R_2}{R_1 + R_2} = 9 \cdot \frac{33}{100 + 33} \approx 2,23 \text{ В.}$$

С такими резисторами можно безопасно измерять до 12 В.



а) Монтажная схема: как собрать на макетной плате



б) Принципиальная электрическая схема

Рис. 83: Делитель напряжения для измерения батарейки 9 В на выводе GPIO2

Конденсатор 100 nF (маркировка «104») сглаживает помехи. **Важно:** «минус» батарейки обязательно соединяется с GND платы, иначе АЦП не с чем сравнивать напряжение.

Листинг 31: Вольтметр с усреднением

```
const int VIN_PIN = 2;
const float K = (100.0 + 33.0) / 33.0;    // коэффициент делителя

float readVoltage() {
    uint32_t sum = 0;
    for (int i = 0; i < 32; i++) sum += analogReadMilliVolts(VIN_PIN);
    return (sum / 32.0) / 1000.0 * K;    // вольты на входе делителя
}

void setup() { Serial.begin(115200); }

void loop() {
    Serial.printf("U = %.2f B\n", readVoltage());
    delay(500);
}
```

Итоги урока

- `analogReadMilliVolts()` даёт напряжение в милливольтках с заводской поправкой; диапазон — примерно 0–3,1 В.
- Для проектов с Wi-Fi аналоговые датчики подключаем к ADC1 (**GPIO1–GPIO10**).
- Шум убираем усреднением, а дрожание управления — порогом.
- Сервопривод управляется ШИМ 50 Гц: длительность импульса 1–2 мс задаёт угол 0–180°.
- Большие напряжения измеряем через делитель: $U_{\text{АЦП}} = U_{\text{ВХ}} \cdot R_2 / (R_1 + R_2)$.

Проверь себя

1. Какое число выдаст 12-битный АЦП на половине диапазона?
2. Почему в проекте с Wi-Fi нельзя подключить датчик к **GPIO15**?
3. Какое напряжение будет на АЦП, если через делитель 100/33 кΩ измерять 5 В?

Задания для самостоятельной работы

1. Управляй ручкой яркостью светодиода на **GPIO4** (не забудь гамма-коррекцию из урока 6).
2. Подключи фоторезистор вместо потенциометра и сделай ночник, который включается в темноте.
3. Сравни показания своего вольтметра и настоящего мультиметра в пяти точках. Построй график ошибки.
4. Сделай «шлагбаум»: при нажатии кнопки сервопривод поднимается на 90°, через 3 секунды опускается.

Урок 12. Шина I²C: экран

Цель	разобраться, как по двум проводам общаются несколько устройств, и вывести информацию на OLED-экран
Опирается на	урок 2 (цифровой сигнал), урок 10 (подтяжка), урок 11 (потенциометр)
Практика	вольтметр с экраном: напряжение крупными цифрами и полоска-шкала
Понадобится	OLED-дисплей SSD1306 128×64 с интерфейсом I ² C, потенциометр 10 кΩ

12.1 Два провода на всех

Экрану нужно передать много данных — целую картинку. Можно было бы использовать UART, но у него есть недостаток: он соединяет только *два* устройства. А если к плате нужно подключить экран, датчик температуры и часы? Для этого придумали шину I²C (читается «ай-ту-си», от англ. *Inter-Integrated Circuit*).

Всего два провода:

- **SDA** (*Serial Data*) — по нему идут данные;
- **SCL** (*Serial Clock*) — «метроном»: ведущее устройство отбивает такт, и все знают, когда читать очередной бит. Поэтому, в отличие от UART, договариваться о скорости заранее не нужно.

На шине одно **ведущее** устройство (наша плата) и несколько **ведомых**. У каждого ведомого свой **адрес** — число от 8 до 119. Ведущий начинает разговор словами «эй, устройство номер 0x3C!», и отвечает только то, чей это адрес. Как учитель, который вызывает учеников по фамилии.

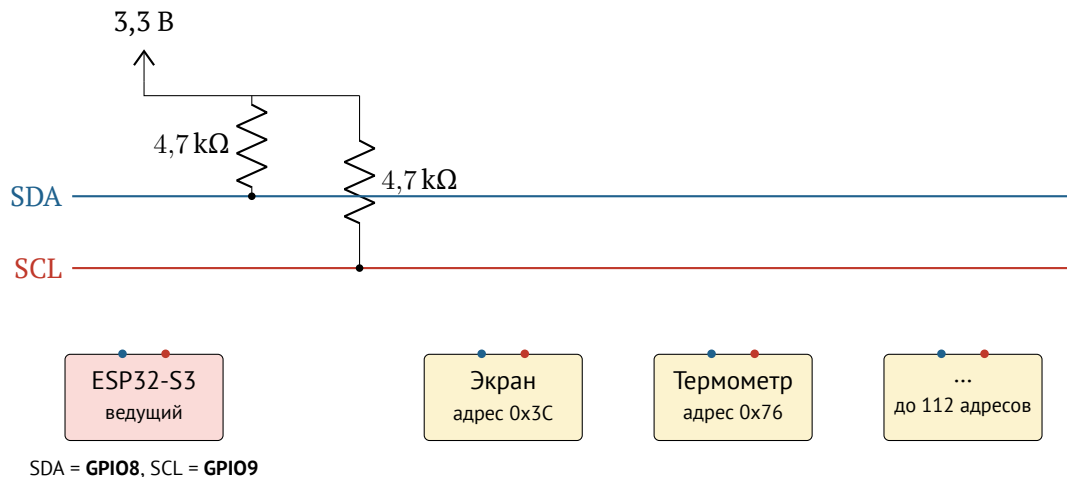


Рис. 84: Шина I²C: одна пара проводов, много устройств

Устройства на шине не выдают 1 — они умеют только притягивать линию к земле (0). А единицу создают **подтягивающие резисторы** — точно так же, как у кнопки из урока 10. Благодаря этому, даже если два устройства заговорят одновременно, ничего не сгорит. На модулях экранов и датчиков подтягивающие резисторы обычно уже распаяны.

12.2 Как выглядит разговор по I²C

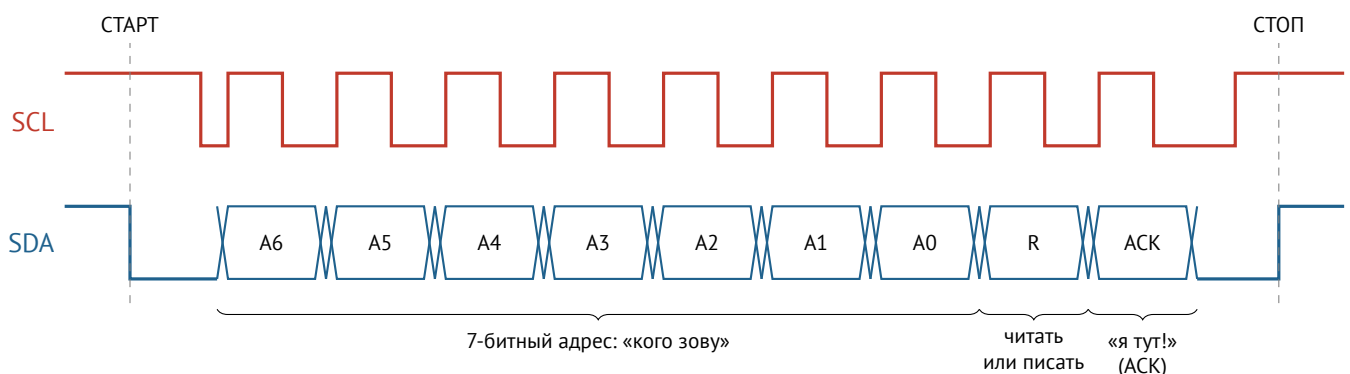
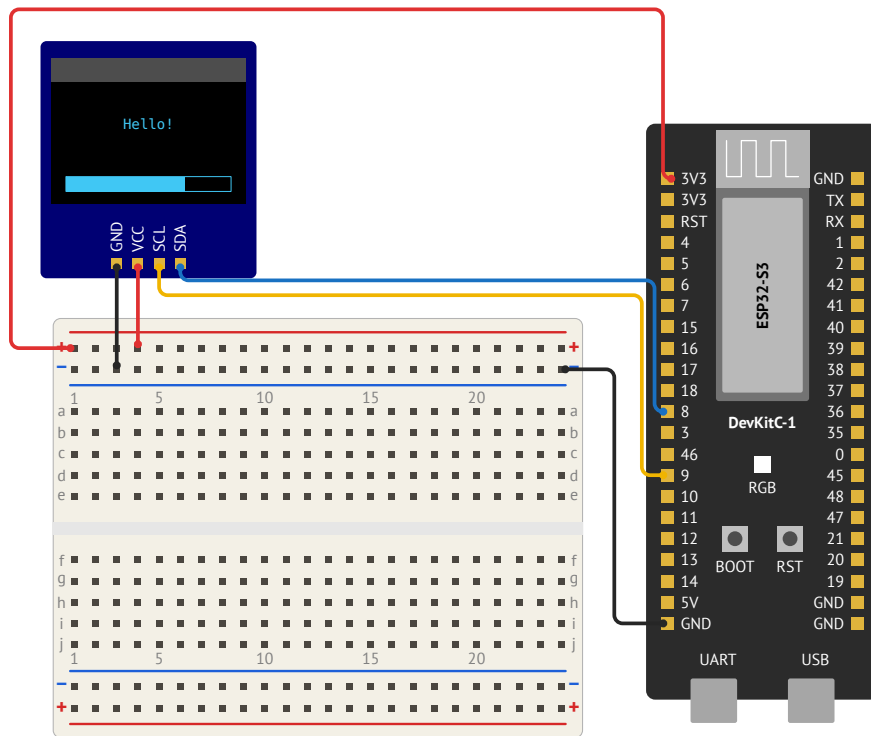


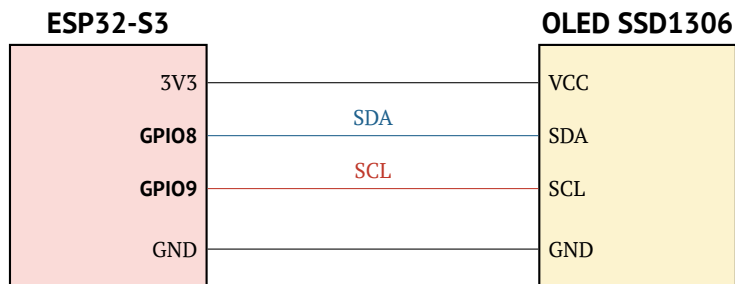
Рис. 85: Начало разговора по I²C. Данные на SDA меняются, пока SCL = 0, и читаются, пока SCL = 1. Ведомое устройство подтверждает, что услышало, — притягивает SDA к 0 (ACK)

12.3 Подключаем экран

OLED-экран состоит из $128 \times 64 = 8192$ крошечных светящихся точек — **пикселей**. Подключим его к плате: SDA → **GPIO8**, SCL → **GPIO9**, питание — от 3,3 В. Экран удобно разместить над макетной платой и соединить проводами «мама-папа».



а) Монтажная схема: как собрать на макетной плате



б) Принципиальная электрическая схема

Рис. 86: Подключение OLED-экрана к шине I²C**Осторожно**

Порядок выводов у разных экранов бывает разным: GND–VCC–SCL–SDA или VCC–GND–SCL–SDA. Перед подключением прочитай подписи на своём модуле!

12.4 Сканер шины

Первое, что стоит сделать с новым модулем, — проверить, отвечает ли он, и узнать его адрес. Программа-сканер по очереди «вызывает» все адреса и сообщает, кто откликнулся:

Листинг 32: Сканер I²C

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  delay(500);
  Wire.begin(8, 9);           // SDA, SCL
  Serial.println("Сканирование...");
}
```

```

int found = 0;
for (uint8_t addr = 0x08; addr < 0x78; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) { // 0 = устройство ответило ACK
        Serial.printf(" найдено устройство: 0x%02X\n", addr);
        found++;
    }
}
Serial.printf("Всего: %d\n", found);
}

void loop() {}

```

Для экрана SSD1306 обычно увидишь адрес 0x3C. Если не нашлось ничего — проверь провода и не перепутаны ли SDA и SCL.

12.5 Первая надпись на экране

Установи через Tools → Manage Libraries библиотеки **Adafruit SSD1306** и **Adafruit GFX**. Библиотека рисует картинку в памяти платы, а команда `display()` отправляет её на экран целиком.

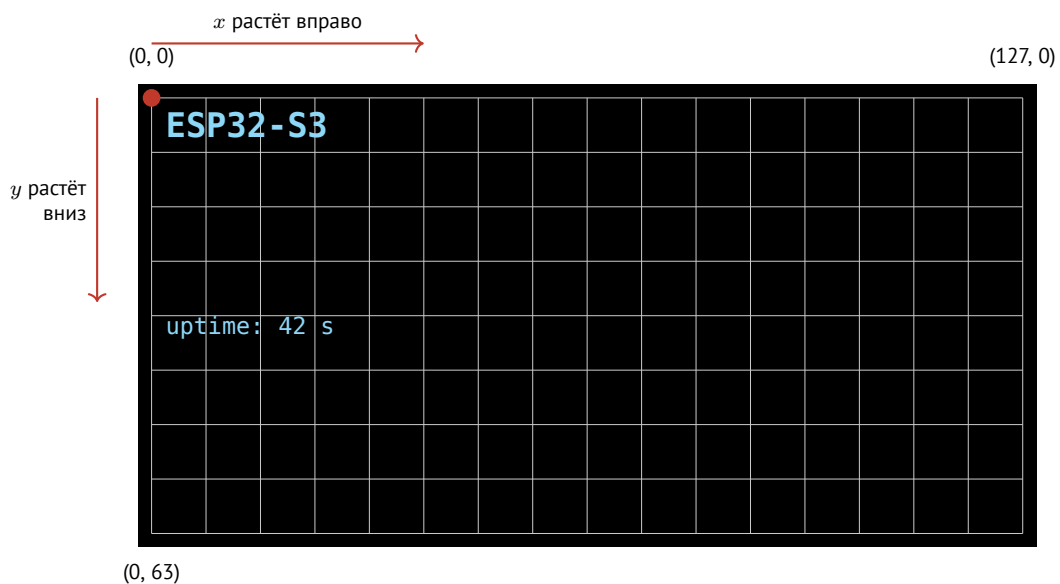


Рис. 87: Координаты пикселей экрана: начало — в левом верхнем углу, и *y* растёт вниз

Листинг 33: Надпись на OLED-экране

```

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1); // -1: без вывода сброса

void setup() {
    Serial.begin(115200);
    Wire.begin(8, 9);
    Wire.setClock(400000); // Fast mode, 400 кГц
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {

```

```
    Serial.println("Дисплей не найден");  
    while (true) delay(1000);  
}  
display.setTextColor(SSD1306_WHITE);  
}  
  
void loop() {  
    display.clearDisplay();           // очистить буфер  
    display.setTextSize(2);  
    display.setCursor(0, 0);  
    display.print("ESP32-S3");  
    display.setTextSize(1);  
    display.setCursor(0, 30);  
    display.printf("uptime: %lu s", millis() / 1000);  
    display.display();               // отправить буфер на экран  
    delay(200);  
}
```

Совет

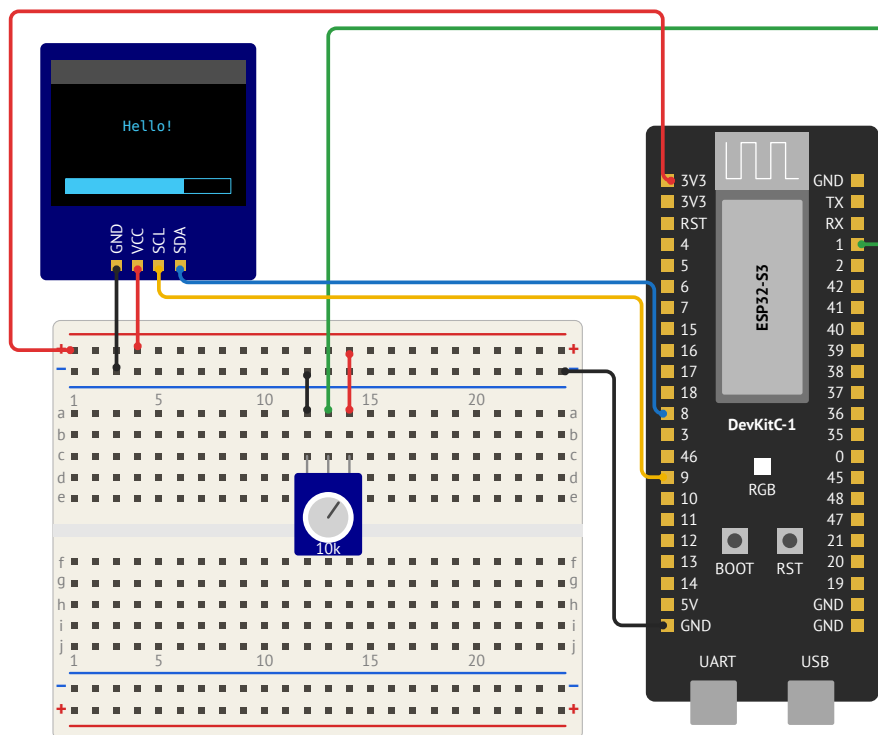
Стандартные шрифты Adafruit GFX не содержат русских букв — для кириллицы нужны специальные шрифты или библиотека U8g2. В примерах обойдёмся латиницей.

12.6 Практика: вольтметр с экраном

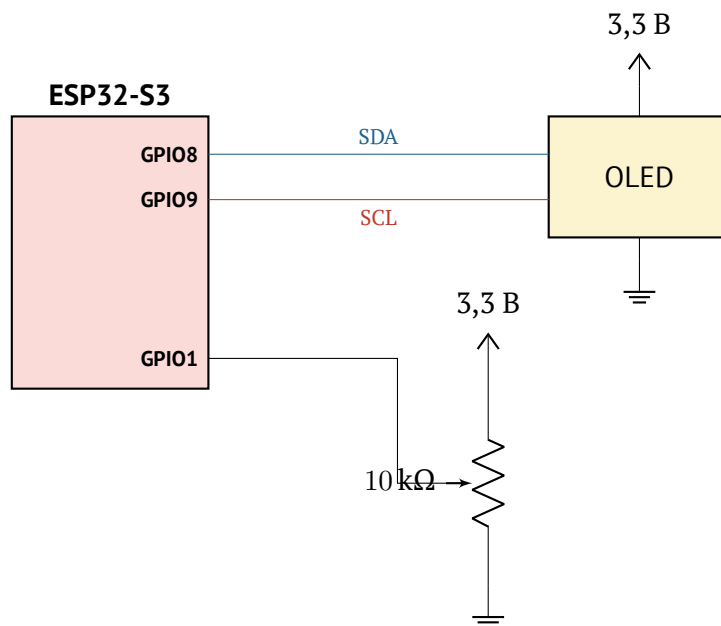
Соединим экран с потенциометром из урока 11: на экране будет напряжение на движке крупными цифрами и полоска-шкала, как у настоящего прибора.



Рис. 88: Экран вольтметра: заголовок, напряжение крупно и полоска



а) Монтажная схема: как собрать на макетной плате



б) Принципиальная электрическая схема

Рис. 89: Вольтметр: потенциометр на GPIO1 и экран на I²C

Листинг 34: Вольтметр с экраном

```
// Вольтметр с экраном: напряжение на ручке крупно и полоска-шкала
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

const int POT_PIN = 1;
```

```

Adafruit_SSD1306 display(128, 64, &Wire, -1);

int readMilliVolts() {
    uint32_t sum = 0;
    for (int i = 0; i < 16; i++) sum += analogReadMilliVolts(POT_PIN);
    return sum / 16;
}

void setup() {
    Serial.begin(115200);
    Wire.begin(8, 9); // SDA, SCL
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println("Дисплей не найден");
        while (true) delay(1000);
    }
    display.setTextColor(SSD1306_WHITE);
}

void loop() {
    int mv = readMilliVolts();
    display.clearDisplay();
    display.setTextSize(1);
    display.setCursor(0, 0);
    display.print("Voltmeter");
    display.setTextSize(3);
    display.setCursor(0, 18);
    display.printf("%.2fV", mv / 1000.0);
    display.drawRect(0, 54, 128, 10, SSD1306_WHITE); // рамка шкалы
    display.fillRect(2, 56, mv * 124 / 3300, 6, SSD1306_WHITE);
    display.display(); // отправить картинку на экран
    delay(100); // 10 кадров в секунду хватит
}

```

Экран обновляется 10 раз в секунду — чаще не нужно: глаз всё равно не заметит разницы (вспомни инерцию зрения из урока 6), а отправка картинки по шине занимает время.

Итоги урока

- I²C — два провода (SDA и SCL), подтягивающие резисторы и адреса устройств.
- С новым модулем сначала запускаем сканер и узнаём адрес.
- Библиотека экрана рисует в памяти, `display()` отправляет картинку целиком.
- Координаты пикселей начинаются в левом верхнем углу, *y* растёт вниз.

Проверь себя

1. Сколько проводов нужно, чтобы подключить к плате три устройства I²C (не считая питания)?
2. Как ведущее устройство выбирает, с кем разговаривать?
3. В каком углу экрана пиксель с координатами (127, 63)?

Задания для самостоятельной работы

1. Нарисуй на экране смайлик с помощью `drawCircle()` и `fillCircle()`.
2. Сделай «осциллограф»: рисуй график напряжения на ручке за последние 128 измерений.
3. Покажи на экране IP-адрес платы из урока 7 и «записку», пришедшую через веб-страницу (урок 8).

Урок 13. FreeRTOS: несколько дел одновременно

Цель	разделить программу на независимые задачи, безопасно передавать данные между ними, задействовать оба ядра
Опирается на	урок 5 (<code>millis()</code> вместо <code>delay()</code>), урок 10 (прерывания), урок 11 (АЦП)
Практика	две задачи мигают светодиодами; две задачи по очереди «говорят» в Serial Monitor
Понадобится	схема из четырёх светодиодов (рис. 35), потенциометр на GPIO1 (рис. 79)

13.1 Зачем нужна операционная система

Чем больше умеет устройство, тем больше дел в `loop()`: проверить кнопку, измерить напряжение, обновить экран, ответить браузеру... Приходится следить, чтобы ни одно дело не задерживало остальные. Представь повара, который варит суп, жарит котлеты и печёт пирог: если он будет стоять над супом, котлеты сгорят.

FreeRTOS — это маленькая **операционная система реального времени**, встроенная в ESP32-S3. Она позволяет написать каждое дело как отдельную программу — **задачу** (task) — со своим бесконечным циклом. А специальный «диспетчер» — **планировщик** — сам переключает процессор между задачами тысячу раз в секунду. Кажется, будто всё выполняется одновременно. А у ESP32-S3 два ядра, поэтому две задачи действительно могут работать одновременно!

Это интересно

Ты уже пользовался FreeRTOS, не зная об этом: `setup()` и `loop()` выполняются внутри задачи `loopTask` на ядре 1, Wi-Fi работает в своих задачах на ядре 0, а `delay()` — это команда FreeRTOS «я пока отдохну, отдай процессор другим».

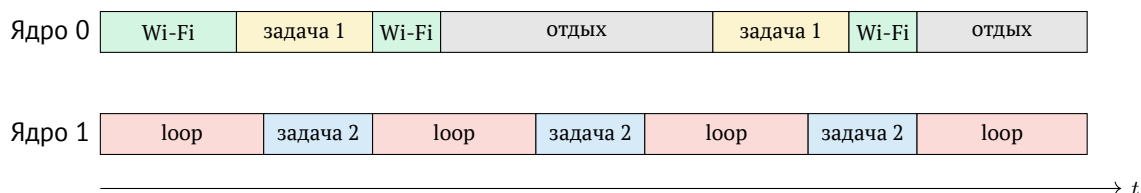


Рис. 90: Как задачи делят два ядра: пока одна отдыхает, процессор занят другой

13.2 Жизнь задачи

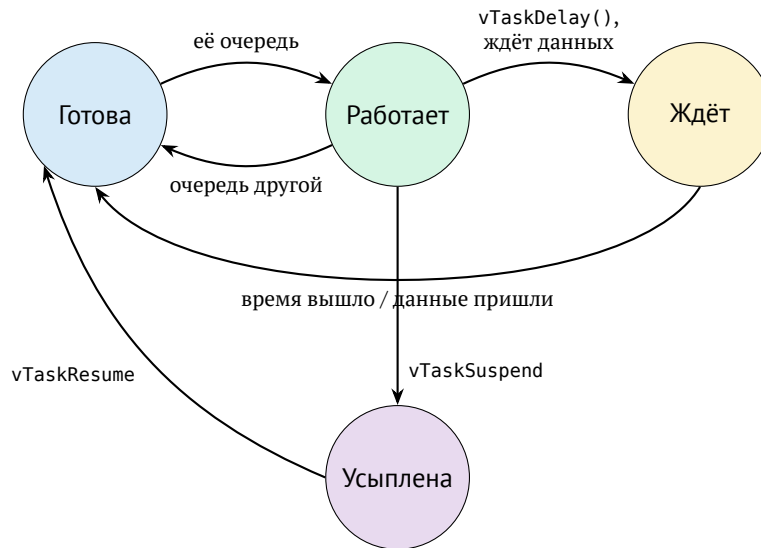


Рис. 91: Состояния задачи. Задача, которая ждёт, не тратит процессорное время

13.3 Первые задачи

Вспомни задание из урока 5: два светодиода мигают с разной частотой. С `millis()` это требовало аккуратности. С задачами всё проще: каждая задача мигает своим светодиодом и просто «спит» сколько нужно. Используй схему из четырёх светодиодов (рис. 35): нам понадобятся **GPIO4** и **GPIO7**.

Листинг 35: Две задачи мигают светодиодами на разных ядрах

```

void blinkTask(void* param) {
    int pin = (int)(intptr_t)param;           // номер вывода передан параметром
    int period = (pin == 4) ? 300 : 700;
    pinMode(pin, OUTPUT);
    for (;;) {                                // задача никогда не завершается сама
        digitalWrite(pin, !digitalRead(pin));
        vTaskDelay(pdMS_TO_TICKS(period));    // отдаём процессор другим задачам
    }
}

void setup() {
    Serial.begin(115200);
    //          функция   имя   стек   параметр   приор.   дескр.   ядро
    xTaskCreatePinnedToCore(blinkTask, "b1", 2048, (void*)(intptr_t)4, 1, NULL, 0);
    xTaskCreatePinnedToCore(blinkTask, "b2", 2048, (void*)(intptr_t)7, 1, NULL, 1);
}

void loop() {
    Serial.printf("loop() на ядре %d\n", xPortGetCoreID());
    delay(2000);
}

```

Функция `xTaskCreatePinnedToCore()` создаёт задачу. Её параметры: функция задачи, имя, размер стека (память для переменных задачи), параметр, приоритет (кто важнее), дескриптор и номер ядра.

13.4 Очередь: передаём данные между задачами

Задачи часто работают вместе: одна измеряет, другая показывает. Передавать данные удобно через **очередь** — как через ленту конвейера: одна задача кладёт значения с одного конца, другая забирает с другого. Если лента пуста, забирающая задача спит и не тратит время.



Рис. 92: Очередь развязывает задачи: каждая работает в своём темпе

Листинг 36: Производитель и потребитель (потенциометр на GPIO1)

```
QueueHandle_t q;

void sensorTask(void*) {
    for (;;) {
        int mv = analogReadMilliVolts(1);
        xQueueSend(q, &mv, 0); // не ждать, если очередь полна
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void printTask(void*) {
    int mv;
    for (;;) {
        if (xQueueReceive(q, &mv, portMAX_DELAY) == pdTRUE) { // спим до прихода данных
            Serial.printf("U = %d mB\n", mv);
        }
    }
}

void setup() {
    Serial.begin(115200);
    q = xQueueCreate(5, sizeof(int));
    xTaskCreatePinnedToCore(sensorTask, "sensor", 3072, NULL, 2, NULL, 1);
    xTaskCreatePinnedToCore(printTask, "print", 4096, NULL, 1, NULL, 1);
}

void loop() { vTaskDelete(NULL); } // loopTask больше не нужен
```

13.5 Практика: мьютекс — «ключ» от общего ресурса

Когда две задачи пользуются одной вещью — например, Serial, — они могут мешать друг другу. Проверим! Две задачи печатают свои фразы по одной букве. Сначала поставь в программе `USE_MUTEX = false`:

Листинг 37: Две задачи и мьютекс

```
// Две задачи печатают в Serial. Мьютекс не даёт им перебивать друг друга
SemaphoreHandle_t serialMutex; // «ключ» от Serial
const bool USE_MUTEX = true; // поставь false и сравни!

void say(const char* text) {
    if (USE_MUTEX) xSemaphoreTake(serialMutex, portMAX_DELAY); // взять ключ
```

```

    for (const char* c = text; *c; c++) {           // печатаем по одной букве,
        Serial.print(*c);                          // как будто это долгая работа
        delayMicroseconds(300);
    }
    Serial.println();
    if (USE_MUTEX) xSemaphoreGive(serialMutex);     // вернуть ключ
}

void catTask(void*) {
    for (;;) { say("Кошка говорит: мяу-мяу-мяу"); vTaskDelay(pdMS_TO_TICKS(7)); }
}

void dogTask(void*) {
    for (;;) { say("Собака говорит: гав-гав-гав"); vTaskDelay(pdMS_TO_TICKS(5)); }
}

void setup() {
    Serial.begin(115200);
    serialMutex = xSemaphoreCreateMutex();          // создать до запуска задач!
    xTaskCreatePinnedToCore(catTask, "cat", 3072, NULL, 1, NULL, 0);
    xTaskCreatePinnedToCore(dogTask, "dog", 3072, NULL, 1, NULL, 1);
}

void loop() { vTaskDelete(NULL); }                 // loop() нам не нужен

```

Без мьютекса в Serial Monitor получится каша: «Кошка говСобака говорит: гаорит: мяу...». Задачи перебивают друг друга на полуслове. Теперь поставь true: фразы идут ровно, по очереди.

Мьютекс (от англ. *mutual exclusion* — взаимное исключение) работает как ключ от единственной комнаты: задача берёт ключ (xSemaphoreTake), делает свою работу и возвращает ключ (xSemaphoreGive). Пока ключ у одной задачи, другая ждёт.



Рис. 93: Мьютекс не даёт задачам перебивать друг друга

Инструмент FreeRTOS	Когда пригодится
Задача (xTaskCreate)	отдельное дело со своим циклом
Очередь (xQueue...)	передать данные из одной задачи в другую (или из прерывания)
Мьютекс (xSemaphoreCreateMutex)	не дать двум задачам одновременно пользоваться Serial, экраном, общей переменной
Уведомление (xTaskNotify)	разбудить конкретную задачу: «событие произошло»
Программный таймер (xTimerCreate)	вызывать короткую функцию периодически

Таблица 21: Главные инструменты FreeRTOS

Осторожно

Если задаче не хватает памяти стека, плата перезагружается с сообщением Stack canary watchpoint triggered. Тогда увеличь третий параметр xTaskCreatePinnedToCore() — например, с 2048 до 4096.

Итоги урока

- Задача — функция с бесконечным циклом; `vTaskDelay()` отдаёт процессор другим.
- Данные между задачами передаём через очередь.
- Общий ресурс (Serial, экран) защищаем мьютексом, а ещё лучше — закрепляем за одной задачей.

Проверь себя

1. Чем задача, которая ждёт в `vTaskDelay()`, отличается от программы в `delay()` без FreeRTOS?
2. Для чего нужна очередь?
3. Что произойдёт, если задача взяла мьютекс и забыла его вернуть?

Задания для самостоятельной работы

1. Запусти четыре задачи — по одной на каждый светодиод «бегущего огня», с разными периодами.
2. Добавь в программу с очередью третью задачу, которая зажигает светодиод, если напряжение больше 2 В.
3. Переведи кнопку на прерывание (урок 10): обработчик будит задачу через `vTaskNotifyGiveFromISR()`.

Урок 14. Сон и энергосбережение

Цель	разобраться в режимах сна ESP32-S3, научиться будить чип таймером и кнопкой, посчитать, сколько проработает батарейка
Опирается на	урок 10 (кнопка), урок 11 (измерение напряжения), урок 7 (Wi-Fi как главный потребитель)
Практика	«датчик на батарейке»: просыпается раз в 30 секунд или от кнопки, измеряет и снова засыпает
Понадобится	кнопка и светодиод (рис. 73), потенциометр (рис. 79), мультиметр (по желанию)

14.1 Зачем устройству спать

Метеостанция на даче, датчик протечки под ванной, умный ошейник для кота — все они работают от батареек месяцами. Секрет прост: почти всё время они **спят**, а просыпаются на секунду — измерить, отправить данные — и снова засыпают. Работающая ESP32-S3 с Wi-Fi потребляет в десятки тысяч раз больше, чем спящая.

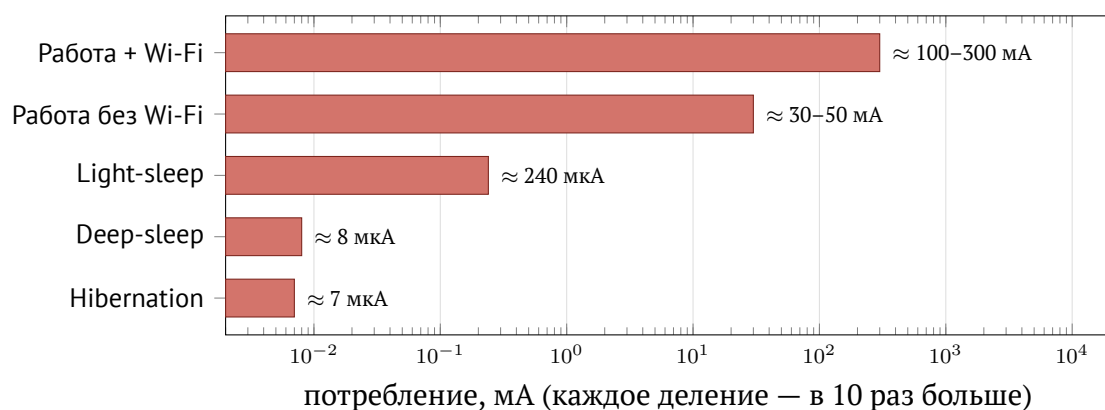


Рис. 94: Сколько потребляет сам чип в разных режимах (примерно, по документации). Отладочная плата с преобразователем питания и светодиодом питания потребляет заметно больше

Режим	Что работает	Что при пробуждении
Light-sleep (лёгкий сон)	всё «заморожено», память сохранена	программа продолжается со следующей строки
Deep-sleep (глубокий сон)	только часы и маленькая память RTC	перезапуск: <code>setup()</code> начинается заново
Hibernation (спячка)	только часы	перезапуск

Главное о глубоком сне: при пробуждении программа начинается сначала, а обычные переменные забываются. Сохранить данные можно в маленькой памяти RTC — для этого переменную помечают `RTC_DATA_ATTR`.

14.2 Сколько проработает батарейка

Типичный «датчик на батарейке» живёт по такому расписанию:

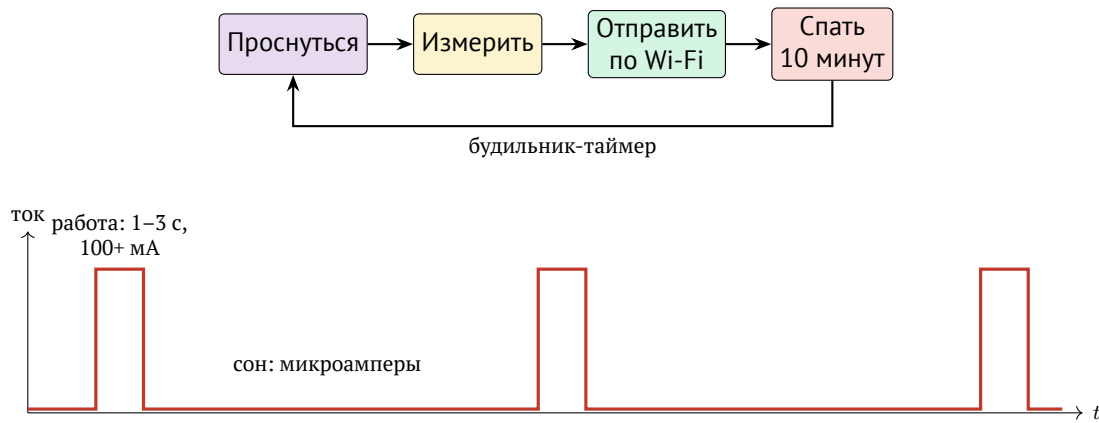


Рис. 95: Устройство почти всё время спит. Похоже на ШИМ из урока 6 с очень маленьким коэффициентом заполнения!

Средний ток считается так же, как среднее напряжение ШИМ:

$$I_{\text{ср}} = \frac{I_{\text{работа}} \cdot t_{\text{работа}} + I_{\text{сон}} \cdot t_{\text{сон}}}{t_{\text{работа}} + t_{\text{сон}}} = \frac{120 \text{ мА} \cdot 2 \text{ с} + 0,01 \text{ мА} \cdot 598 \text{ с}}{600 \text{ с}} \approx 0,41 \text{ мА}.$$

Аккумулятор ёмкостью 2000 мА·ч прослужит $2000/0,41 \approx 4900$ часов — около **полугода**. А без сна, при 120 мА, — меньше суток!

14.3 Сон по будильнику-таймеру

Листинг 38: Пробуждение по таймеру со счётчиком в RTC-памяти

```
#include "esp_sleep.h"

RTC_DATA_ATTR int bootCount = 0;           // переживает deep-sleep

const uint64_t SLEEP_US = 10ULL * 1000000; // 10 секунд

void setup() {
    Serial.begin(115200);
    delay(500);
    bootCount++;
    Serial.printf("Пробуждение №%d, причина: %d\n",
                  bootCount, esp_sleep_get_wakeup_cause());

    // ... здесь измерения и отправка данных ...

    esp_sleep_enable_timer_wakeup(SLEEP_US);
    Serial.println("Засыпаю");
    Serial.flush();
    esp_deep_sleep_start();                 // сюда программа уже не вернётся
}

void loop() {}                             // не вызывается
```

Осторожно

При подключении через разъём USB плата во сне «исчезает» из компьютера, и Serial Monitor отключается. Это нормально. Если новая прошивка не загружается, войди в режим прошивки вручную (BOOT + RST, урок 4).

14.4 Пробуждение кнопкой

Разбудить чип может и уровень на выводе (подходят **GPIO0–GPIO21**). Кнопка на **GPIO5** подходит. Во сне обычная подтяжка из урока 10 отключается, поэтому включаем специальную подтяжку, которая работает и во сне:

```
#include "driver/rtc_io.h"                // функции rtc_gpio_*

rtc_gpio_pullup_en(GPIO_NUM_5);          // подтяжка, работающая во сне
rtc_gpio_pulldown_dis(GPIO_NUM_5);
esp_sleep_enable_ext0_wakeup(GPIO_NUM_5, 0); // проснуться при LOW (кнопка нажата)
esp_deep_sleep_start();
```

14.5 Практика: датчик на батарее

Соберём всё вместе. Используем кнопку и светодиод (рис. 73) и потенциометр вместо «настоящего» датчика (рис. 79). Плата просыпается раз в 30 секунд или от кнопки, измеряет напряжение, коротко мигает светодиодом и снова засыпает. Счётчики пробуждений хранятся в RTC-памяти.

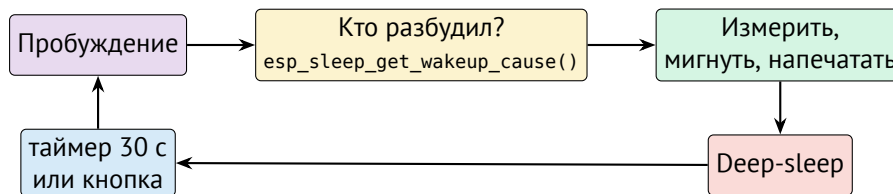


Рис. 96: Жизненный цикл «датчика на батарее»

Листинг 39: Датчик на батарее

```
// «Датчик на батарее»: просыпается раз в 30 с или от кнопки,
// измеряет напряжение, мигает светодиодом и снова засыпает
#include "driver/rtc_io.h"

const int LED_PIN = 4;
const int BTN_PIN = 5;
const int POT_PIN = 1;                // «датчик» -- потенциометр
const uint64_t SLEEP_US = 30ULL * 1000000; // 30 секунд

RTC_DATA_ATTR int wakeCount = 0;      // эти переменные
RTC_DATA_ATTR int buttonCount = 0;    // переживают сон

void setup() {
  Serial.begin(115200);
  delay(300);
  rtc_gpio_deinit((gpio_num_t)BTN_PIN); // вернуть кнопку в обычный режим
  pinMode(LED_PIN, OUTPUT);
  wakeCount++;

  if (esp_sleep_get_wakeup_cause() == ESP_SLEEP_WAKEUP_EXT0) {
    buttonCount++; // разбудили кнопкой
    Serial.println("Меня разбудила кнопка!");
  }

  int mv = analogReadMilliVolts(POT_PIN);
  Serial.printf("Пробуждение %d (кнопкой: %d), U = %d мВ\n", wakeCount, buttonCount,
  ↪ mv);
```

```
digitalWrite(LED_PIN, HIGH);           // короткая вспышка: «я работаю»
delay(100);
digitalWrite(LED_PIN, LOW);

esp_sleep_enable_timer_wakeup(SLEEP_US); // будильник 1: таймер
rtc_gpio_pullup_en((gpio_num_t)BTN_PIN); // будильник 2: кнопка
rtc_gpio_pulldown_dis((gpio_num_t)BTN_PIN);
esp_sleep_enable_ext0_wakeup((gpio_num_t)BTN_PIN, 0);
Serial.println("Сплю...");
Serial.flush();
esp_deep_sleep_start();
}

void loop() {}                          // сюда программа не доходит
```

Итоги урока

- Глубокий сон снижает ток до микроампер, но пробуждение — это перезапуск программы.
- Нужные данные храним в RTC_DATA_ATTR, причину пробуждения узнаём через esp_sleep_get_wakeup_cause().
- Будить можно таймером, кнопкой, сенсорным входом.
- Время жизни от батареи определяется долей времени, которую устройство не спит.

Проверь себя

1. Что произойдёт с обычной переменной после глубокого сна?
2. Во сколько раз работающая с Wi-Fi плата потребляет больше спящей?
3. Как изменится время работы батарейки, если просыпаться раз в минуту, а не раз в 10 минут?

Задания для самостоятельной работы

1. Измерь ток платы мультиметром в работе и во сне.
2. Добавь отправку измерения на веб-сервер или в Serial по Wi-Fi и посчитай, сколько проработает батарейка.
3. Показывай число пробуждений на OLED-экране (урок 12) и гаси экран перед сном.

Приложение А. Справочник

Выводы, с которыми нужно быть осторожным

Выводы	Особенность
GPIO0, 3, 45, 46	«загрузочные»: влияют на запуск; GPIO0 — кнопка BOOT
GPIO19, 20	USB (D−, D+)
GPIO26–32	память программ — не использовать
GPIO35–37	заняты памятью PSRAM на модулях с Octal PSRAM (N8R8, N16R8)
GPIO43, 44	UART0 (TX, RX) — сообщения при загрузке
GPIO11–20	ADC2 — не работает при включённом Wi-Fi
GPIO22–25	не существуют

Шпаргалка по языку C++

Запись	Смысл
<code>int x = 5;</code>	переменная целого типа со значением 5
<code>const int PIN = 4;</code>	константа: значение не меняется
<code>x = x + 1; x++; x += 1;</code>	три способа увеличить x на 1
<code>a / b, a % b</code>	деление (для целых — нацело) и остаток
<code>== != < > <= >=</code>	сравнение
<code>&& !</code>	логические «и», «или», «не»
<code>if (усл) { ... } else { ... }</code>	выполнить одну из двух веток
<code>for (int i = 0; i < N; i++) { ... }</code>	повторить N раз, i = 0, 1, ..., N − 1
<code>while (усл) { ... }</code>	повторять, пока условие верно; break — выйти
<code>int a[] = {1, 2, 3}; a[0]</code>	массив и его первый элемент
<code>int sum(int a, int b) { return a + b; }</code>	функция с параметрами и результатом
<code>void hello() { ... }</code>	функция без результата
<code>// текст</code>	комментарий

Шпаргалка по функциям Arduino-ESP32

Функция	Что делает	Урок
<code>pinMode(pin, OUTPUT / INPUT_PULLUP)</code>	настроить вывод	5, 10
<code>digitalWrite(pin, HIGH / LOW)</code>	выдать 1 или 0	5
<code>digitalRead(pin)</code>	прочитать 0 или 1	10
<code>millis(), delay(ms)</code>	время с момента включения; пауза	5
<code>ledcAttach(pin, freq, bits)</code>	включить ШИМ на выводе	6
<code>ledcWrite(pin, duty)</code>	задать коэффициент заполнения	6
<code>ledcWriteTone(pin, freq)</code>	звук заданной частоты	6
<code>WiFi.begin(ssid, pass)</code>	подключиться к сети	7
<code>WiFi.softAP(ssid, pass)</code>	создать свою сеть	7
<code>MDNS.begin("имя")</code>	имя <code>имя.local</code>	7
<code>server.on(путь, функция)</code>	обработчик адреса веб-сервера	8
<code>Serial.begin(), print(), readStringUntil()</code>	связь с компьютером	9
<code>attachInterrupt(pin, isr, FALLING)</code>	прерывание от вывода	10
<code>analogReadMilliVolts(pin)</code>	измерить напряжение, мВ	11
<code>Wire.begin(sda, scl)</code>	запустить шину I ² C	12
<code>xTaskCreatePinnedToCore(...)</code>	создать задачу FreeRTOS	13
<code>esp_deep_sleep_start()</code>	уйти в глубокий сон	14

Полезные формулы

Закон Ома	$I = U/R, \quad R = U/I$
Резистор для светодиода	$R = (U_{\text{пит}} - U_F)/I$
Частота и период	$f = 1/T$
Коэффициент заполнения ШИМ	$D = t_{\text{вкл}}/T, \quad U_{\text{ср}} = D \cdot U_{\text{пит}}$
Число вариантов в N битах	2^N
Шаг квантования АЦП	$q = U_{\text{макс}}/2^N$
Длина волны	$\lambda = c/f$
Делитель напряжения	$U_{\text{вых}} = U_{\text{вх}} \cdot R_2/(R_1 + R_2)$

Что изучать дальше

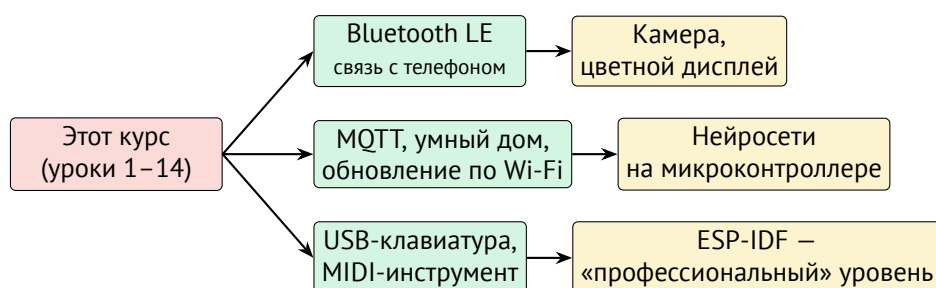


Рис. 97: Куда двигаться после этого курса

Полезные источники

- Документация на ESP32-S3 — <https://www.espressif.com/en/support/documents/technical-documents>
- Документация Arduino-ESP32 — <https://docs.espressif.com/projects/arduino-esp32/>
- Руководство ESP-IDF — <https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/>
- Описание платы — *ESP32-S3-DevKitC-1 User Guide* на <https://docs.espressif.com>
- Учебник по HTML и CSS — <https://developer.mozilla.org/ru/docs/Learn>